Braid Groups in Lean : Formalizing Dehornoy's Subword Reversing Algorithm

Hannah Fechtner
Carnegie Mellon University

Abstract

We formalize the group-theoretic definition of infinite and finite Artin braid groups in Lean. We implement a polynomial-time procedure for determining equivalence in the braid group known as Dehornoy's subword reversing algorithm. Termination and correctness of this algorithm are shown using rectangular grids, which are a specific form of van Kampen diagram. The literature only hints at a proof linking rectangular grids to braid equivalence. We present a novel, three-stage proof of this result, introducing partial grids and associated size metrics.

Contents

1 Introduction **3**
1.1 Mathematical Braids . . . 3
1.2 The Lean Theorem Prover . . . 5
1.3 Word Problem Algorithm . . . 6
1.4 Outline . . . 7

2 Defining Braids **8**
2.1 Artin-Tits Groups . . . 8
2.2 The Braid Group . . . 10
2.3 Presented Monoids . . . 11
2.4 Semi-Thue Systems . . . 12
2.5 The "Braid" Monoid . . . 13
2.6 Finite Braid Groups and Finite Braid Monoids . . . 16

3 Localization **17**
3.1 Ore Localization . . . 17

4 Common Multiples **23**

5 Grids **26**
5.1 Definition and Basic Properties . . . 26
5.2 Stability . . . 29
5.3 Cancellativity . . . 31
5.4 Grid Existence . . . 32

6 Grids and Rewriting **32**
6.1 Reading Grids Backwards . . . 35
6.2 Empty Arrows . . . 36
6.3 An Example . . . 36
6.4 Grid-Style Rewriting . . . 39
6.5 Building Grid-Style Rewriting . . . 40
6.6 Partial Grids . . . 45
6.7 Building the Partial Grid . . . 49
6.8 Empty Middle Frontiers . . . 51
6.9 Conclusion . . . 52

7 Metrics **52**
7.1 Grids . . . 53
7.2 Partial Grids . . . 56
7.3 Semi-Thue Derivations . . . 59
7.4 Alternative Choices . . . 60

8 The Reversing Procedure **64**
8.1 Monoid Solver . . . 64
8.2 Group Solver . . . 69
8.3 Finite Braid Group Solver . . . 70

9 Future Work **71**

10 Acknowledgements **72**

References **73**

A Maps **76**

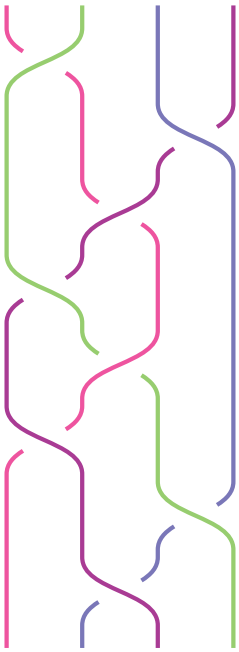
B Dependency Graph **78**

1 Introduction

We begin by defining the basic objects of study, including the algebraic definition of braids, the Lean theorem prover, and the subword reversing algorithm. At the end of this chapter, we give an overview of the entire project.

1.1 Mathematical Braids

Braids were given their first full definition and exposition as a mathematical object by Emil Artin in 1925 [2]. He gave both a topological and algebraic definition of these physical objects. A physical braid is a set of strings in three-dimensional space, affixed at top and bottom to the "floor" and "ceiling." These strings can be twisted together or crossed, but must always flow downwards. No doubling-back is allowed. This is the sort of image to keep in mind:



The topological definition lies outside the scope of this project. We work solely with Artin's algebraic definition, occasionally using images of the corresponding physical objects to aid in exposition.

Definition 1.1 (Presented Group)**.** Consider a set S and a binary relation R on elements of the free group over S, F_S. The **presented group** $\langle S \,|R\rangle$ is the quotient of F_S by the congruence generated over R (the reflexive, symmetric, transitive, multiplicative closure of R).

Definition 1.2. Artin's braid group on n strands is given by the presentation

$$B_n := \left\langle \sigma_1, \ldots, \sigma_{n-1} \;\middle|\; \begin{array}{ll} \sigma_i\sigma_j = \sigma_j\sigma_i & \text{for } |i-j| \geq 2, \\ \sigma_i\sigma_j\sigma_i = \sigma_j\sigma_i\sigma_j & \text{for } |i-j| = 1 \end{array} \right\rangle$$

The infinite braid group B_∞ can be defined as the direct limit $B_\infty := \bigcup_{n\in\mathbb{N}} B_n$ or equivalently by the presentation

$$B_\infty := \left\langle \sigma_1, \sigma_2, \sigma_3 \dots \;\middle|\; \begin{array}{ll} \sigma_i\sigma_j = \sigma_j\sigma_i & \text{for } |i-j| \geq 2, \\ \sigma_i\sigma_j\sigma_i = \sigma_j\sigma_i\sigma_j & \text{for } |i-j| = 1 \end{array} \right\rangle$$

The generator σ_i represents the strand in position i $(1 \leq i < n)$ crossing over the strand to its right.

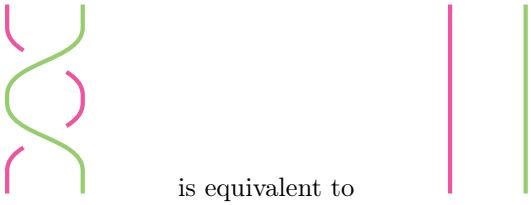


${\sigma_i}^{-1}$ represents the strand in position i crossing under the strand to its right.



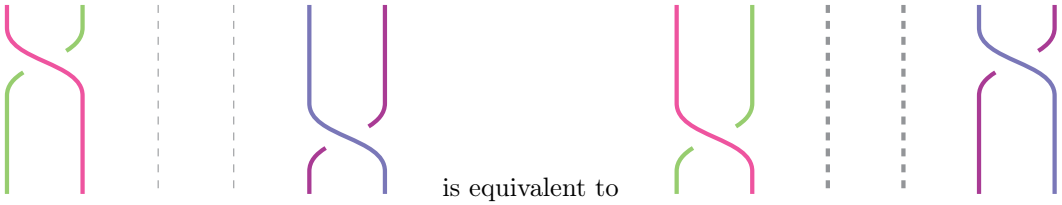
For example, the braid at the opening of this chapter is coded ${\sigma_1}^{-1}\sigma_3{\sigma_2}^{-1}\sigma_1{\sigma_2}^{-1}\sigma_1\sigma_3\sigma_2$.

The physical representation shows how inverses untangle : here is $\sigma_i^{-1}\sigma_i$:

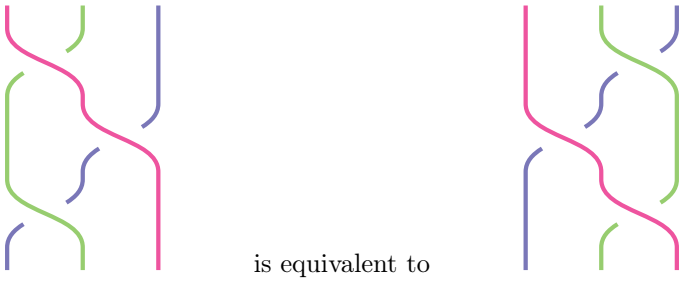


The defining relations of the braid group can be visualized as follows :

Crossings far apart can be slid up and down (if $|i-j| \geq 2$, $\sigma_i\sigma_j$ is equivalent to $\sigma_j\sigma_i$) :



An adjacent strand can be pulled through a crossing ($\sigma_i\sigma_{i+1}\sigma_i$ is equivalent to $\sigma_{i+1}\sigma_i\sigma_{i+1}$) :



Of course, to rigorously show that these relations are sufficient to capture equivalence in the physical objects, one must give a topological definition and the proof that the algebraic and topological definitions coincide. The interested reader can find a modern exposition in Patrick Dehornoy's textbook "The Calculus of Braids", chapters one and two [16].

1.2 The Lean Theorem Prover

Lean is a functional programming language and interactive theorem prover based on dependent type theory. Leonardo de Moura launched Lean in 2013; the language is now on version 4 [8]. Mathematicians worldwide use Lean to formally verify proofs across mathematics, from undergraduate basics to the cutting edge of research. In Lean, one can implement algorithms and execute them using the `#eval` command along with stating and proving theorems about their properties, all in the same file. While algorithms of course must be computable in order to evaluate, Lean is not limited to computable functions or constructive proofs. One can choose to use Lean with constructive or classical logic. When proving correctness of an algorithm, for example, the algorithm must be computable but the proofs of correctness and termination need not be.

A large collaborative project to formalize mathematics in one central repository, named Mathlib, is ongoing, and currently counts 772 contributors [34].[1] As far as we can tell, there has been no formalization of Artin's braid group in Mathlib or elsewhere in Lean, nor in any other theorem prover. Coxeter groups, another type of presented group which is similar in structure to the braid groups, have recently been formalized in Lean.[2]. There has been work in formalized category theory which could be related to braid groups [33, 31, 6], including basic definitions in Lean[3], but the major required theorem[4] is not yet formalized, and the word problem in such categorical structures is as hard as that of the unknot problem [19], compared to polynomial-time complexity for braid groups [13]. Thus, a connection to category theory does not simplify the project, and lies beyond the scope of this paper. Knots and a few invariants thereof were formalized in Isabelle/HOL by Prathamesh in 2015 [32]. However, the mathematics used to study braids (presented groups, localizations, orderings) differs from that used for knots (polynomial invariants). There are also a number of computational packages for working with braids, for example in MATLAB [35], SAGE[5], and GAP [29]. Lastly, two theorems about braid groups (unrelated to this project) very recently appeared on Prove2Me,[6] an AI-based website where users employ their agents to solve mathematical "missions" proposed by humans. Upon a brief perusal, we were unable to find the Lean code on the Prove2Me site, and so are unsure of what definition of the braid groups was used.[7]

As to the formalization of a word problem decision procedure, existing similar work is limited. A recent paper on decidability in presented monoids does offer a (slow) Rocq algorithm (`homog_dec`) which can be used for any presented monoid whose relations preserve length [7]. This could be applied to determine equivalence in braid *monoids* but cannot be extended to braid groups.

[1] Mathlib statistics, as of October 12, 2026

[2] See the project page at https://www.majiajun.org/formalizing-coxeter-group-hecke-algebra-and-kazhdan-lusztig-theory-in-lean/ and the Mathlib file at https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/Coxeter/Basic.html

[3] Braid Monoidal Categories Mathlib page

[4] From Joyal and Street [24] : Letting X be the single generating object in the free braided monoidal category, the set of endomorphisms on the tensor product of n copies of X is isomorphic to the braid group on n strands.

[5] https://doc.sagemath.org/html/en/reference/groups/sage/groups/braid.html

[6] Prove2Me link

[7] This project's Lean definition of braid groups has been available on the internet since late 2024, as a pull request to Mathlib.

1.3 Word Problem Algorithm

The word problem in a group is to determine whether two representatives represent the same equivalence class. In general, the word problem of a presented group is unsolvable [5, 30]. For the braid group in particular, this problem is solvable in polynomial time. A large number of algorithms exist, in part prompted by the early-2000s hope that braid group problems could be used for post-quantum security protocols [27, 10]. Early algorithms with exponential complexity included Artin combing [1] and Garside's method of universal denominators [21], although both methods have subsequently received updates which run in polynomial time [22, 36]. A more modern solution by Birman, Ko, and Lee introduced an alternative presentation of the braid group, yielding a normal form with computational advantages [4]. Other methods such as Dynnikov coordinates [20, 13] and the Lawrence-Krammer representation [26, 3] give exceptionally simple algorithms which work quite efficiently, but the proofs of correctness involve advanced topology. Dehornoy's most recent algorithm is handle reduction, which is experimentally quite fast (but there is no proof of polynomial-time complexity) [13].

For this project, we have implemented Dehornoy's earlier method: subword reversing (as introduced in a journal article [12] and elucidated in his textbook [16]). While this may not be the fastest method, it runs in quadratic time, relies on a manageable amount of theory, and could be generalized to larger classes of groups [15]. Moreover, this solution builds infrastructure that could be reused in a future implementation of other algorithms, such as handle reduction.

The primary contribution lies in the treatment of this algorithm's termination. Dehornoy wrote about this algorithm over a period of approximately twenty-five years. Most of Dehornoy's work presents a compelling sketch of why subword reversing terminates via associating rectangular grid diagrams to individual instances of reversing, but does not give a proof for this method. Bridging this gap required the development of a new proof of termination, done in three stages. It does appear that in one of Dehornoy's earliest papers he gives a different proof which may be fully rigorous as written. This proof is quite delicate (we will discuss this further in Chapter 6). It seems that Dehornoy preferred the visual intuition and simplicity of the grid-based method, so we are pleased to have been able to fill in this method.

The heart of the algorithm is a reversing procedure on braid words. To underscore that we are working with words and not braid group elements, we use $\overline{\sigma_i}$ in place of σ_i^{-1}. The reversing procedure consists of repeatedly rewriting negative-positive pairs $\overline{\sigma_i}\sigma_j$ as follows:

If $|i-j| = 0$, $\overline{\sigma_i}\sigma_i$ is deleted (more precisely, replaced by the empty word ε).
If $|i-j| > 1$, $\overline{\sigma_i}\sigma_j$ is replaced by $\sigma_j\overline{\sigma_i}$
If $|i-j| = 1$, $\overline{\sigma_i}\sigma_j$ is replaced by $\sigma_j\sigma_i\overline{\sigma_j}\,\overline{\sigma_i}$.

Note that these rewritings preserve equivalence under the braid relations. The proofs are straightforward algebraic manipulations; here is an instance of the third case, with numerical indices for readability:

$$\begin{aligned}
\overline{\sigma_2}\sigma_3 &\equiv \overline{\sigma_2}\sigma_3\sigma_2\overline{\sigma_2} && (\text{as } \varepsilon \equiv \sigma_2\overline{\sigma_2})\\
&\equiv \overline{\sigma_2}\sigma_3\sigma_2\sigma_3\overline{\sigma_3}\,\overline{\sigma_2} && (\text{as } \varepsilon \equiv \sigma_3\overline{\sigma_3})\\
&\equiv \overline{\sigma_2}\sigma_2\sigma_3\sigma_2\overline{\sigma_3}\,\overline{\sigma_2} && (\text{as } \sigma_3\sigma_2\sigma_3 \equiv \sigma_2\sigma_3\sigma_2)\\
&\equiv \sigma_3\sigma_2\overline{\sigma_3}\,\overline{\sigma_2} && (\text{as } \overline{\sigma_2}\sigma_2 \equiv \varepsilon)
\end{aligned}$$

Now, let us show reversing in action. We begin with $\overline{\sigma_1}\,\sigma_0\overline{\sigma_2}\sigma_1$; here is the sequence of rewritings:

$$\begin{aligned}
&\overline{\sigma_1}\,\sigma_0\overline{\sigma_2}\sigma_1\\
&\sigma_0\sigma_1\overline{\sigma_0}\,\overline{\sigma_1}\,\overline{\sigma_2}\sigma_1\\
&\sigma_0\sigma_1\overline{\sigma_0}\,\overline{\sigma_1}\sigma_1\sigma_2\overline{\sigma_1}\,\overline{\sigma_2}\\
&\sigma_0\sigma_1\overline{\sigma_0}\sigma_2\overline{\sigma_1}\,\overline{\sigma_2}\\
&\sigma_0\sigma_1\sigma_2\overline{\sigma_0}\,\overline{\sigma_1}\,\overline{\sigma_2}
\end{aligned}$$

So, reversing $\overline{\sigma_1}\,\sigma_0\overline{\sigma_2}\sigma_1$ gives $\sigma_0\sigma_1\sigma_2\overline{\sigma_0}\,\overline{\sigma_1}\,\overline{\sigma_2}$. Note that the final word is in positive-negative form, with no negative-positive pairs appearing.

To determine if two braid group words are equivalent, we may simply check if ab^{-1} is equivalent to the empty word. The full subword reversing algorithm will reverse its input, exchange the positive and negative parts, and reverse again, returning true if that final reversing produces the empty string, and false otherwise.

1.4 Outline

Throughout the text, clickable Lean icons in the righthand margin link to the relevant lines in the GitHub formalization. The repository, frozen at the time of this paper's writing, can be found at https://github.com/hannahfechtner/braids_better/tree/098309403cda787eaf1fa954e07ff0ca3b2927ee. Information about the Lean and Mathlib versions needed to build the code are available in the `README` file.

Chapters 2–5 discuss formalization of known results, following Patrick Dehornoy's textbook *The Calculus of Braids* [16]. Chapters 6–8 expand on Dehornoy's work and contain new material necessary to implement his subword reversing algorithm in a theorem prover.

Chapter 2 develops the algebraic framework needed for the reversing algorithm, including the braid group, braid monoids, and Semi-Thue systems. Before we implement the algorithm itself, we spend the next five chapters preparing theorems which will be used to show its termination and correctness.

Chapters 3–5 prove the localization result that connects equivalence in the braid monoid with equivalence in the braid group, a connection that will be essential to the proof of correctness. Chapter 3 develops the necessary theory of Ore localization. Chapters 4 and 5 then show that the braid monoid satisfies the hypotheses required for this localization: Chapter 4 formalizes Dehornoy's proof of the existence of common multiples, while Chapter 5 proves cancellativity. The proof of cancellativity introduces grid diagrams, which encode braid monoid equivalences and will play a central role throughout the remainder of the project.

Chapters 6 and 7 develop the structures and metrics needed to prove termination of the reversing algorithm. Chapter 6 extends the interpretation of grid diagrams by introducing partial grids and grid-style rewriting, and establishes the correspondence between these new structures and reversing rewrites. Chapter 7 defines metrics on these structures. The material in these two chapters will yield a well-founded measure that decreases as reversing proceeds, providing the basis for the termination proof. The constructions and results developed in these two chapters appear, to the best of my knowledge, to be novel.

Chapter 8 at last brings all these strands together in the implementation of the subword reversing algorithm, first for braid monoid words and then for braid group words. The results of the preceding chapters are combined to prove correctness and termination. The procedure is also extended to the finite braid group. Finally, Chapter 9 concludes with directions for future work.

2 Defining Braids

This chapter develops the algebraic structures that will be used throughout the remainder of the paper. We begin by defining Artin–Tits groups and specializing this construction to Artin's infinite braid group. We then turn to two structures necessary for subword reversing. We begin with presented monoids, and we define the infinite braid monoid and establish its basic properties. We next discuss Semi-Thue systems, which we will use to model string rewriting. Together, these definitions and results provide the algebraic foundation for the subword reversing algorithm. Lastly, we return to all the structures discussed, this time in the finite setting. We introduce the finite braid groups and monoids and develop mappings between the infinite and finite braid group/monoids.

2.1 Artin-Tits Groups

Braid groups are a special case of Artin-Tits groups.

Definition 2.1. Given a semigroup G and $a, b \in G$, **Artin-Tits relations** are those of the form $aba... = bab...$, where each side of the equality has the same length.

Definition 2.2. Artin-Tits groups are presented groups $\langle S|R\rangle$ in which all relations R are Artin-Tits, with at most one relation for each pair of generators. We may succinctly package the information for the relations into a function $m : (S \times S) \to \mathbb{N}$, whose graph can be arranged into a symmetric matrix. Then all relations are of the form

$$\underbrace{aba\cdots}_{\text{length } m(a,b)} = \underbrace{bab\cdots}_{\text{length } m(a,b)}$$

for generators $a, b \in S$, viewed as elements of the free group over S. If no Artin-Tits relation exists between generators a and b, we will set $m(a, b) := 0$.[8]

Artin-Tits groups are very similar to Coxeter groups, which have the additional property that for every generator a, $a^2 = 1$. We are thus able to lightly edit much of the work done by Newell Jensen, Mitchell Lee, and Óscar Álvarez, who formalized the definition and many properties of Coxeter groups.[9]

In Lean, following Jensen, Lee, and Álvarez, we give the set of generators as the type , and the matrix m by the `ArtinTitsMatrix` structure. The `off_diagonal` property ensures that different generators are distinct.

```
structure ArtinTitsMatrix ( : Type*) where
  M : Matrix
  isSymm : M.IsSymm := by decide
  off_diagonal i i' : i  i' → M i i'  1 := by decide
```

A simple recursive definition produces an alternating string of group elements:[10]

```
def Group.alternate [Monoid G] (a b : G) (n : ) : G := match n with
  | 0 => 1
  | n' + 1 => a * alternate b a n'
```

[8] Often in this case $m(a, b) := \infty$, which is not suitable for formalization.

[9] https://github.com/leanprover-community/mathlib4/blob/971f540b1b5f10f11d8f73b8d43986759b9f6dcd/Mathlib/GroupTheory/Coxeter/Basic.lean

[10] This is generalized to monoid elements in the code

We generate the Artin-Tits relation for any two generators with `ArtinTits.relation`; instead of giving an equality, we give a free group element equivalent to the identity:

$$\Big(\underbrace{aba\cdots}_{\text{length } m(a,b)}\Big)\cdot\Big(\underbrace{bab\cdots}_{\text{length } m(a,b)}\Big)^{-1}$$

The occurrences of `FreeGroup.of` in the below definition lift an element of a set to the corresponding free group element.

```
def ArtinTits.relation (M : ArtinTitsMatrix ) (a b : ) : FreeGroup () :=
  Group.alternate (FreeGroup.of a) (FreeGroup.of b) (M a b) * (Group.alternate
    (FreeGroup.of b) (FreeGroup.of a) (M a b))¹
```

We gather all the relations into a single set:

```
def ArtinTits.relation_set : Set (FreeGroup ) :=
  Set.range (Function.uncurry (relation M))
```

Finally, using the already-existing `PresentedGroup` structure from Mathlib, we define `ArtinTitsGroup` which will build the corresponding Artin-Tits group given a matrix m.

```
def ArtinTitsGroup (M : ArtinTitsMatrix ) := PresentedGroup (relation_set M)
```

At this point, we must decide whether or not to duplicate the `PresentedGroup` API, specializing it to Artin-Tits groups. If the construction were not canonical, we would duplicate the API so that their implementation would be opaque to users. Here, the definition as a presented group is canonical, and one can expect the user to know that an Artin-Tits group is a presented group. Often in this case no custom API is given, especially when the code is intended to go into Mathlib, for this leads to unnecessary extra code to maintain. There is a bit of leeway for non-Mathlib code, and also when the specialization leads to simpler hypotheses for certain theorems. The latter is true for the universal property for Artin-Tits groups. We recall the universal property for presented groups in general :

Theorem 2.3. (Universal Property for Presented Groups) Given a set of generators S and a set of relations R on S^*, the universal property for the presented group $\langle S\,|R\rangle$ states that for any group G and function $f : S \to G$ respecting the relations R, there exists a unique group homomorphism $g : \langle S\,|R\rangle \to G$ such that $g([\![s]\!]_R) = f(s)$ for every $s \in S$.

Here, we decide to implement a custom universal property for the Artin-Tits groups, as we can simplify the hypothesis that the function to be lifted respects the relations.

Theorem 2.4. (Universal Property for Artin-Tits Groups) Given a set of generators S, a set of Artin-Tits relations R, and a function m which generates R, the universal property for the Artin-Tits group $\langle S\,|R\rangle$ states that for any group G and function $f : S \to G$ such that for all $a, b \in S$,

$$\underbrace{f(a)\,f(b)\,f(a)\cdots}_{\text{length } m(a,b)} = \underbrace{f(b)\,f(a)\,f(b)\cdots}_{\text{length } m(a,b)}$$

there exists a unique group homomorphism $g : \langle S\,|R\rangle \to G$ such that $g([\![s]\!]_R) = f(s)$ for every $s \in S$.

In Lean, we can phrase this property in terms of the matrix m and `Group.alternate`, without needing to reference `ArtinTits.relation` and `ArtinTits.relation_set`; we simply require that the function f satisfies

```
def ArtinTits.IsLiftable (M : ArtinTitsMatrix ) [Group G] (f :  → G) : Prop :=
  ∀ i j, Group.alternate (f i) (f j) (M i j) = Group.alternate (f j) (f i) (M i j)
```

Then we can prove that `IsLiftable` functions satisfy the liftability requirement for the presented group universal property. We do so by unpacking the definition of `ArtinTits.relation_set` and `ArtinTits.relation`; that way, they may remain opaque to the user. Then our custom universal property needs only an instance of `IsLiftable` :

```
def ArtinTits.toGroup  {G : Type*} [Group G] {f :  → G} (M : ArtinTitsMatrix )
    (hf : IsLiftable M f) : ArtinTitsGroup M →* G
```

By contrast, we do not duplicate the rest of the `PresentedGroup` API (such as extensionality and induction principles) as doing so would not simplify theorems for the user.

2.2 The Braid Group

We now implement the specific case of the "infinite" braid group, hereafter simply referred to as the braid group. The set of generators is $S = \{\sigma_i \mid i \in \mathbb{N}\}$ (we will say i is the *index* of the generator σ_i). This set is represented by the natural numbers in Lean. This does require 0-indexing the generators, in contrast to Artin's original definition. If this is bothersome, it is simple at the end of the project to write a wrapper shifting all indices up by one. Recall the braid relation:

Definition 2.5. The braid relation R_B is defined as the smallest relation satisfying the following two clauses. Let $i, j \in \mathbb{N}$. Then

- If $|i - j| = 1$, then $R_B(\sigma_i\sigma_j\sigma_i,\, \sigma_j\sigma_i\sigma_j)$ (the braid relation, Artin-Tits of length 3)
- If $|i - j| \geq 2$, then $R_B(\sigma_i\sigma_j,\, \sigma_j\sigma_i)$ (the commutative relation, Artin-Tits of length 2)

That is, if $i = j$, the relation has length 0 (it is unneeded); if $|i - j| = 1$ the relation has length 3; otherwise, the relation has length 2. In Lean, we write the corresponding Artin-Tits matrix as follows, and obtain the braid group using the `ArtinTits` API :

```
def BraidMatrixInf : ArtinTitsMatrix  where
  M := Matrix.of fun i j :
    if i = j then 0 else (if i.dist j = 1 then 3 else 2)
  isSymm := by
    grind [Matrix.IsSymm, Matrix.transpose, Matrix.of_apply,
      EmbeddingLike.apply_eq_iff_eq, Nat.dist]
  off_diagonal := by aesop

def BraidGroupInf := ArtinTitsGroup BraidMatrixInf
```

In our mathematical writeup, we have let σ_i represent an element of the set of generators S, as well as an element of the free group over S when defining R. To give this as an element of the braid group, we would write $[\![\sigma_i]\!]_{R_B}$. In Lean, we begin with a natural number `(i : )` as a generator. We write `FreeGroup.of i` for the free group element. Finally, `(PresentedGroup.of i : BraidGroupInf)` will give us the braid group element. We abbreviate `(PresentedGroup.of i : BraidGroupInf)` by `i` as so :

```
def  (k : ) : BraidGroupInf := PresentedGroup.of k
```

While convenient in Lean, `i` is unfortunately nearly-overlapping notation with σ_i, though they represent quite different mathematical objects. To belabor the point, the former (`i`) is an element of the braid group, while the latter (σ_i) is an element of the generating set.

For the braid group, we give a specialized API. Again, the construction of the braid group as a presented group is not meant to be hidden. The custom API will make the rest of the project move more smoothly — no need to constantly unfold the definition of `BraidGroupInf`. For example, for the

universal property, we may omit mention of `BraidMatrixInf` and simplify the required hypothesis to

```
def BraidGroupInf.IsLiftable {G : Type*} [Group G] (f :  → G) : Prop :=
  (∀ i j : , i.dist j = 1 → f i * f j * f i = f j * f i * f j)
  (∀ i j : , 2  i.dist j → f i * f j = f j * f i)
```

Beyond specializations of the universal property, induction principles, et cetera, we moreover prove useful lemmas such as $[\![\sigma_{i+1}\sigma_i\sigma_{i+1}]\!] = [\![\sigma_i\sigma_{i+1}\sigma_i]\!]$. In Lean, this is written as follows :

```
theorem BraidGroupInf.braid {i : } :
     i *  (i+1) *  i =  (i+1) *  i *  (i+1)
```

Such lemmas are mathematically immediate, but in Lean require careful traversal of the structures at play (quotient groups, congruences, setoids, etc).

2.3 Presented Monoids

We next consider a structure akin to presented groups, but lacking inverses : presented monoids. These may be defined as a free monoid quotiented by the congruence generated by a given relation. That is the implementation chosen here, as we are able to build upon the `Congruence` structure which already is in Mathlib as well as the rich API surrounding quotients.

Definition 2.6 (Presented Monoid)**.** We let $\langle S \,|R\rangle^+$ denote the presented monoid on a set S and relation R. We implement this by generating the free monoid on S, then quotienting by the congruence generated on R (the reflexive, symmetric, transitive, multiplicative closure of R). Given $x \in S^*$, we write the equivalence class of x as $[\![x]\!]_R^+$. We drop the subscript R when it can be inferred from context. Given $x, y \in S^*$, we say $x \equiv_R^+ y$ if they are equivalent under the congruence generated by R, once again dropping the subscript when possible.

In Lean, we first must give a formal definition of S-strings. We can select from lists, free monoids, strings, etc. However, `Congruence` must be applied to a type endowed with a `Mul` structure. `FreeMonoid` is the only type in Mathlib that has been given a `Mul` structure. So, we represent the S-strings by `FreeMonoid` instead of `List`.

Then, given a set of relations `rels` over a type , `PresentedMonoid` constructs the monoid with generators `x :` and relations `rels` as a quotient of a congruence relation over `rels`. Here `conGen` generates a congruence.

```
def PresentedMonoid (rels : FreeMonoid  → FreeMonoid  → Prop):= (conGen
    rels).Quotient
```

We provide basic properties of the presented monoid, such as extensionality, the universal property, and lifts of a function f from S^* to $\langle S \,|R\rangle^+$ using various formulations of the required conditions on f (respecting the congruence on R).

Presented monoids are also called abstract rewriting systems, string rewriting systems, and Thue systems. They model the idea of rewriting parts of a string according to a given symmetric relation. (Such derivations will be discussed more fully in the following section, so the following definition is not yet linked to Lean code).

Definition 2.7. [R-derivation] Given a set S and a binary relation R on S^*, an R-derivation is a finite list of S-strings $R_0, R_1, ..., R_n$ such that for every k, $0 \leq k < n$, there exist $(x, y) \in R$ such that $R_k = axb$ and $R_{k+1} = ayb$.

Theorem 2.8. For any *symmetric* relation R, there exists an R-derivation linking R_0 to R_n if and only if they are equivalent in the corresponding presented monoid : $[\![R_0]\!]_R^+ = [\![R_n]\!]_R^+$

A presented monoid is thus a symmetric rewriting system under another name. We give a robust API for a user looking to prove two words are equivalent in a presented monoid, with over a dozen lemmas. For example, we show:

Lemma 2.9. For $x, y, z, w \in S^*$, if $(x, y) \in R$ and $z \equiv_R^+ w$, then $xz \equiv^+ yw$.

Beyond inspiration for future use cases, theorem 2.8 gives us an alternate induction principle for presented monoids:

Theorem 2.10. Consider $a, b \in S^*$ such that $a \equiv_R^+ b$. Let P be a property on pairs of elements of S^*. $P(a, b)$ holds if the following conditions are met:

- *(reflexivity)* $P(a, a)$ holds for all $a \in S^*$.
- *(one-step reduction)* If $(x, y) \in R$, then for any $c, d \in S^*$, $P(cxd, cyd)$.
- *(symmetric one-step reduction)* If $(x, y) \in R$, then for any $c, d \in S^*$, $P(cyd, cxd)$.
- *(transitivity)* For all $a, b, c \in S^*$, if $P(a, b)$ and $P(b, c)$, then $P(a, c)$.

2.4 Semi-Thue Systems

We return to the definition of R-derivations in Definition 2.7, now discussing their implementation in Lean. We could give a definition almost exactly mirroring Definition 2.7, for example requiring there exist some natural number n and a list of $n + 1$ A-strings, with the property that each consecutive pair of A-strings in the list is an instance of a one-step reduction. However, it is much cleaner to give an inductive definition. We begin by defining the $\underset{R}{\longrightarrow}$ notation for the one-step reduction operator :

Definition 2.11. Consider a set A and a binary relation R on A-strings. For all A-strings c and d, if $R(a, b)$ holds, then we have $cad \underset{R}{\longrightarrow} cbd$.

Next, we take its reflexive, transitive closure.

Definition 2.12.

Consider a set A and a binary relation R on A-strings. Then $\xrightarrow[R]{*}$ is the smallest relation satisfying :

- reflexivity : For all $a \in A^*$, $a \xrightarrow[R]{*} a$
- closure : For all $a, b \in A^*$, $a \underset{R}{\longrightarrow} b$ implies $a \xrightarrow[R]{*} b$
- transitivity : For all $a, b, c \in A^*$, if $a \xrightarrow[R]{*} b$ and $b \xrightarrow[R]{*} c$, then $a \xrightarrow[R]{*} c$

This relation is used in the traditional definition of a Semi-Thue system.

Definition 2.13. A Semi-Thue system consists of a tuple (A, R) (where A is a set and R a binary relation over A-strings) along with the relation $\xrightarrow[R]{*}$.

In Lean, we call this `SemiThue`. For this project, it is convenient to provide a second definition of Semi-Thue systems. First, we define a new relation $\underset{R}{\twoheadrightarrow}$:

Definition 2.14. Given a set A and a binary relation R over A-strings, let $\underset{R}{\twoheadrightarrow}$ be the smallest relation satisfying the below two properties :

- reflexivity : For all $a \in A^*$, $a \underset{R}{\twoheadrightarrow} a$
- one-step transitivity : For all $a, b_1, x, b_2, y \in A^*$, if $a \underset{R}{\twoheadrightarrow} b_1 x b_2$ and xRy, then $a \underset{R}{\twoheadrightarrow} b_1 y b_2$

Note that the second condition is equivalent to the following, which makes the pen-and-paper notation for recording derivations cleaner:

- *alternative one-step transitivity* : For all $a, b_1, x, b_2, y \in A^*$, if $a \underset{R}{\twoheadrightarrow} b_1 x b_2$ and $b_1 x b_2 \underset{R}{\longrightarrow} b_1 y b_2$, then $a \underset{R}{\twoheadrightarrow} b_1 y b_2$

Whichever formulation of one-step transitivity we make, we note the initial word is always preserved throughout an $\underset{R}{\twoheadrightarrow}$–derivation. This is the main difference between our two definitions. Their behavior otherwise is nearly identical :

Theorem 2.15. For all $a, b \in A^*$, $a \underset{R}{\twoheadrightarrow} b$ iff $a \overset{*}{\underset{R}{\longrightarrow}} b$.

Theorem 2.15 allows us to replace $\overset{*}{\underset{R}{\longrightarrow}}$ with $\underset{R}{\twoheadrightarrow}$ in the definition of Semi-Thue systems, to obtain a variant which we call `SemiThueDerivation` in Lean.

The definition for both $\overset{*}{\underset{R}{\longrightarrow}}$ and $\underset{R}{\twoheadrightarrow}$ is straightforward in Lean (for the latter, using the original one-step transitivity condition). As to the choice of type for A-strings, we make a different decision than that for presented monoids. Here, we use lists, called `List` in Lean. The focus for Semi-Thue systems is on rewriting, which has a more computational flair. The API surrounding lists, with computationally-minded functions such as `getLast`, is far easier to use. The downside is that any work discussing both presented monoids and Semi-Thue systems requires repeated conversions between lists and free monoids. The occasional hassle of dealing with `FreeMonoid.toList` and `List.toFreeMonoid` is outweighed by the convenience of using `List` for Semi-Thue systems.

2.5 The "Braid" Monoid

With both presented monoids and Semi-Thue derivations in place, we can now specialize these general constructions to the braid generators and relations. At this point, we have referenced a proof that the braid *group* aligns with the topological definition of braids. Although it is tempting to think of a monoid in which the braid relations hold as a "braid monoid" (perhaps braids with only overcrossings), until we can connect such a monoid to the braid group mathematically, we have no geometric interpretation of the monoid. It is, for the moment, an arbitrarily defined mathematical structure with no connection to the braid group.

Definition 2.16. The "infinite" braid monoid B^+, from now on simply referred to as the braid monoid, is given by the following presentation:

$$B^+ := \left\langle \sigma_0, \sigma_1, \sigma_2 \dots \;\middle|\; \begin{array}{ll} \sigma_i \sigma_j = \sigma_j \sigma_i & \text{for } |i-j| \geq 2, \\ \sigma_i \sigma_j \sigma_i = \sigma_j \sigma_i \sigma_j & \text{for } |i-j| = 1 \end{array} \right\rangle^+$$

Definition 2.17. The set of **braid monoid words** is

$$BW^+ := \left(\bigcup_{i \in \mathbb{N}} \sigma_i \right)^*$$

We denote the **relation** used in the definition of the braid monoid as R_{B^+}.

The absolute-value condition in the relations leads to a doubling of cases in many proofs. Our Lean implementation of the braid monoids is direct; we do not generalize to some sort of "Artin-Tits monoid", and thus there is no requirement for the relation to be symmetric. To simplify future proofs, we define a different relation for our formalization.

Definition 2.18. The simplified relation $R^s_{B^+}$ is defined as follows:

$$R^s_{B^+} := \begin{cases} \sigma_i\sigma_j = \sigma_j\sigma_i & \text{for } j - i \geq 2 \\ \sigma_i\sigma_{i+1}\sigma_i = \sigma_{i+1}\sigma_i\sigma_{i+1} & \text{for } j - i = 1 \end{cases}$$

Fact 2.19. $\langle \sigma_0, \sigma_1, \sigma_2, ... | R^s_{B^+} \rangle = B^+$. We have the analogous property for $R^s_{B^+_n}$ and B^+_n.

Proof. (Sketch) Presented monoids are generated by taking the congruence over the given relation (hence the symmetric closure), and R_{B^+} is the symmetric closure of $R^s_{B^+}$. □

For our Lean definition of the relations for B^+, note that in the second case $j = i + 1$. We thus define the relation for the braid monoid as follows:

```
inductive braid_monoid_rels_inf : FreeMonoid → FreeMonoid → Prop
  | adjacent (i : ): braid_monoid_rels_inf (of i * of (i+1) * of i) (of (i+1) *
    of i * of (i+1))
  | separated (i j : ) (h : i + 2  j) : braid_monoid_rels_inf (of i * of j) (of j
    * of i)
```

This can be plugged in to our previously-defined `PresentedMonoid` definition, neatly giving us the braid monoid.

We next define a few properties of braid monoid elements.

Definition 2.20. The **length** of a braid monoid element b is written $|b|$. We define $|[\![a]\!]|$ as the length of a (number of letters; the empty word ε has length 0). This is well-defined, as a quick inspection of the relations shows length is preserved under the braid monoid equivalence.

Definition 2.21. The **set of generators** of a braid monoid element is given by the function gen. $\mathsf{gen}([\![a]\!])$ is defined as the set of generators appearing in a. This is similarly well-defined.

Definition 2.22. The **index** of a braid monoid word b is the index of the largest generator occurring in b. We denote this idx, defining $\mathsf{idx}(b) := \max\{i \mid \sigma_i \in \mathsf{gen}(b)\}$.

Lemma 2.23. The reversing function on free monoid words, r, reverses the order of the letters in a free monoid word. $a \equiv^+ b$ if and only if $r(a) \equiv^+ r(b)$. This is proved in each direction by structural induction on the equivalence. It is a useful tool to derive left common multiples and right cancellativity from right common multiples and left cancellativity respectively (or vice versa).

We will thus need to define length, set of symbols in, and reversing for free monoid elements. Let us return for a moment to the choice made earlier about the type for S-strings. We were forced to select free monoids instead of lists. We now see one downside to this choice : `List` is much richer than `FreeMonoid`. `List` already has functions `length`, `toFinset`, `reverse`, etc. Although it is simple to implement these for `FreeMonoid`, we must tread carefully. While free monoids are in fact defined as lists in Lean, they are meant to perform different roles. Free monoids generally bend more to mathematical theorems in abstract algebra, whereas lists are more used for computation. Even when the two share similarly-minded functionality, it is from a different viewpoint and with different implementation details. For example, if the idea is to apply a function to every element of our string and then combine them, we have `FreeMonoid.lift` but `List.foldr`. It is a balancing act to keep the two structures distinct, but also usable. Each has lemmas the other lacks.

We have decided to add definitions and lemmas to `FreeMonoid` if they will be able to be defined on the eventual *presented* monoid, the braid monoid. Such definitions seem reasonably "algebraic" since they can be defined for even more complex algebraic structures. Thus, we add in `FreeMonoid.length`, `FreeMonoid.reverse`, and `FreeMonoid.symbols` (the set of generators appearing in an element), because these play nicely with the braid monoid structure.

We take a different approach for `List` definitions that do not remain invariant in the braid structure, such as `getLast` (this returns the last element in a list). Equivalent braids need not have the same last generator — $\sigma_1\sigma_3$ and $\sigma_3\sigma_1$ certainly do not! In this case, we have decided to use `FreeMonoid.toList` when needed. This is a bit cumbersome, but thankfully quite rare for this project.

We amass simple results of equivalence classes for words:

Lemma 2.24. If $\sigma_i \equiv^+ a$, then $a = \sigma_i$. Thus, $[\![\sigma_i]\!] = \{\sigma_i\}$

Lemma 2.25. Let $(w, x) \in R_{B^+}$. If $w \equiv^+ a$, then $a = w$ or $a = x$. Similar reasoning for x shows us $[\![w]\!] = \{w, x\} = [\![x]\!]$

Proof. For the case of the commutative relation, $(\sigma_i\sigma_j, \sigma_j\sigma_i) \in R_{B^+}$ with $|i - j| \geq 2$, since we know that both length and set of generators is preserved under equivalence, our conclusion is immediate. For the case of the braid relation, $(\sigma_i\sigma_j\sigma_i, \sigma_j\sigma_i\sigma_j) \in R_{B^+}$ with $|i - j| = 1$, the information about length and set of generators is not enough as those allow for, say, $a = \sigma_j\sigma_i\sigma_i$. Instead, we induct on the equivalence $w \equiv^+ a$ following Theorem 2.10. □

For long elements of BW^+, it is frequently difficult to prove equivalence by the rewriting conditions for presented monoids (i.e. Lemma 2.9), as parentheses need to be moved around so that Lean's rewrite tactic, `rw`, can exactly pattern-match the terms. Once we move to a specific presented monoid, we have concrete relations to work with. We can associate all parentheses to the left, and then use one of the following lemmas:

Lemma 2.26. For all $x \in BW^+$, if $|i - j| > 1$, then

$$\left(x \cdot [\![\sigma_i]\!]^+\right) \cdot [\![\sigma_j]\!]^+ = \left(x \cdot [\![\sigma_j]\!]^+\right) \cdot [\![\sigma_i]\!]^+$$

Lemma 2.27. For all $x \in BW^+$, if $|i - j| = 1$, then

$$\left(\left(x \cdot [\![\sigma_i]\!]^+\right) \cdot [\![\sigma_j]\!]^+\right) \cdot [\![\sigma_i]\!]^+ = \left(\left(x \cdot [\![\sigma_j]\!]^+\right) \cdot [\![\sigma_i]\!]^+\right) \cdot [\![\sigma_j]\!]^+$$

For example, say we are trying to transform the element $[\![\sigma_2\sigma_7\sigma_3\sigma_4\sigma_5\sigma_4\sigma_0\sigma_1]\!]$ into $[\![\sigma_2\sigma_7\sigma_5\sigma_3\sigma_4\sigma_5\sigma_0\sigma_1]\!]$. First, we separate all of the generators : $[\![\sigma_2]\!] \cdot [\![\sigma_7]\!] \cdot [\![\sigma_3]\!] \cdot [\![\sigma_4]\!] \cdot [\![\sigma_5]\!] \cdot [\![\sigma_4]\!] \cdot [\![\sigma_0]\!] \cdot [\![\sigma_1]\!]$. Next, we associate all the parentheses to the left :

$$\Big(\Big(\Big(\big(\big(([\![\sigma_2]\!] \cdot [\![\sigma_7]\!]) \cdot [\![\sigma_3]\!]\big) \cdot [\![\sigma_4]\!]\big) \cdot [\![\sigma_5]\!]\Big) \cdot [\![\sigma_4]\!]\Big) \cdot [\![\sigma_0]\!]\Big) \cdot [\![\sigma_1]\!]$$

Then, Lean's `rw` tactic used with lemma **??** will assign $(([\![\sigma_2]\!] \cdot [\![\sigma_7]\!]) \cdot [\![\sigma_3]\!])$ as x, and rewrite $\Big(\big(\big(\big(([\![\sigma_2]\!] \cdot [\![\sigma_7]\!]) \cdot [\![\sigma_3]\!]\big) \cdot [\![\sigma_4]\!]\big) \cdot [\![\sigma_5]\!]\big) \cdot [\![\sigma_4]\!]\Big)$ to $\Big(\big(\big(\big(([\![\sigma_2]\!] \cdot [\![\sigma_7]\!]) \cdot [\![\sigma_3]\!]\big) \cdot [\![\sigma_5]\!]\big) \cdot [\![\sigma_4]\!]\big) \cdot [\![\sigma_5]\!]\Big)$, giving us

$$\Big(\Big(\Big(\big(\big(([\![\sigma_2]\!] \cdot [\![\sigma_7]\!]) \cdot [\![\sigma_3]\!]\big) \cdot [\![\sigma_5]\!]\big) \cdot [\![\sigma_4]\!]\Big) \cdot [\![\sigma_5]\!]\Big) \cdot [\![\sigma_0]\!]\Big) \cdot [\![\sigma_1]\!]$$

We then use `rw` with lemma 2.26, explicitly setting x as $([\![\sigma_2]\!] \cdot [\![\sigma_7]\!])$ to obtain

$$\Big(\Big(\Big(\big(\big(([\![\sigma_2]\!] \cdot [\![\sigma_7]\!]) \cdot [\![\sigma_5]\!]\big) \cdot [\![\sigma_3]\!]\big) \cdot [\![\sigma_4]\!]\Big) \cdot [\![\sigma_5]\!]\Big) \cdot [\![\sigma_0]\!]\Big) \cdot [\![\sigma_1]\!]$$

which can be simplified to $[\![\sigma_2\sigma_7\sigma_5\sigma_3\sigma_4\sigma_5\sigma_0\sigma_1]\!]$, as desired. The once-off hassle of separating generators and associating parentheses to the left makes multi-step calculations significantly simpler, without requiring tedious, targeted applications of associativity of multiplication.

2.6 Finite Braid Groups and Finite Braid Monoids

Above, we have discussed only infinite braid groups and infinite braid monoids and the equivalences on them, $\equiv$ and $\equiv^+$. We may also consider the finite braid groups/monoids. Note that a braid on n strands has $n-1$ generators. Those generators are numbered $\sigma_0, \sigma_1, ...\sigma_{n-2}$.

Definition 2.28. The finite braid group B_n, representing braids on n strands, is given by

$$B_n := \left\langle \sigma_0, \ldots, \sigma_{n-2} \;\middle|\; \begin{array}{ll} \sigma_i\sigma_j = \sigma_j\sigma_i & \text{for } |i-j| \geq 2, \\ \sigma_i\sigma_j\sigma_i = \sigma_j\sigma_i\sigma_j & \text{for } |i-j| = 1 \end{array} \right\rangle$$

We denote the defining relation (on the righthand side) as R_{B_n} and the equivalence on elements of B_n by $\equiv_n$.

Note that B_0 and B_1 are both the trivial group. The finite braid monoids are given using the same generators and relations as for the finite braid groups :

Definition 2.29.

$$B_n^+ := \left\langle \sigma_0, \ldots, \sigma_{n-2} \;\middle|\; \begin{array}{ll} \sigma_i\sigma_j = \sigma_j\sigma_i & \text{for } |i-j| \geq 2, \\ \sigma_i\sigma_j\sigma_i = \sigma_j\sigma_i\sigma_j & \text{for } |i-j| = 1 \end{array} \right\rangle^+$$

Here, the relation is denoted $R_{B_n^+}$ and the equivalence, $\equiv_n^+$.

Definition 2.30. We define the set of generators in B_n (and B_n^+) as $S_n = \{\sigma_i | 0 \leq i \leq n-2\}$. Then the set of finite-braid group words is $BW_n := (S_n \cup \overline{S_n})^*$ and the set of finite-braid monoid words is $BW_n^+ := S_n^*$.

Let us now move to the Lean implementation. For infinite braid groups and monoids, the generators are kept track of via their index, of type $\mathbb{N}$. For B_n and B_n^+, the indices of the generators are natural numbers, but bounded above by $n-1$ (in Lean, this is the predecessor of n, `n.pred`). We thus use the `Fin` type. It is a bundled type: an object of type `Fin n.pred` contains a natural number k and a proof that $k < n-1$. Aside from the change in type of the indices, the implementations of the finite braid groups and braid monoids is almost exactly akin to that of their infinite counterparts. Thus, there are clear similarities between the finite and infinite braid groups (and respective monoids). When we prove properties about one, it would be nice to have some kind of "converter" to port them to the other group/monoid.

This is especially true as working with the finite braid groups/monoids is difficult in Lean. First, we are always juggling a hypothesis of the form $k < n-1$. Moreover, natural number subtraction is truncated, which makes many arithmetic proofs long. In the case of the braid relations, where one often must prove something of the form $i+2 \leq j$, this complicates things enormously. Thus, we work within the finite braid monoid as sparingly as possible. Luckily, the relation on finite braids is a restriction on that for infinite braids.

Theorem 2.31. For braid words a, b, if $a \equiv_n b$, then $a \equiv_m b$ for all $m > n$, and also $a \equiv b$. If a, b are braid *monoid* words, and if $a \equiv_n^+ b$, then $a \equiv_m^+ b$ for all $m > n$, and also $a \equiv^+ b$.

There is a small hiccup here : when we say $a \equiv_n b$ our formalization requires $a, b \in B_n$, but when we say $a \equiv_k b$ the formalization requires $a, b \in B_k$. It is quick to see that we have $BW_0 \subset BW_1 \subset BW_2 ... \subset BW$, and so this poses little trouble to the paper-and-pencil mathematician. In Lean, we must cast from `Fin n.pred` to `Fin m.pred` or $\mathbb{N}$.

When can we transfer an equivalence from the infinite braid group or monoid to the corresponding finite structure? Let us begin with the monoid case.

Theorem 2.32. Consider $a, b \in BW^+$ such that $a \equiv^+ b$. Let $\mathsf{idx}(a) = i_a$ (that is, the largest generator index occurring in the word a is i_a) and $\mathsf{idx}(b) = i_b$. Then for any $n \geq \max(i_a, i_b)$, we have $a \equiv^+_n b$.

The proof proceeds by induction on the derivation $a \equiv^+ b$. In our formalization, we must cast the infinite braid monoid element "down" to a finite braid monoid element. We call this procedure `FreeMonoid.mapNatToFin` :

```
def FreeMonoid.mapNatToFin (n : ) (a : FreeMonoid ) (bound : ∀ x ∈ a, x < n) :
    FreeMonoid (Fin n)
```

The above theorems will eventually be used to apply our solver to the finite case and show that it is correct for finite braid group inputs. We leave the finite braid groups/monoids aside until the final chapter – the algorithm is built for the infinite braid group, and then proved to work for the finite group as well. We are not yet able to give a theorem analogous to Theorem 2.32 for the braid groups. In fact, as the very last step of our project, we will use the correctness specification of our solver to prove this.

In sum, we have defined the infinite and finite braid groups, as well as a number of structures needed to implement and reason about subword reversing (presented monoids and Semi-Thue systems).

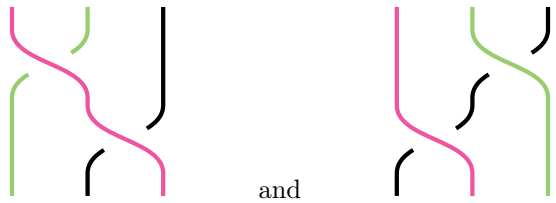
3 Localization

We next elucidate the connection between the braid group and the "braid" monoid by constructing an embedding of B^+ into B. In this chapter, we discuss Ore localization, the procedure by which this is done. We specialize it to our particular use case, first to localizing by the entire monoid, then to localizing by an entire *presented* monoid. We merely state the required conditions on the (presented) monoid, and leave the proofs to the following two chapters. (Note that there is no discussion of the finite braid groups/monoids).

3.1 Ore Localization

Given a monoid M and a submonoid M_0 of M, a localization procedure produces an associated group of fractions, essentially "adding inverses" for every element of M_0. The usual procedure requires a commutative monoid. However, braid concatenation is not commutative.

For example, one may see that $\sigma_1\sigma_2$ does not equal $\sigma_2\sigma_1$ through properties of braid relations (no relations match $\sigma_1\sigma_2$). Or visually, neither of these two braids may be deformed into the other :



and

Instead, we use the Ore localization, which requires different monoid properties. We denote the (left)[11] Ore localization of M by M_0 as $[M_0^{-1}]M$.

[11] There also exists a *right* Ore localization $M[M_0^{-1}]$, used by Dehornoy. Here we choose to use the left localization, as that is what has already been implemented in Mathlib. Either can be used for the braid group, as it is cancellative and has common multiples on both sides.

The following two conditions suffice for left Ore localization; we will refer to them as Ore conditions throughout this project:

Definition 3.1. We say that a monoid M and a submonoid M_0 of M satisfy the Ore conditions if the following two criteria hold :

1. Left Common Multiples: For any $a \in M$, $b \in M_0$, there exist $c \in M_0$ and $d \in M$ such that $ca = db$.

2. Right Cancellativity : For $a, b \in M$ and $c \in M_0$, if $ac = bc$, then $a = b$.

The second condition can be slightly weakened.[12] Ore localization with a weakened cancellativity condition has already been implemented in Lean by Jakob von Raumer, with assistance from Kevin Klinge and Andrew Yang.[13] It is found under ring theory; the file first gives the case of a monoid. We will specialize this code to the case of localizing by the entire monoid, and then to presented monoids. We briefly summarize the mathematics from the formalization of von Raumer.

We begin with a monoid M and a submonoid $M_0 \subseteq M$ satisfying the Ore conditions. We consider all pairs of the form (x, y) with $x \in M$ and $y \in M_0$. We give the following equivalence relation on pairs:

Definition 3.2 ($\sim$)**.** We say $(x_1, y_1) \sim (x_2, y_2)$ iff there exist $a \in M$, $b \in M_0$ such that $ax_1 = bx_2$ and $ay_1 = by_2$.

Theorem 3.3. $\sim$ is an equivalence relations on pairs (x, y).

The proofs of the theorem follows directly from the definition of $\sim$. Next, we consider equivalence classes of pairs under $\sim$. These equivalence classes will be the elements of the localization. We use the abbreviation $\frac{x}{y}$ the equivalence class of (x, y) under $\sim$.

Definition 3.4. Given a monoid M and a submonoid $M_0$of M satisfying the Ore conditions, we define the monoid $[M_0^{-1}]M$ as the set $\{\frac{x}{y} | \, x \in M, y \in M_0\}$, with $\frac{1_M}{1_{M_0}}$ as the identity (note that 1_M is in fact equal to 1_{M_0}) and a binary operation $\bullet$ defined as follows: $\frac{x}{y} \bullet \frac{z}{w} = \frac{az}{by}$ where $a \in M$ and $b \in M_0$ are such that $aw = bx$.

The proof that the definition is independent of the choice of common multiples a and b is straightforward, as is that for associativity of the operator. We next consider a mapping from M into $[M_0^{-1}]M$.

Theorem 3.5. The function h from M into $[M_0^{-1}]M$, defined by $h(x) := \frac{x}{1}$, is a homomorphism.

Proof. We have $h(x_1x_2) = \frac{x_1x_2}{1}$. To calculate $h(x_1) \bullet h(x_2) = \frac{x_1}{1} \bullet \frac{x_2}{1}$, following definition 3.4 we look for a left common multiple of x_1 and 1. As $1 \cdot x_1 = x_1 \cdot 1$, we may let $a = x_1$ and $b = 1$. Hence $\frac{x_1}{1} \bullet \frac{x_2}{1} = \frac{x_1 \cdot x_2}{1 \cdot 1}$. Thus, $h(x_1x_2) = h(x_1) \bullet h(x_2)$. □

We now add one more condition. Together with our original Ore conditions, these form the cancellative Ore conditions:

3. Left Cancellativity: For $a, b \in M$ and $c \in M_0$, if $ca = cb$, then $a = b$.

In the context of the cancellative Ore conditions, h is an embedding of M into $[M_0^{-1}]M$:

[12] We could merely require that common factors on the left be turned in to common factors on the right. That is, for $a, b \in M$, $c \in M_0$, if $ac = bc$ then there exists $d \in M_0$ such that $da = db$.

[13] Their work can be found here.

Theorem 3.6. Given a monoid M with a submonoid M_0 satisfying the *cancellative* Ore conditions, the function $h(x) := \frac{x}{1}$ is injective.

Proof. Assume $h(x_1) = h(x_2)$; that is, $\frac{x_1}{1} = \frac{x_2}{1}$. Then $(x_1, 1) \sim (x_2, 1)$. There exist $a \in M, b \in M_0$ such that $ax_1 = bx_2$ and $a \cdot 1 = b \cdot 1$. Hence $a = b$ and so $bx_1 = bx_2$. By left-cancellativity, $x_1 = x_2$ as desired. □

We now lay the full cancellative Ore conditions aside for the moment. For this project, we first specialize to the case of localizing by the entire monoid, viewed as a submonoid of itself. Mathlib already contains the typeclass `OreSet`, which represents a pair of a monoid and a submonoid satisfying the (weakened) Ore conditions. Thus, we want to show that if a monoid has left common multiples and right cancellativity, then it paired with the full submonoid of itself is an instance of `OreSet`. To take in the hypotheses that the monoid M satisfies the left common multiple and right cancellativity criteria, we use mixins. These are typeclasses that extend a structure to add in an extra property. For example, we define

```
class IsCommonLeftMultipleMul (M : Type*) [Mul M] where
  common_left_multiple : ∀ a b : M,  c d : M, c * a = d * b
```

The above code is nonconstructive, as the existential quantifier merely asserts the existence of the common multiples. It does not provide them as data. This is in line with general Mathlib conventions. However, the existing Ore localization code is set up to be used constructively, because `OreSet` has fields for the common-multiple data. We take advantage of this design, and give both constructive and non-constructive versions for braids. The constructive version will be useful for a concurrent project, developing a widget to visualize braids in the Lean InfoView. The widget will use the data from constructive proofs to display common multiples, equivalences, canonical forms, etc. Here is the analogous constructive version of `IsCommonLeftMultipleMul` :

```
class HasCommonLeftMultipleData (M : Type*) [Mul M] where
  cl : M → M → M
  cl : M → M → M
  cl_spec : ∀ a b : M, cl a b * a = cl a b * b
```

We continue our discussion in the non-data-carrying setting only. The nonconstructive version is used in the implementation of the braid word problem solver, for Ore localization is used only in the proof of correctness of the algorithm, not in its definition. As such, it need not be constructive. We thus use the following mixin hypotheses added to a monoid M:

```
variable {M : Type*} [Monoid M] [IsRightCancelMul M] [IsCommonLeftMultipleMul M]
```

With the above assumed, we are able to show that the full submonoid, `( : Submonoid M)`, is an instance of `OreSet` :

```
noncomputable instance : OreLocalization.OreSet ( : Submonoid M) where ...
```

The proof uses `Classical.choose` to noncomputably provide the witnesses that the constructive `OreSet` requires, and `Classical.choose_spec` to assert that those witnesses have the desired property of being common multiples.

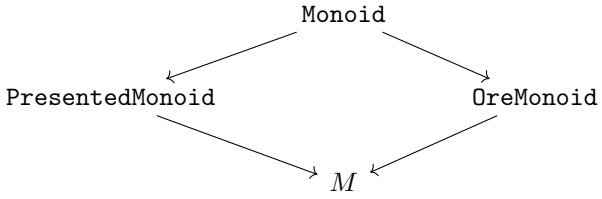
We now may define the Ore localization of M by itself, as Lean can infer that the required Ore conditions are met from the above two lines of code:

```
abbrev "OreLocalizationSelf" => OreLocalization ( : Submonoid M) M
```

Theorem 3.7. For any monoid M satisfying the Ore conditions, $[M^{-1}]M$ has a group structure.

Proof. We already have shown that $[M^{-1}]M$ has a monoid structure. All that remains is to define inverses. For any $\frac{x}{y} \in [M^{-1}]M$, we define $(\frac{x}{y})^{-1} := \frac{y}{x}$. We omit the straightforward proof that $\frac{y}{x}$ satisfies the properties of an inverse. □

Next, we consider *presented* monoids. We aim to plug in a presented monoid satisfying the Ore conditions to the existing Ore localization API. It is now clear why mixins are required. If we instead took in the hypotheses by defining an extension of the `Monoid` class satisfying the Ore hypotheses called `OreMonoid`, much like `CancelMonoid`, we would run into an issue. Both the `PresentedMonoid` and `OreMonoid` instances would carry with them a `.toMonoid` function. If we try to give Lean a type `M` with instances `[PresentedMonoid M] [OreMonoid M]`, Lean can infer the `Monoid` instance on `M` in two different ways. This is referred to as a "bad diamond".



Thus, we choose to use mixins to take in the hypotheses.

Theorem 3.8. Let $M = \langle S \,|R\rangle^+$ satisfy Ore conditions. Then $[M^{-1}]M$ is isomorphic to the presented group on those same generators and relations, $\langle S \,|R\rangle$.

The proof will proceed by a careful inspection of the following universal properties of the Ore localization and of presented groups; we begin by defining those universal properties. For simplicity, we let $\mathsf{PML}(S, R)$ denote $\left[(\langle S \,|R\rangle^+)^{-1}\right] \langle S \,|R\rangle^+$.

Definition 3.9 (Lift)**.** Given a map f from a set S to a monoid M, we may lift f to a function from the free group on S to M, which we will denote $\overline{f}$. We denote the lift of f to a function from the free monoid on S as $\overline{f}^+$

Theorem 3.10. [Universal property of the Ore localization of a presented monoid] Consider a presented monoid $M = \langle S \mid R\rangle^+$ satisfying the Ore conditions. For any group G and any map $f : S \to G$ respecting the relations R (for $(r_1, r_2) \in R$, we have $\overline{f}^+(r_1) = \overline{f}^+(r_2)$), there exists a unique homomorphism

$$f^* : [M^{-1}]M \to G$$

such that, for every $s \in S$,

$$f^*\left(\frac{[\![s]\!]_R^+}{1}\right) = f(s).$$

Proof. This is merely a specialization of the universal property for Ore localizations, which is again a specialization of that for quotients. □

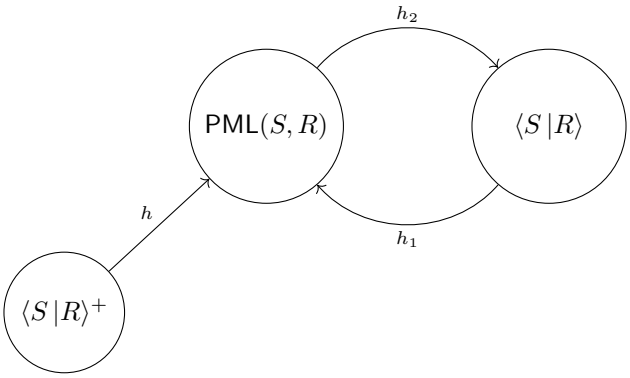
Remark. Since the identity map on $\mathsf{PML}(S, R)$ is a group homomorphism, the uniqueness clause of the universal property implies that any group homomorphism $g : \mathsf{PML}(S, R) \to \mathsf{PML}(S, R)$ satisfying $g\left(\frac{[\![s]\!]_R^+}{1}\right) = \frac{[\![s]\!]_R^+}{1}$ for all $s \in S$ must be the identity map.

Our next task will be to define two homomorphisms between the presented group $\langle S \,|R\rangle$ and the Ore localization of the associated presented monoid $\langle S \,|R\rangle^+$.

Definition 3.11. We define a homomorphism h_1 from $\langle S \,|R\rangle$ to $\mathsf{PML}(S,R)$ using the universal property for presented groups to lift the function $f(s) = \frac{[\![s]\!]_R^+}{1_{\langle S,R\rangle^+}} : S \to \mathsf{PML}(S,R)$. The function f is the composition of h from Theorem 3.5 (recall $h(s) = \frac{s}{1}$) with $[\![\cdot]\!]_R^+$ (in Lean, this is `PresentedMonoid.of` when applied to a single generator).

We define a homomorphism h_2 from $\mathsf{PML}(S,R)$ to $\langle S \,|R\rangle$ by using the universal property for Ore localizations of presented monoids to lift the function $m \mapsto [\![m]\!]_R$.

Remark. Note that for any $s \in S^*$, $h_2\left(\frac{[\![s]\!]_R^+}{1}\right) = [\![s]\!]_R$. The proof proceeds by induction on s; the case where s is a single generator is a consequence of theorem 3.10.



Finally, we put the pieces together to complete the proof of Theorem 3.8. Composing h_1 with h_2 gives a homomorphism from $\langle S \,|R\rangle$ to $\langle S \,|R\rangle$ – but by uniqueness portion of the universal property on presented group (Theorem 2.3), that must be the identity! Similarly, composing h_2 with h_1 gives the identity by Theorem 3.10. Thus, h_1 is a bijection (and h_2 is as well). Hence we have an isomorphism between $\mathsf{PML}(S,R)$ and $\langle S \,|R\rangle$. Thus, the localization of a presented monoid is isomorphic to the presented group on those same generators/relations, as desired. □

As to the Lean implementation of the above proof, we must note that the API for presented monoids takes a relation as a function of two `FreeMonoid` arguments, while the API for presented groups requires the relations to be given as a set of `FreeGroup` elements. To translate from the former to the latter, we use the function

```
def free_group_set_of_function (rels : FreeMonoid  → FreeMonoid  → Prop) : Set
    (FreeGroup )
```

We further provide a number of lemmas about the interaction of `free_group_set_of_function` with lifts and the relations. For example, consider a function f from S to a group G_1. $\overline{f}$ applied to any element of `free_group_set_of_function rels` equals one if and only if $\overline{f}^+(r_1) = \overline{f}^+(r_2)$ for all (r_1, r_2) for which `rels` holds:

```
theorem free_group_set_of_function_lift_eq_one_iff {G : Type} [Group G]
        {rels : FreeMonoid  → FreeMonoid  → Prop} (f :  → G) :
    (∀ r ∈ free_group_set_of_function rels, (FreeGroup.lift f) r = 1)
    (∀ r r, rels r r → FreeMonoid.lift f r = FreeMonoid.lift f r)
```

Such lemmas help us prove the required hypotheses for the universal properties in the definition of h_1 and h_2. The verification that h_1 and h_2 are inverses of one another is straightforward, showing that $\mathsf{PML}(S,R)$ and $\langle S \,|R\rangle$ are isomorphic.

Returning to the mathematical development, we aim to show that for a cancellative presented monoid, equivalence in the associated presented group of positive words implies equivalence in the

presented monoid of those same words. For the rest of this section, we discuss only presented monoids which satisfy the *cancellative* Ore conditions. (We say that the monoid M satisfies the (cancellative) Ore conditions if M considered with the full submonoid of itself satisfies the (cancellative) Ore conditions.)

Definition 3.12. We define the function $\iota : \langle S \,|R\rangle^+ \to \langle S \,|R\rangle$ by $\iota := h_2 \circ h$, where h is the function from Theorem 3.5.

Theorem 3.13. For a monoid $\langle S \,|R\rangle^+$ satisfying the cancellative Ore conditions, ι is an embedding of $\langle S \,|R\rangle^+$ into $\langle S \,|R\rangle$.

Proof. By Theorem 3.6, h is an injective homomorphism. h_2 is as well – it is a homomorphism by its definition, and we exhibited above that h_1 is its two-sided inverse. Composition preserves both injectivity and the property of being a homomorphism, so ι is an embedding of $\langle S \,|R\rangle^+$ into $\langle S \,|R\rangle$. □

Finally, we may show that equivalence in the group implies equivalence in the monoid for positive words :

Theorem 3.14. Let $\langle S \,|R\rangle$ be a left-cancellative monoid satisfying the Ore conditions. If $a \equiv_R b$ for **positive** words $a, b \in S^*$ (those containing no inverses)[14], then $a \equiv_R^+ b$.

Proof. We know that $[\![a]\!] = [\![b]\!]$. By Remark 3.11 about h_2, we know that $[\![a]\!] = h_2(\frac{[\![a]\!]^+}{1})$, and $[\![b]\!] = h_2(\frac{[\![b]\!]^+}{1})$. Thus $\frac{[\![a]\!]^+}{1} = \frac{[\![b]\!]^+}{1}$. By definition of h, we have $h([\![a]\!]^+) = h([\![b]\!]^+)$, and so by h's injectivity, $[\![a]\!]^+ = [\![b]\!]^+$. Thus $a \equiv_R^+ b$ as desired. □

The next two chapters will be dedicated to showing the braid monoid satisfies the cancellative Ore conditions. Once we have done so, we will be able to embed the braid monoid (no scare quotes now!) into the braid group. Then Theorem 3.14 can be used to state that if two positive braid words are equivalent, they are also equivalent in the braid monoid.

Theorem 3.15. For positive $a, b \in BW$ such that $a \equiv_{R_B} b$, we have $a \equiv_{R_{B^+}}^+ b$.

That is, there exists a proof of their equivalence in which undercrossings never appear.[15]

[14] Note that in our implementation we would need to use `free_group_set_of_function`, and thus we more precisely assume $a \equiv_{\texttt{free_group_set_of_function}(R)} b$ instead of the looser $a \equiv_R b$.

[15] This is not a simple application of Theorem 3.14. Our formalizations of the relations R_B and R_{B^+} do not match correctly – we do not know that `free_group_set_of_function` applied to our implementation of R_{B^+} (recall, this is the simplified $R^S_{B^+}$) equals our implementation of R_B. That is, we do not have `free_group_set_of_function (braid_monoid_rels_inf) = braidRelationInf`. As an instance of an Artin-Tits relation, R_B (`braidRelationInf`) is symmetric and contains the element 1. Our direct implementation of $R^s_{B^+}$ (`braid_monoid_rels_inf`) is not symmetric and contains no occurrence of reflexivity (which would be used to generate the identity element for R_B). Thus, we must implement a non-simplified monoid relation R_M as follows.

Definition 3.16. We implement the (infinite) braid monoid relation R_M by the following :

```
inductive braid_monoid_rels_inf_one_refl_symm : FreeMonoid → FreeMonoid → Prop
  | adjacent (i j : ) (h : i.dist j = 1) :
      braid_monoid_rels_inf_one_refl_symm (of i * of j * of i) (of j * of i * of j)
  | separated (i j : ) (h : i.dist j  2) :
      braid_monoid_rels_inf_one_refl_symm (of i * of j) (of j * of i)
  | basic : braid_monoid_rels_inf_one_refl_symm 1 1
```

Note that the `basic` case could be replaced by any instance of reflexivity.

Lemma 3.17. We have our desired condition that
`free_group_set_of_function (braid_monoid_rels_inf_one_refl_symm) = braidRelationInf`.

The following (and final) theorem is not necessary for our project, but is a nice consequence of the above discussion :

Theorem 3.19. Let $\langle S \,|R\rangle$ be a left-cancellative monoid satisfying the Ore conditions. Let c be an element of $\langle S \,|R\rangle$. Then for some $a, b \in \langle S \,|R\rangle^+$, $c = (\iota(b))^{-1}\iota(a)$.

Proof. By surjectivity of h_2, we know that there is some $x \in \mathsf{PML}(S, R)$ such that $h_2(x) = c$. By definition of PML, $x = \frac{r}{s}$ for $r, s \in \langle S \,|R\rangle^+$. We claim that $c = (\iota(s))^{-1}\iota(r)$.

$$
\begin{aligned}
c &= h_2\left(\frac{r}{s}\right) \\
&= h_2\left(\frac{1 \cdot r}{1 \cdot s}\right) \\
&= h_2\left(\frac{1}{s} \bullet \frac{r}{1}\right) && \text{letting } a = 1 \text{ and } b = 1 \text{ in definition 3.4 of } \bullet \\
&= h_2\left(\left(\frac{s}{1}\right)^{-1} \bullet \frac{r}{1}\right) && \text{definition of inverse in 3.7} \\
&= \left(h_2\left(\frac{s}{1}\right)\right)^{-1} \cdot h_2\left(\frac{r}{1}\right) && h_2 \text{ is a homomorphism} \\
&= \left(h_2\left(h(s)\right)\right)^{-1} \cdot h_2\left(h(r)\right) && \text{definition of } h \\
&= \left(\iota(s)\right)^{-1} \iota(r) && \text{definition of } \iota
\end{aligned}
$$

□

Thus, specialized to braids, we see that any braid is equivalent to one in positive-negative form (that is, all overcrossings appearing before any undercrossings).

4 Common Multiples

In this chapter, we show the first of the two required conditions of Ore localization : common multiples. The proof for the common-multiples property follows Dehornoy exactly.[16] Here we give a brief sketch – if no proof is alluded to, it is by an inductive argument.

Definition 4.1. The $\sigma_{i,j}$ braid monoid word represents the ith strand moving to position j. Given $i, j \in \mathbb{N}$, we define

$$
\sigma_{i,j} = \begin{cases} \varepsilon & \text{if } i = j, \\ \sigma_i \cdot \sigma_{i+1,j} & \text{if } i < j, \\ \sigma_{i-1} \cdot \sigma_{i-1,j} & \text{if } i > j. \end{cases}
$$

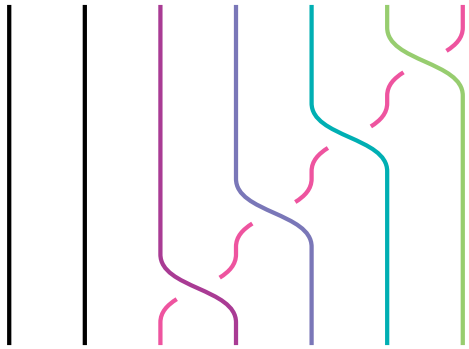
For example, $\sigma_{6,2} = \sigma_5\sigma_4\sigma_3\sigma_2$. As a physical braid,[17] we see the strand in position 6 (remember, we are 0-indexing) moving to position 2 :

Lemma 3.18. The presented monoids with generating relations R_M and $R^S_{B^+}$ are isomorphic.

Thus, we may return to Theorem 3.14 and safely apply it in this case.

[16] Dehornoy distinguishes between the braid monoid *words* $\underline{\sigma}_{i,j}$ and $\underline{\Delta}_n$ and the braid monoid *elements* $\sigma_{i,j}$ and Δ_n respectively. Here, for simplicity's sake, we speak only of the braid monoid words and omit their underlines.

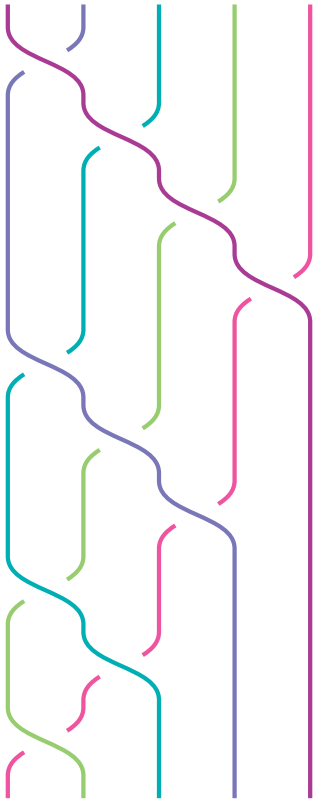
[17] We can only make this geometric connection once the localization procedure has been completed and we have an embedding of the braid monoid into the braid group



Definition 4.2. The Δ_n braid monoid word represents a half twist of the first $n+1$ strands in the braid. We define

$$\Delta_0 = \varepsilon$$
$$\Delta_{n+1} = \sigma_{0,n+1}\ \Delta_n$$

For example, $\Delta_4 = \sigma_0\sigma_1\sigma_2\sigma_3\sigma_0\sigma_1\sigma_2\sigma_0\sigma_1\sigma_0$.



We next note that any generator can be completed into a delta braid of sufficiently large index :

Theorem 4.3. For $n, i \in \mathbb{N}$ such that $0 < i < n$, there exists a value in BW^+, which we will denote $\partial_n(i)$, such that $\sigma_i \partial_n(i) \equiv^+ \Delta_n$.

Thus, for any $k \in \mathbb{N}$ with $k > i$ and $k > j$, Δ_k a common multiple for σ_i and σ_j.

Definition 4.4. The flip function f_n is defined on generators of BW^+ by $f_n(\sigma_i) = \sigma_{n-1-i}$ (where $i < n$). We may lift f_n to a word in BW^+ (bounded by n) by applying f_n to each of the word's generators. By abuse of notation, we denote this also by f_n.

The flip function helps us move generators, and hence entire words in BW^+, past a half-twist.

Lemma 4.5. For any $a \in BW^+$ strictly bounded above by n, $a\Delta_n \equiv^+ \Delta_n f_n(a)$.

We will use the flip function in the witness to the following theorem:

Theorem 4.6. Let $a \in BW^+$ have length at most ℓ. Let the set of indices of the generators in a be strictly bounded above by n. Then there exists a word $p_{\ell,n}(a)$ such that $a\, p_{\ell,n}(a) \equiv^+ (\Delta_n)^l$.

Proof. (Sketch) Let $a = \sigma_{a_1}\sigma_{a_2}...\sigma_{a_{\ell-1}}\sigma_{a_\ell}$. Define

$$p_{\ell,n}(a) = p_{\ell,n}(\sigma_{a_1}\sigma_{a_2}...\sigma_{a_{\ell-1}}\sigma_{a_\ell}) = \partial_n(a_\ell)\ f_n(\partial_n(a_{\ell-1}))\ \partial_n(a_{\ell-2})\ f_n(\partial_n(a_{\ell-3}))...^{18}$$

For example, when $\ell = 3$,

$$p_{3,n}(a) = p_{3,n}(\sigma_{a_1}\sigma_{a_2}\sigma_{a_3}) = \partial_n(a_3)\ f_n(\partial_n(a_2))\ \partial_n(a_1)$$

.

To show correctness of this particular witness, we calculate

$$\begin{aligned}
a\, p_{3,n}(a) &= \sigma_{a_1}\sigma_{a_2}\sigma_{a_3}\partial_n(a_3)\ f_n(\partial_n(a_2))\ \partial_n(a_1) \\
&\equiv^+ \sigma_{a_1}\sigma_{a_2}\Delta_n\ f_n(\partial_n(a_2))\ \partial_n(a_1) && \text{by Lemma 4.3} \\
&\equiv^+ \sigma_{a_1}\sigma_{a_2}\ \partial_n(a_2)\Delta_n\ \partial_n(a_1) && \text{by Lemma 4.5} \\
&\equiv^+ \sigma_{a_1}\Delta_n\Delta_n\ \partial_n(a_1) && \text{by Lemma 4.3} \\
&= \sigma_{a_1}\Delta_n\Delta_n\ f_n(f_n(\partial_n(a_1))) && f_n \text{ is an involution} \\
&\equiv^+ \sigma_{a_1}\ \Delta_i\ f_n(\partial_i(a_1))\Delta_n && \text{by Lemma 4.5} \\
&\equiv^+ \sigma_{a_1}\ \partial_n(a_1)\ \Delta_n\Delta_n && \text{by Lemma 4.5} \\
&\equiv^+ \Delta_n\Delta_n\Delta_n && \text{by Lemma 4.3} \\
&= (\Delta_n)^3
\end{aligned}$$

The full proof of correctness proceeds by induction on ℓ. □

Theorem 4.7. (Right Common Multiples) For all a and b in BW^+, there exist c and d in BW^+ such that $ac \equiv^+ bd$.

Proof. Set ℓ as the maximum of the lengths of a and b. Fix some $n \in \mathbb{N}$ strictly greater than the index of the largest generator occurring in either a or b. Then by Theorem 4.6 we have

$$a\, p_{\ell,n}(a) \equiv^+ (\Delta_n)^\ell \equiv^+ b\, p_{\ell,n}(b)$$

so $p_{\ell,n}(a)$ and $p_{\ell,n}(b)$ are the desired witnesses. □

While not needed for Ore localization, showing left common multiples is simple, thanks to the reversing function r introduced in Lemma 2.23.

[18] The ellipses do not continue on indefinitely. Writing it in this manner avoids casework on the parity of l, which determines whether the flip function is applied to $\partial_i(a_0)$)

Theorem 4.8. (Left Common Multiples) For all a and b in BW^+, there exist c and d in BW^+ such that $ca \equiv^+ db$.

Proof. Apply Theorem 4.7 to $r(a)$ and $r(b)$ to obtain c' and d' with $r(a) \cdot c' \equiv^+ r(b) \cdot d'$. Then use $r(c')$ and $r(d')$ as our witnesses. We aim to show that $r(c') \cdot a \equiv^+ r(d') \cdot b$. By Lemma 2.23, it suffices to show that $r(r(c') \cdot a) \equiv^+ r(r(d') \cdot b)$. But by definition of r, this simplifies to $r(a)\ c' \equiv^+ r(b)\ d'$, which we have.

□

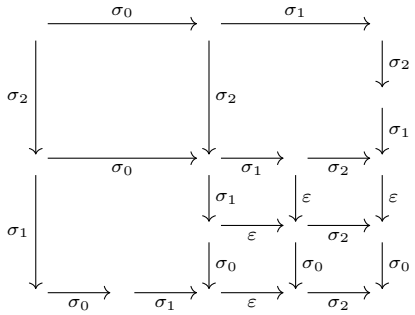
We have thus shown the first of two Ore conditions for the braid monoid. Cancellativity is a bit trickier; it is proved using an alternative, diagrammatic representation of equivalence in the braid monoid, called a grid. This structure will be discussed in the following chapter.

5 Grids

We next prove the remaining condition for Ore localization – cancellativity of the braid monoid. To do so, following Dehornoy, we model braid monoid equivalence using rectangular grid diagrams[19]. Later, these grids will be given further interpretation and reused in the proofs of both correctness and termination of the subword reversing algorithm.

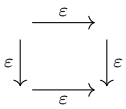
5.1 Definition and Basic Properties

Here is an example of a grid, with source $(\sigma_2\sigma_1, \sigma_0\sigma_1)$ (seen down the left side and across the top side) and target $(\sigma_0\sigma_1\sigma_2, \sigma_2\sigma_1\sigma_0)$ (seen across the bottom side and down the right side). We will see that the labels on all paths from top-left to bottom-right are equivalent in the braid monoid.

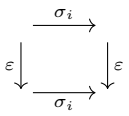


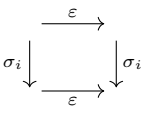
Intuitively, a grid is defined inductively as either a cell, or two grids abutted horizontally or vertically so that the relevant edges match. We give the data for a grid as a 4-tuple of braid monoid words, listing out the left, top, bottom, and right sides. We will later see that this uniquely determines the inner edges of the grid. We begin with the definition of cells:

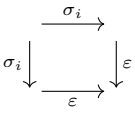
Definition 5.1. The set of **cells** C is the smallest subset of $(BW^+)^4$ such that:

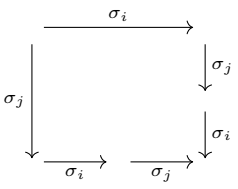
- $\langle \varepsilon, \varepsilon, \varepsilon, \varepsilon \rangle \in C$. Visually, 

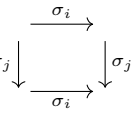
[19] Grids are a type of van Kampen diagram, planar diagrams which exhibit the fact that a given word in a presented group is equivalent to the identity element. Van Kampen diagrams were first introduced by their namesake in 1933 [25]. A full exposition can be found in the classic text by Lyndon and Schupp [28], which Nicholas Touikan summarizes in a visual, accessible manner in his online textbook [37].

- $\langle \varepsilon, \sigma_i, \sigma_i, \varepsilon \rangle \in C$ for any $i \in \mathbb{N}$. Visually, 

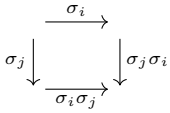
- $\langle \sigma_i, \varepsilon, \varepsilon, \sigma_i \rangle \in C$ for any $i \in \mathbb{N}$. Visually, 

- $\langle \sigma_i, \sigma_i, \varepsilon, \varepsilon \rangle \in C$ for any $i \in \mathbb{N}$. Visually, 

- For $i, j \in \mathbb{N}$ with $|i - j| = 1$, $\langle \sigma_j, \sigma_i, \sigma_i\sigma_j, \sigma_j\sigma_i \rangle \in C$. Visually, 

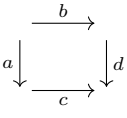
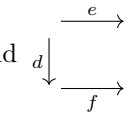
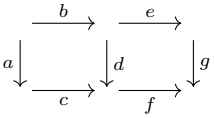
- For $i, j \in \mathbb{N}$ with $|i - j| > 1$, $\langle \sigma_j, \sigma_i, \sigma_i, \sigma_j \rangle \in C$. Visually, 

Often we simplify our diagrams combining adjacent arrows and labels. This does not change the cell or structure; it is merely notation. Thus, for the second-to-last case of the above definition, we could draw:

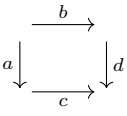
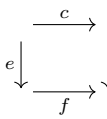
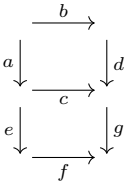


Definition 5.2. The set of **grids** G is defined inductively as follows

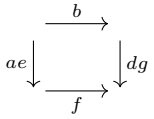
- Any cell is a grid; $C \subseteq G$
- Horizontal abutting: If $\langle a, b, c, d \rangle, \langle d, e, f, g \rangle \in G$, then $\langle a, be, cf, g \rangle \in G$.

  Visually, if  and  are grids, then so too is 

- Vertical abutting: If $\langle a, b, c, d \rangle, \langle e, c, f, g \rangle \in G$, then $\langle ae, b, f, dg \rangle \in G$.

  Visually, if  and  are grids, then so too is 

We may also draw a grid omitting its inner arrows, simply showing the four sides and the words labelling them. Later we will prove a uniqueness property for grids (Theorem 5.9), so this representation does not omit unrecoverable information. The last-drawn grid would thus be



Definition 5.3. Given a grid $\langle a, b, c, d\rangle \in G$, its **source** (later also called spine) is (a, b) and its **target** is (c, d)

For example, the above grid has source (ae, b) and target (f, dg).

Theorem 5.4. If $\langle a, b, c, d\rangle \in G$, $ac \equiv^+ bd$.

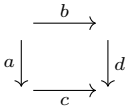
Proof. By structural induction on the grid. □

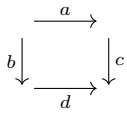
Definition 5.5. Given a grid $\langle a, b, c, d\rangle \in G$, the **length of its diagonal** is the length of ac.

Remark. The length of a grid's diagonal is also equal to $|bd|$, as $ac \equiv^+ bd$ by Theorem 5.4 and equivalent words have the same length by Definition 2.20.

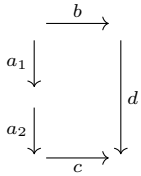
We next amass simple properties of grids that will be helpful in proving more complex theorems. All proofs are via structural induction on the given grid.

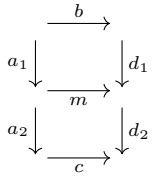
Lemma 5.6. If $\langle a, b, c, d\rangle \in G$, then $\langle b, a, d, c\rangle \in G$.

Visually, given 

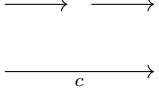
, we also have 

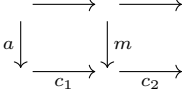
Lemma 5.7. (Splitting) Given $\langle a_1a_2, b, c, d\rangle \in G$, there exist grids $\langle a_1, b, m, d_1\rangle$ and $\langle a_2, m, c, d_2\rangle$ with $d = d_1d_2$.

That is, 

can be split horizontally into two subgrids as so: 

Similarly, given $\langle a, b_1b_2, c, d\rangle \in G$, there exist grids $\langle a, b_1, c_1, m\rangle$ and $\langle m, b_2, c_2, d\rangle$ with $c = c_1c_2$.

That is, 

can be split vertically into two subgrids so: 

Note that Lemma 5.6 allows vertical splitting to be derived from horizontal splitting.

Lemma 5.8 (Determinative Spines)**.** For braid monoid words a, b, c, d and natural numbers i, j,

- If $\langle \varepsilon, b, c, d\rangle \in G$, then $c = b$ and $d = \varepsilon$.
- If $\langle a, \varepsilon, c, d\rangle \in G$, then $c = \varepsilon$ and $d = a$.
- If $\langle a, a, c, d\rangle \in G$, then $c = \varepsilon$ and $d = \varepsilon$.
- If $\langle \sigma_i, \sigma_j, c, d\rangle \in G$ with $|i - j| = 1$, then $c = \sigma_j\sigma_i$ and $d = \sigma_i\sigma_j$.
- If $\langle \sigma_i, \sigma_j, c, d\rangle \in G$ with $|i - j| > 1$, then $c = \sigma_j$ and $d = \sigma_i$.

Theorem 5.9. (Uniqueness of sinks) Any two grids with the same source must have the same sink.

Proof. (Sketch) Consider two grids $g_1 = \langle a, b, c, d\rangle$ and $g_2 = \langle a, b, c', d'\rangle$. We show $c = c'$ and $d = d'$ by induction on the g_1. The base cases are shown by determinative spines, and the inductive cases split g_2 and apply the inductive hypothesis to the resulting pieces. □

5.2 Stability

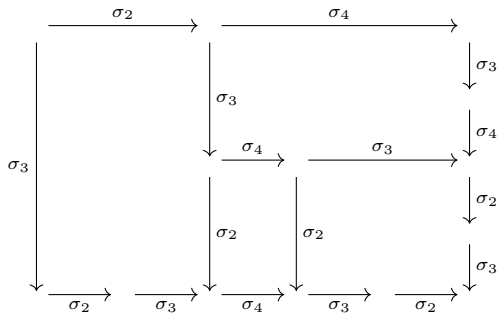
We define a property of grids called stability which will be essential to build grids from monoid equivalence, and thus to use grid properties to show cancellativity in the braid monoid.

Definition 5.10 (Stability)**.** A grid $\langle a, b, c, d\rangle$ is called **stable** if for all $a', b' \in BW^+$ such that $a \equiv^+ a'$ and $b \equiv^+ b'$, there exist $c', d' \in BW^+$ such that we have a grid $\langle a', b', c', d'\rangle$ with $c \equiv^+ c'$ and $d \equiv^+ d'$

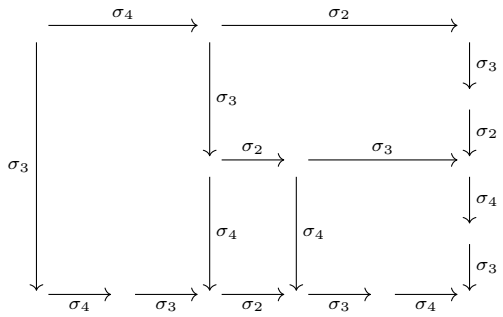
The Lean proof that all grids are stable is done by a triple induction, first on the length of the grid's diagonal. We then induct structurally on the equivalence between a and a' and then that of b and b'. Dehornoy, instead of these two structural inductions, defines a length metric on derivations witnessing equivalence and inducts on the sum of the lengths of the respective derivations associated with $a \equiv a'$ and $b \equiv b'$. At this stage of our Lean proof, we have kept a very simple definition of a derivation that is `Prop`-valued in Lean (it carries only the fact that a derivation exists, not the data of the derivation itself) and thus cannot recurse on the derivation's construction to calculate its length. Instead, we induct structurally, one derivation at a time. This does not increase the total number of cases significantly. We separate out a large portion of the proof into the following lemma:

Lemma 5.11. Let $(w, x) \in R_{B^+}$ and $k \in \mathbb{N}$. All grids having source (σ_k, w) or (σ_k, x) are stable.

Proof. (sketch) We discuss the proof for (σ_k, w); the proof for (σ_k, x) proceeds nearly identically. Consider an arbitrary grid $g = \langle \sigma_k, w, c, d\rangle$. We proceed by casework on w, and then on the distances between k and each of the indices appearing in w. For example, let w be a commutative relation of the form $\sigma_i\sigma_j$ with $|i - j| \geq 2$, with k such that $|i - k| = 1$ and $|j - k| = 1$. For clarity in the following diagram, we instantiate this with $i = 2$, $j = 4$, $k = 3$. We know there exists a grid $g' = \langle \sigma_3, \sigma_2\sigma_3, \sigma_2\sigma_3\sigma_4\sigma_3\sigma_2, \sigma_3\sigma_4\sigma_2\sigma_3\rangle$, as evidenced by the diagram below which constructs it from single cells :



Thus, by uniqueness of sinks (Theorem 5.9) applied to g and g', we must have $c = \sigma_2\sigma_3\sigma_4\sigma_3\sigma_2$ and $d = \sigma_3\sigma_4\sigma_2\sigma_3$. We consider a' and b' such that $\sigma_3 \equiv^+ a'$ and $\sigma_2\sigma_4 \equiv^+ b'$. By Lemma 2.24, $a' = \sigma_3$. By Lemma 2.25, either $b' = \sigma_2\sigma_4$ or $b' = \sigma_4\sigma_2$. In the first case, we take $c' = c$ and $d' = d$. We use the original grid $\langle \sigma_3, \sigma_2\sigma_4, c, d\rangle$ and quickly finish the proof as the required equivalences are true by reflexivity. In the second case, we take $c' = \sigma_4\sigma_3\sigma_2\sigma_3\sigma_4$ and $d' = \sigma_3\sigma_2\sigma_4\sigma_3$ and the following grid:



A quick calculation verifies

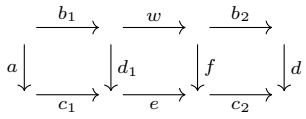
$$\begin{aligned} c &= \sigma_2\sigma_3\sigma_4\sigma_3\sigma_2 \\ &\equiv^+ \sigma_2\sigma_4\sigma_3\sigma_4\sigma_2 \\ &\equiv^+ \sigma_4\sigma_2\sigma_3\sigma_4\sigma_2 \\ &\equiv^+ \sigma_4\sigma_2\sigma_3\sigma_2\sigma_4 \\ &\equiv^+ \sigma_4\sigma_3\sigma_2\sigma_3\sigma_4 \\ &= c' \end{aligned}$$

A similar calculation verifies $d \equiv^+ d'$ and completes the proof. □

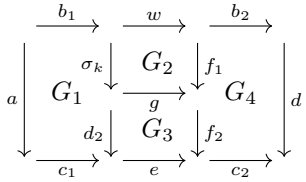
Theorem 5.12. All grids are stable.

For the proof of stability, we present one illustrative case, drawn from the inductive case of the first induction (on diagonal length). Our inductive hypothesis says that for some l, we know that all grids with length strictly less than l are stable. We consider an arbitrary grid $\langle a, b, c, d\rangle$ with diagonal length l.

We select the case where $a \equiv^+ a$ is true by reflexivity (so $a = a'$), and where we are in the one-step reduction case of the induction on $b \equiv^+ b'$. Thus, we assume that $b = b_1 w b_2$ and $b' = b_1 x b_2$, where $(w, x) \in R_{B^+}$. Splitting the grid vertically, we obtain



where $c = c_1 e c_2$. We then consider the case where d_1 has length at least one. We write $d_1 = \sigma_k d_2$, and obtain the following grid and label the subgrids as follows (using capital letters for readability):



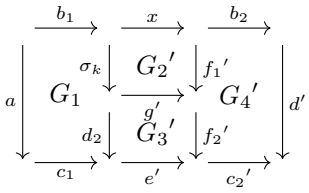
Note that $f = f_1 f_2$. The length of G_3 is $|d_2 e|$. Calculating, we see that we may apply the induction hypothesis to G_3 :

$$|d_2 e| < |\sigma_k d_2 e| \leq |b_1 \sigma_k d_2 e c_2| = |a c_1 e c_2| = |ac| = l$$

G_3 is hence stable, and a similar argument shows that G_4 is as well.[20] By Lemma 5.11, we know that G_2 is also stable.

Unfortunately, lemmas showing preservation of stability under appending are as difficult to prove as Theorem 5.12 itself, as they require inductions on equivalences. Instead, we proceed manually, calculating the required grids and equivalences for this specific case.

First, by stability of G_2, we have ${G_2}'$ with source (σ_k, x) and target $g', {f_1}'$ where $g \equiv^+ g'$ and $f_1 \equiv^+ {f_1}'$. By stability of G_3, we have ${G_3}'$ with source (d_2, g') and target $(e', {f_2}')$ where $e \equiv^+ e'$ and $f_2 \equiv^+ {f_2}'$. We then consider G_4. Note that $f_1 f_2 \equiv^+ {f_1}'{f_2}'$. By stability of G_4, we have a grid ${G_4}'$ with source $({f_1}'{f_2}', b_2)$ and target $({c_2}', d')$ with ${c_2}' \equiv^+ c_2$ and $d' \equiv^+ d$. We build the following grid:



We have just seen from the stability of G_4 that $d \equiv^+ d'$. We must show that ${c_1}'e'{c_2}' \equiv^+ c$. This is immediate from the facts given in the previous paragraph.

As we have seen in the above example, the proof can branch at the induction on the length and the inductions on the equivalences $a \equiv^+ a'$ and $b \equiv^+ b'$, and the lemma can branch at all the combinatorial options for the distance a single generator's index can lie in relation to the indices of elements of the braid monoid relations. Luckily, we do not have worst-case branching behavior, which would lead to a bit more than 100 cases. Our lemma has fourteen cases, and the main proof has twelve, for a total of 26. This is approximately the same as Dehornoy's original proof.

Theorem 5.13. There exists a grid with source (a, b) and target $(\varepsilon, \varepsilon)$ if and only if $a \equiv^+ b$.

Proof. The forward direction is immediate by Theorem 5.4. For the backwards direction, assume that $a \equiv^+ b$. It is quick to show by induction on a that there is a grid $\langle a, a, \varepsilon, \varepsilon \rangle$. By Theorem 5.12, $\langle a, a, \varepsilon, \varepsilon \rangle$ is stable, so we know that there is a grid $\langle a, b, c', d' \rangle$ where $c' \equiv^+ \varepsilon$ and $d' \equiv^+ \varepsilon$. Hence $c' = \varepsilon$ and $d' = \varepsilon$ (length is preserved under braid monoid equivalence), so we have a grid $\langle a, b, \varepsilon, \varepsilon \rangle$ as desired.

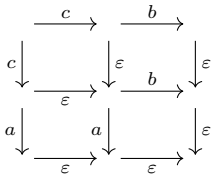
□

We are now ready to prove cancellativity.

5.3 Cancellativity

Theorem 5.14. The braid monoid is left-cancellative; that is, if $ca \equiv^+ cb$, then $a \equiv^+ b$.

Proof. Assume $ca = cb$. Then by Theorem 5.13, there is a grid $\langle ca, cb, \varepsilon, \varepsilon \rangle$. By repeated applications of splitting and determinative spines, the grid can be written as such:



[20] Note that it is entirely possible that the diagonal lengths of G_1 and G_2 may equal l

The lower right subgrid gives us $\langle a, b, \varepsilon, \varepsilon \rangle \in G$. Again by Theorem 5.13, we know $a \equiv^+ b$. □

Theorem 5.15. The braid monoid is right-cancellative; that is, if $ac \equiv^+ bc$, then $a \equiv^+ b$. This is proved from left-cancellativity using a trick with the reversing function similar to that of Theorem 4.8.

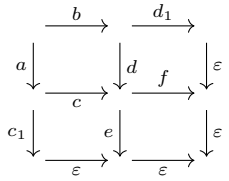
Thus, we have shown the necessary criteria to prove Theorem 3.15, and may conclude that for positive $a, b \in BW$ such that $a \equiv_{R_B} b$, we have $a \equiv^+_{R_{B^+}} b$.

5.4 Grid Existence

Beyond cancellativity, stability (via Theorem 5.13) is also used to show the existence of grids.

Theorem 5.16. For any $a, b \in BW^+$ there are $c, d \in BW^+$ such that there exists a grid with source (a, b) and target (c, d).

Proof. Consider some $a, b \in BW^+$. We know there exist c_1, d_1 such that $ac_1 \equiv^+ bd_1$ by Theorem 4.7. Then by Theorem 5.13 there is a grid $\langle ac_1, bd_1, \varepsilon, \varepsilon \rangle$. Then by repeatedly splitting that grid, there must exist $c, d, e, f \in BW^+$ and the following grid:



The top-left grid gives us $\langle a, b, c, d \rangle \in G$ as desired. □

6 Grids and Rewriting

We have used grids fruitfully in the proof of the cancellativity in the braid monoid, as part of the localization procedure connecting the braid monoid to the braid group. This portion of the project is now complete.

We find continued use for grids in the next stage of the project, in which we discuss the reversing procedure, as informally discussed in the introduction. Recall the reversing relation :

Definition 6.1. The reversing relation R_{rev} is the smallest relation such that

- For all $i \in \mathbb{N}$, $(\overline{\sigma_i}\, \sigma_i, \epsilon) \in R_{\mathsf{rev}}$
- For all $i, j \in \mathbb{N}$ such that $|i - j| = 1$, $(\overline{\sigma_i}\, \sigma_j, \sigma_j \sigma_i \overline{\sigma_j}\, \overline{\sigma_i}) \in R_{\mathsf{rev}}$
- For all $i, j \in \mathbb{N}$ such that $|i - j| > 1$, $(\overline{\sigma_i}\, \sigma_j, \sigma_j \overline{\sigma_i}) \in R_{\mathsf{rev}}$

Note that the reversing relation preserves equivalence under the braid relations – a quick induction shows the following :

Theorem 6.2. For any $x, y \in BW$, if $x \xrightarrow[R_{\mathrm{rev}}]{*} y$, then $x \equiv y$ in the braid group.

We aim to connect reversing to grids in the following manner:

Theorem 6.3. There exists a grid $\langle a, b, c, d \rangle$ if and only if $\overline{a}b \xrightarrow[R_{\mathrm{rev}}]{*} c\overline{d}$.

We will thus be able to use properties of grids (such as existence and uniqueness) to prove properties of our reversing algorithm (such as, respectively, termination and correctness). In fact it turns out that the above theorem is not quite as precise as we will need for termination. We will need to define size metrics on Semi-Thue derivations and on grids to eventually provide a natural-number (and hence well-founded) measure that strictly decreases as we progress through our algorithm. We first present our proof of the theorem in its original form, and then augment it with the size metric in the following chapter. The astute reader will note that the linked code for this section occasionally contains material regarding size (in Lean, a suffix of the form `.length` for derivations and grids), and certain code structure (such as `Type` in place of `Prop`). This may be safely ignored for now, and will be discussed in detail in the following chapter.

Returning to Theorem 6.3 as we have presented it here, the forward direction is nearly immediate by induction on grids:

Theorem 6.4. If there exists a grid $\langle a, b, c, d\rangle$, then $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$.

The reverse poses a much thornier problem. Patrick Dehornoy returned to the issue time and again. In one of his earliest papers, he took a different approach to showing termination for the subword reversing algorithm.

In his article "*Deux propriétés des groupes de tresses*" from 1992 [12], Dehornoy created an extended alphabet which grouped together letters from the original alphabet into single letters in the new alphabet. For example, he added to our S^* the letters $A = \sigma_1\sigma_2$ and $B = \sigma_2\sigma_1$. This was designed so that any application of reversing did not change the length of the word in the new alphabet. Thus, we see that $\overline{\sigma_1}\sigma_2 \xrightarrow[R_{\text{rev}}]{*} \sigma_2\sigma_1\overline{\sigma_2}\,\overline{\sigma_1} = B\overline{A}$, and so the length in the new alphabet remains 2. He carefully defines the new alphabet and shows that it is closed under applying reversing rewrites to negative-positive pairs. Thus, we have termination for reversing over the new alphabet. He then shows that this implies termination for reversing over the original alphabet.

I have not attempted to implement this proof, and so I cannot speak to its rigor. It appears to be a solid, reliable proof. Nevertheless, there is still interest in the grid-based proof, due to its clarity of exposition and connection with other mathematical structures such as van Kampen diagrams. Clearly, Dehornoy preferred the grid representation, as nearly all of his following papers on the matter (approximately 10) discuss grids instead of the extended alphabet. He had a certain freedom in experimenting with different, intuitive options for grids, knowing that he had shown termination by a different method earlier.

In his 2019 textbook, he gives the following sketch [variable names changed to match ours] :

> ... when $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$, consider a finite sequence of reversing steps leading from $\overline{a}b$ to $c\overline{d}$. Let Γ_0 be the diagram obtained by writing vertically and from top to bottom the letters of a and horizontally from left to right those of b starting from a common summit. Then following the sequence of elementary reversings starting with $\overline{a}b$ corresponds to constructing step by step from Γ_0 a grid Γ with source (a, b) and target (c, d). CITE

The sketch is compelling – one can easily envision the grid being built step-by-step. Earlier papers of Dehornoy's which discuss reversing grids give additional ideas. In a 2011 paper [15], he begins with the case of general van Kampen diagrams [21]:

> Lemma 1.1 (folklore). If (S, R) is a semigroup presentation, then two words w and w' of S^* are R-equivalent if and only if there exists an (S, R)-van Kampen diagram for (w, w').

[21] There is another, well-established method of constructing general van Kampen diagrams (using a balloon diagram; see https://ntouikan.ext.unb.ca/MATH6022/IntroCGGT/html_output/section-14.html), which differs from Dehornoy's description

Proof (sketch). If $(w_0, ..., w_p)$ is an R-derivation from w to w', then drawing paths labeled with the successive words w_i one below the other and identifying the unchanged letters yields a van Kampen diagram for (w, w'). Conversely, if K is a van Kampen diagram for (w, w'), one obtains an R-derivation from w to w' by enumerating the labels in a sequence of paths from the source of K to its target that differ by one face at a time.

Again, this feels intuitively justifiable, but does not suffice for a rigorous proof. In that same article (written before the textbook came out), he also gives a description of how one might extend this method to rectangular grids, using dotted lines to identify edges that would have touched in a regular van Kampen diagram (essentially, ε-labelled edges in a grid, but missing their orientation).[22] The below figure is from page 7 of Dehornoy's 2011 "*The Subword Reversing Method*" :

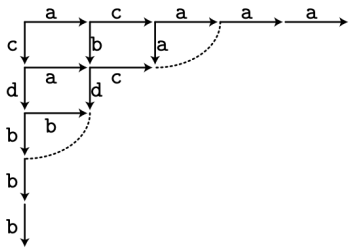


FIGURE 4. Reversing diagram associated with the last diagram in Figure 3: the only difference is that we insist that all edges are horizontal oriented to the right or vertical oriented to bottom. Again we are stuck as there is no relation c... = d... in the presentation.

In 2006, Dehornoy returned to the idea of the extended alphabet, and showed grids where each cell edge corresponded to a letter of the extended alphabet [18]. The below figure is from page 10 of Dehornoy and Wiest's "*On Word Reversing in Braid Groups*" :

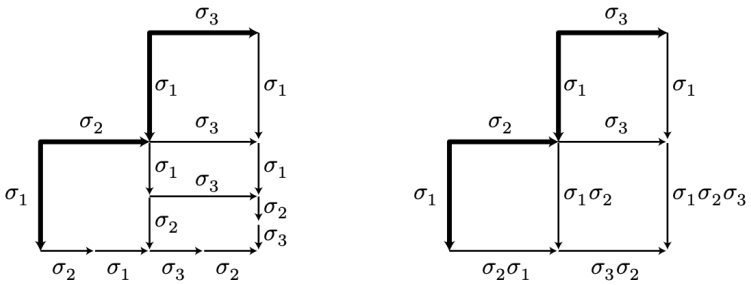


FIGURE 5. The reversing diagram (left) and the reversing grid (right) associated with the sequence $\sigma_1^{-1}\sigma_2\sigma_1^{-1}\sigma_3 \curvearrowright \sigma_2\sigma_1\sigma_2^{-1}\sigma_1^{-1}\sigma_1^{-1}\sigma_3 \curvearrowright \sigma_2\sigma_1\sigma_2^{-1}\sigma_1^{-1}\sigma_3\sigma_1^{-1} \curvearrowright \sigma_2\sigma_1\sigma_2^{-1}\sigma_3\sigma_1^{-1}\sigma_1^{-1} \curvearrowright \sigma_2\sigma_1\sigma_3\sigma_2\sigma_3^{-1}\sigma_2^{-1}\sigma_1^{-1}\sigma_1^{-1}$: one draws a zigzag path labelled by the initial word, and, then, one iteratively fills the open top-left corners using the braid relations.

This approach is tempting. However, it was not immediately applicable to our algorithm's termination, as we are working over the original alphabet. The proof translating termination from one alphabet to the other is non-trivial, and a metric defined on the extended-alphabet grid would not increase with each one of our individual rewrites. In any case, Dehornoy's sketch of extended-alphabet grids would have needed a more detailed treatment for formalization, and would have still

[22] Dehornoy had also used the curved lines for ε-labelled edges (no arrows, unsigned) in a paper from 2000. [14].

involved the two largest parts of our argument in this chapter : translating one rewriting sequence for another and a definition of a grid-under-construction. In some sense, such extended-alphabet grids are less satisfying, as they hide many steps of the algorithm, all combined into a single cell.

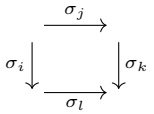
Returning to our definition of grids, my early naive attempts based on keeping track of the cells filled in unfortunately fail to capture the complexity of the construction required. It certainly feels so simple, so intuitive, and just out of reach. Below, we give a novel (to the best of my knowledge) proof of this claim. The proof defines two intermediate structures: grid-style rewriting and partial grids. First, we convert the normal rewriting to grid-style rewriting. Then we show grid-style rewriting corresponds to a partial grid. Finally, we connect the idea of partial grids to regular grids.

6.1 Reading Grids Backwards

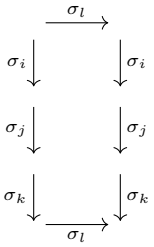
We begin by enlarging our definition of grids. At the moment, we have only discussed reading words off grids following the direction of the arrows. We can expand our interpretation of a grid structure by defining a procedure for reading *against* the direction of an arrow.

Instead of the labels on arrows representing elements of BW^+ (the set of braid monoid words), we now allow them to represent elements of BW (the set of braid group words). We note that we are discussing words, not elements of the braid group. To make this clear, following Dehornoy, we continue to write $\overline{\sigma_i}$ in place of σ_i^{-1}. Notably, $\overline{\sigma_i}\sigma_i \neq \varepsilon$, for they are clearly different strings. However, $\overline{\sigma_i}\sigma_i \equiv \varepsilon$.

We obtain such $\overline{\sigma_i}$ from the grid by reading against the direction of the arrows. Thus, we can read two paths from the bottom-left to the top-right in the following grid. The upper path is denoted $\overline{\sigma_i}\,\sigma_j$ and the lower is $\sigma_l\overline{\sigma_k}$.



For a side labelled with a word of length > 1, we read the letters of the word in opposite order, and add an overbar to each letter. Consider the following grid:

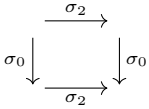
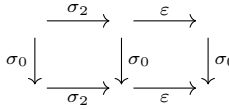


The upper bottom-left to top-right path would be read as $\overline{\sigma_k}\,\overline{\sigma_j}\,\overline{\sigma_i}\sigma_l$, and the lower $\sigma_l\overline{\sigma_k}\,\overline{\sigma_j}\,\overline{\sigma_i}$. Given a word $w = \sigma_{a_0}\sigma_{a_1}...\sigma_{a_n}$, let $\overline{w}$ denote $\overline{\sigma_{a_n}}\,...\overline{\sigma_{a_1}}\,\overline{\sigma_{a_0}}$

Notably, we have only given examples of reading a downwards-pointing arrow upwards. We could read left-to-right arrows backwards as well, using the same procedure. However, we will not find a use for this in the following algorithm.

In Lean, we implement such words as elements of `List (Nat x Bool)`. For example, `[(4, false), (3, true)]` represents $\overline{\sigma_4}\,\sigma_3$. Note that we choose `List` over `FreeMonoid` – this is a nod to Theorem 6.3, which discusses Semi-Thue rewriting, not presented monoids.

6.2 Empty Arrows

Recall that we omitted ε-labelled arrows in our definition of grids. We did not distinguish between the two following grids:  and  . For our proof of theorem 6.3, the more granular detail proof provided by tracking empty arrows will prove crucial. So, we again add more detail to the definition of our grid.

We add a new symbol $\mathbb{1}$ to our alphabet. $\mathbb{1}$ represents an ε-arrow.

Definition 6.5. The **augmented braid generators** are given by the set $S^{\mathbb{1}} := \{\mathbb{1}\} \cup \bigcup_{i\in\mathbb{N}}\{\sigma_i\}$

The **augmented braid alphabet** is the set

$$\mathscr{A}^{\mathbb{1}} := S^{\mathbb{1}} \cup \overline{S^{\mathbb{1}}} = \{\mathbb{1}, \overline{\mathbb{1}}\} \cup \bigcup_{i\in\mathbb{N}}\{\sigma_i, \overline{\sigma_i}\}$$

The set of **augmented braid words** $BW^{\mathbb{1}}$ is given by

$$BW^{\mathbb{1}} := \left(\mathscr{A}^{\mathbb{1}}\right)^* = \left(\{\mathbb{1}, \overline{\mathbb{1}}\} \cup \bigcup_{i\in\mathbb{N}}\{\sigma_i, \overline{\sigma_i}\}\right)^*$$

A subset of $BW^{\mathbb{1}}$, the set of **positive augmented braid words** $BW^{\mathbb{1},+}$ is given by

$$BW^{\mathbb{1},+} := \left(S^{\mathbb{1}}\right)^*$$

Remark. We occasionally find use for the terminology "negative augmented braid word", which is an element of $\left(\overline{S^{\mathbb{1}}}\right)^*$.

In Lean, $BW^{\mathbb{1}}$ is represented by the type `List (Option ℕ × Bool)`. The `Bool` keeps track of whether there is an overline on the generator; a value of `none` for `Option ℕ` indicates the generator is a $\mathbb{1}$, and a value of `some n` for `Option ℕ` means the generator is σ_n.

Definition 6.6. We define the following functions to map between BW and $BW^{\mathbb{1}}$:

$i^{\mathbb{1}} : BW \to BW^{\mathbb{1}}$ is simply the inclusion map.

$s^{\mathbb{1}} : BW^{\mathbb{1}} \to BW$ is the lift of

$$f(x) = \begin{cases} \sigma_i & \text{if } x = \sigma_i, \\ \varepsilon & \text{if } x = \mathbb{1}. \end{cases}$$

Lemma 6.7. The function $s^{\mathbb{1}}$ is a left-inverse of $i^{\mathbb{1}}$.

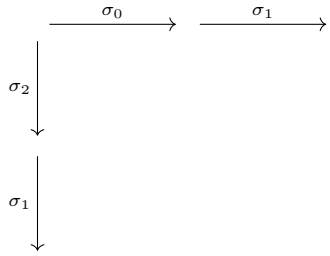
6.3 An Example

Recall the part of Theorem 6.3 to be proved : if $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$, then there exists a grid $\langle a, b, c, d\rangle$. To motivate the definitions and other decisions in the upcoming proof, we present an illustrative example. Let $a = \sigma_2\sigma_1$, $b = \sigma_0\sigma_1$, $c = \sigma_0\sigma_1\sigma_2$, and $d = \sigma_2\sigma_1\sigma_0$. Consider the following derivation,

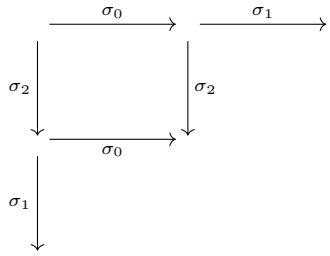
one of many which witnesses $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$:

$$
\begin{aligned}
\overline{\sigma_2\sigma_1}\sigma_0\sigma_1 &= \overline{\sigma_1}\,\overline{\sigma_2}\sigma_0\sigma_1 \\
&\to_{R_{\text{rev}}} \overline{\sigma_1}\,\sigma_0\overline{\sigma_2}\sigma_1 && (1)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\overline{\sigma_0}\,\overline{\sigma_1}\,\overline{\sigma_2}\sigma_1 && (2)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\overline{\sigma_0}\,\overline{\sigma_1}\sigma_1\sigma_2\overline{\sigma_1}\,\overline{\sigma_2} && (3)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\overline{\sigma_0}\sigma_2\overline{\sigma_1}\,\overline{\sigma_2} && (4)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\sigma_2\overline{\sigma_0}\,\overline{\sigma_1}\,\overline{\sigma_2} && (5)\\
&= \sigma_0\sigma_1\sigma_2\overline{\sigma_2\sigma_1\sigma_0}
\end{aligned}
$$

We aim to construct a grid $\langle \sigma_2\sigma_1, \sigma_0\sigma_1, \sigma_0\sigma_1\sigma_2, \sigma_2\sigma_1\sigma_0 \rangle$: $\begin{array}{ccc} \cdot & \xrightarrow{\sigma_0\sigma_1} & \cdot \\ {\scriptstyle \sigma_2\sigma_1}\downarrow & & \downarrow{\scriptstyle \sigma_2\sigma_1\sigma_0} \\ \cdot & \xrightarrow[\overline{\sigma_0\sigma_1\sigma_2}]{} & \cdot \end{array}$. We begin by writing out the spine of the grid :

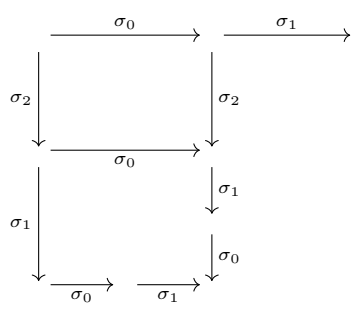


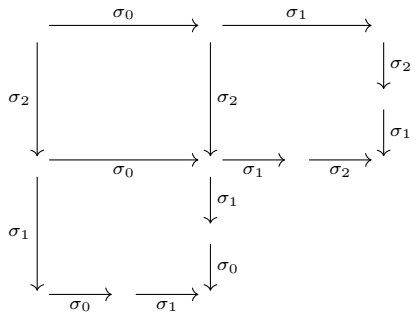
The first rewrite of $\overline{\sigma_2}\sigma_0$ to $\sigma_0\overline{\sigma_2}$ corresponds to inserting the cell $\begin{array}{ccc} \cdot & \xrightarrow{\sigma_0} & \cdot \\ {\scriptstyle \sigma_2}\downarrow & & \downarrow{\scriptstyle \sigma_2} \\ \cdot & \xrightarrow[\sigma_0]{} & \cdot \end{array}$:

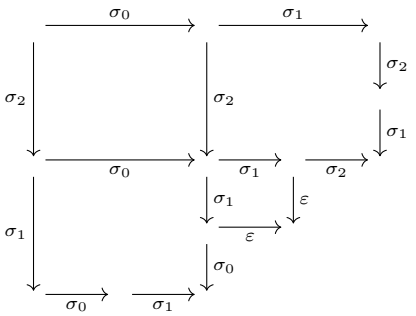


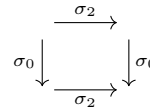
We define the frontier of a partially-completed grid as its southeasternmost bottom-left-to-top-right path. The frontier of the above diagram is $\overline{\sigma_1}\sigma_0\overline{\sigma_2}\sigma_1$, exactly the same as step (1) of the reversing derivation.

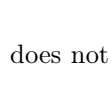
This procedure continues easily for the next three steps of the derivation, lines 2 through 4, with corresponding cells fitting into the frontier of diagram neatly:

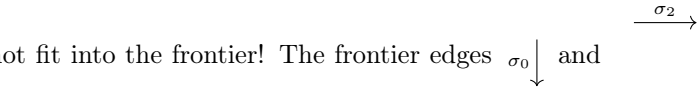


,


, and


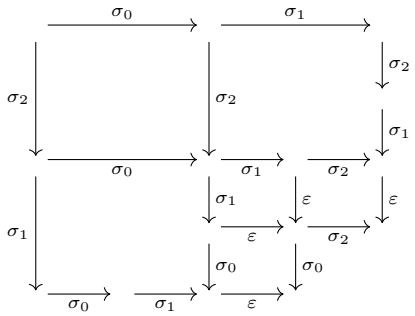
We encounter an issue when moving to line 5. There, we replace $\overline{\sigma_0}\sigma_2$ with $\sigma_2\overline{\sigma_0}$. The corresponding cell is 

. But this does not fit into the frontier! The frontier edges 

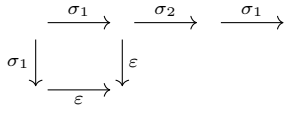
and 

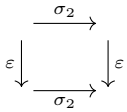
are separated by ε-labelled edges.

To insert this cell, we must first add in the following two cells (which we will call *trivial* cells), which do not correspond to a step of the derivation :

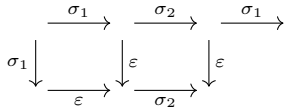


Further, note that adding in trivial cells must be done recursively. To see why, consider the following snippet of a grid :



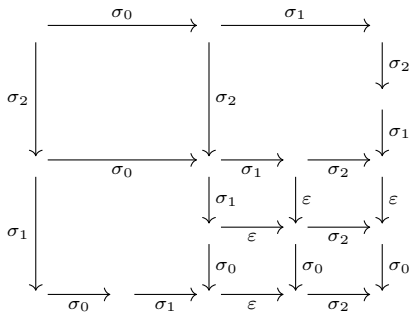
When we add in the appropriate trivial cell 

we obtain the following grid snippet:



This snippet now requires another trivial cell to be inserted.

Returning to our main example, we may now complete the grid following the rewrite in step (5), obtaining



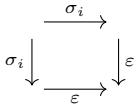
This has frontier $\sigma_0\sigma_1\mathbb{1}\sigma_2\overline{\sigma_0}\,\overline{\mathbb{1}}\,\overline{\sigma_1}\,\overline{\sigma_2}$. Note that $s^{\mathbb{1}}(\sigma_0\sigma_1\mathbb{1}\sigma_2\overline{\sigma_0}\,\overline{\mathbb{1}}\,\overline{\sigma_1}\,\overline{\sigma_2}) = \sigma_0\sigma_1\sigma_2\overline{\sigma_0}\,\overline{\sigma_1}\,\overline{\sigma_2}$, which equals step (5) of the derivation.

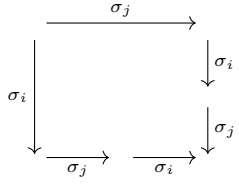
To give a rigorous proof, we must first give a definition of partially-constructed grids. Then, we must develop a procedure for inserting trivial cells. We *could* enact this procedure on the partial grids. It turns out to be simpler to translate the given derivation to one tracking ε's explicitly, and then have a clean, straightforward translation from the new derivation to partial grids. This is largely due to strings being simpler to work with than two-dimensional diagrams. We define this new derivation in the following section.

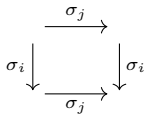
6.4 Grid-Style Rewriting

We define a relation R_{grid} which tracks ε-labelled arrows, which are represented by $\mathbb{1}$ in our alphabet. This relation is defined to correspond neatly with grid cells; if we have $R_{\mathsf{grid}}(x, y)$, we know there is a cell with x as the left-top border and y as the bottom-right border (reading, as always in the section, north-east, and using an overbar to denote travelling opposite an arrow's direction).

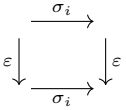
Definition 6.8. The relation R_{nontriv} is very similar to R_{rev}. It differs only in the first case (see below); rewriting under R_{rev} simply deletes an occurrence of $\overline{\sigma_i}\sigma_i$, whereas under R_{nontriv}, $\overline{\sigma_i}\sigma_i$ would be replaced by $\mathbb{1}\overline{\mathbb{1}}$. R_{nontriv} is the smallest relation satisfying

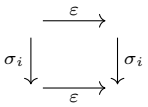
$R_{\mathsf{nontriv}}(\overline{\sigma_i}\sigma_i, \mathbb{1}\overline{\mathbb{1}})$ for all $i \in \mathbb{N}$. This corresponds to the cell 

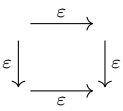
$R_{\mathsf{nontriv}}(\overline{\sigma_i}\sigma_j, \sigma_j\sigma_i\overline{\sigma_j}\,\overline{\sigma_i})$ when $|i-j| = 1$. This corresponds to the cell 

$R_{\mathsf{nontriv}}(\overline{\sigma_i}\sigma_j, \sigma_j\overline{\sigma_i})$ when $|i-j| \geq 2$. This corresponds to the cell 

The relation R_{triv} tracks "trivial" rewrites which do not occur in an R_{rev}– derivation. R_{triv} is the smallest relation satisfying

$R_{\mathsf{triv}}(\overline{\mathbb{1}}\sigma_i, \sigma_i\overline{\mathbb{1}})$ for all $i \in \mathbb{N}$. This corresponds to the cell 

$R_{\mathsf{triv}}(\overline{\sigma}_i\ \mathbb{1}, \mathbb{1}\ \overline{\sigma}_i)$ for all $i \in \mathbb{N}$. This corresponds to the cell 

$R_{\mathsf{triv}}(\overline{\mathbb{1}}\ \mathbb{1},\ \mathbb{1}\ \overline{\mathbb{1}})$. This corresponds to the cell 

Finally, R_{grid} is the union of R_{triv} and R_{nontriv}.

We next make precise the close relationship between R_{rev} and R_{nontriv} :

Theorem 6.9. If $R_{\mathsf{rev}}(x, y)$, there exists $y' \in BW^{\mathbb{1}}$ such that $R_{\mathsf{nontriv}}(i^{\mathbb{1}}(x), y')$. This y' happens to be unique, and have the property that $s^{\mathbb{1}}(y') = y$.

We next give terminology for augmented braid words which can or can not be reduced by the above relations.

Definition 6.10. An element $b \in BW^{\mathbb{1}}$ is **trivially-reducible** if it is of the form $b_1 x b_2$, where $(x, y) \in R_{\mathsf{triv}}$.

Otherwise, it is called **trivially-irreducible**. This is equivalent to saying b has no subwords of the form $\overline{\sigma_i}\mathbb{1}$, $\overline{\mathbb{1}}\sigma_i$, or $\overline{\mathbb{1}}\mathbb{1}$.

We have analogous definitions for nontrivially-reducible and nontrivially-irreducible, and R_{grid} - reducible and R_{grid} - irreducible.

Lemma 6.11. For any $b \in BW$, $i^{\mathbb{1}}(b)$ is trivially-irreducible.

We give a specific fact about irreducibility to be used in a later proof :

Lemma 6.12. If g is trivially-irreducible and $s^{\mathbb{1}}(g) = c\overline{d}$ for some $c, d \in BW^+$, then there exist $e, f \in BW^{\mathbb{1},+}$ such that $g = e\overline{f}$.

We name the important words of length two :

Definition 6.13. Given letters $\alpha, \beta \in S^{\mathbb{1}}$, a word of the form $\overline{\alpha}\beta$ is called an **open pair**. A word in which at least one of α or β equals $\mathbb{1}$ is called a **trivial open pair**. A word of the form $\overline{\sigma_i}\sigma_j$ is called a **sigma-sigma open pair**. We use the adjective "open" to signify there is an opening for a grid cell in the corresponding diagram.

We have the analogous vocabulary for pairs of the form $\alpha\overline{\beta}$, with "closed" replacing "open".

6.5 Building Grid-Style Rewriting

This section's main result is the generation of an R_{grid} derivation from an R_{rev} derivation :

Theorem 6.14. Let $a, b, c, d \in BW^+$. If $\overline{a}b \xrightarrow[R_{\mathrm{rev}}]{*} c\overline{d}$, then there exist positive augmented words e, f such that $i^{\mathbb{1}}(\overline{a})i^{\mathbb{1}}(b) \xrightarrow[R_{\mathrm{grid}}]{*} e\overline{f}$. Moreover, deleting all instances of $\mathbb{1}$ and $\overline{\mathbb{1}}$ from e leaves us with c; that is, $s^{\mathbb{1}}(e) = c$. Similarly, $s^{\mathbb{1}}(f) = d$.

Before we begin the proof, let us return to our example from earlier in the chapter. Recall the derivation:

$$
\begin{aligned}
\overline{\sigma_2\sigma_1}\sigma_0\sigma_1 &= \overline{\sigma_1}\,\overline{\sigma_2}\sigma_0\sigma_1 \\
&\to_{R_{\text{rev}}} \overline{\sigma_1}\,\sigma_0\overline{\sigma_2}\sigma_1 && (1)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\overline{\sigma_0}\,\overline{\sigma_1}\,\overline{\sigma_2}\sigma_1 && (2)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\overline{\sigma_0}\,\overline{\sigma_1}\sigma_1\sigma_2\overline{\sigma_1}\,\overline{\sigma_2} && (3)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\overline{\sigma_0}\sigma_2\overline{\sigma_1}\,\overline{\sigma_2} && (4)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\sigma_2\overline{\sigma_0}\,\overline{\sigma_1}\,\overline{\sigma_2} && (5)\\
&= \sigma_0\sigma_1\sigma_2\overline{\sigma_2\sigma_1\sigma_0}
\end{aligned}
$$

The following derivation will suffice for Theorem 6.14, with $e = \sigma_0\sigma_1\mathbb{1}\sigma_2$ and $f = \sigma_2\sigma_1\mathbb{1}\sigma_0$

$$
\begin{aligned}
\overline{\sigma_2\sigma_1}\sigma_0\sigma_1 &= \overline{\sigma_1}\,\overline{\sigma_2}\sigma_0\sigma_1 \\
&\to_{R_{\text{rev}}} \overline{\sigma_1}\,\sigma_0\overline{\sigma_2}\sigma_1 && (1)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\overline{\sigma_0}\,\overline{\sigma_1}\,\overline{\sigma_2}\sigma_1 && (2)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\overline{\sigma_0}\,\overline{\sigma_1}\sigma_1\sigma_2\overline{\sigma_1}\,\overline{\sigma_2} && (3)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\overline{\sigma_0}\,\mathbb{1}\overline{\mathbb{1}}\sigma_2\overline{\sigma_1}\,\overline{\sigma_2} && (4)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\mathbb{1}\overline{\sigma_0}\,\overline{\mathbb{1}}\sigma_2\overline{\sigma_1}\,\overline{\sigma_2} && (4.1)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\mathbb{1}\overline{\sigma_0}\sigma_2\,\overline{\mathbb{1}}\,\overline{\sigma_1}\,\overline{\sigma_2} && (4.2)\\
&\to_{R_{\text{rev}}} \sigma_0\sigma_1\mathbb{1}\sigma_2\overline{\sigma_0}\,\overline{\mathbb{1}}\,\overline{\sigma_1}\,\overline{\sigma_2} && (5)\\
&= \sigma_0\sigma_1\mathbb{1}\sigma_2\overline{\sigma_2\sigma_1\mathbb{1}\sigma_0}
\end{aligned}
$$

Here, the work lies in moving the $\mathbb{1}$s in lines (4) and (4.1). Then, we are relieved to see that in line (4.2), we are poised to make the rewrite in line (5).

Moving on from this example, we begin by defining a general procedure for performing the trivial reductions (i.e. moving the $\mathbb{1}$ and $\overline{\mathbb{1}}$s). The simplest way to define the procedure for trivial reduction is to repeatedly search for and rewrite the first occurrence of a trivial pair. We can give a lexicographic ordering on our alphabet, and in that way show this procedure does terminate.[23] However, we will need to prove a number of properties about this procedure's result (such as trivial irreducibility), so we need a definition more amenable to inductive proofs.

We first define a helper function `concatenate_reduction`, which appends a single letter of our alphabet $\mathscr{A}^{\mathbb{1}}$ to a word $b \in BW^{\mathbb{1}}$ and applies any trivial reductions involving said letter. We will in the future only apply this function to a trivially-irreducible b.

Definition 6.15. Given a single letter $\alpha \in \mathscr{A}^{\mathbb{1}}$ and a word $b \in BW^{\mathbb{1}}$, we define `concatenate_reduction a b` as

[23] For example, we might use the following well-founded relation :

- $\sigma_i < \sigma_j$ iff $i < j$
- $\overline{\sigma_i} < \overline{\sigma_j}$ iff $i < j$
- $\sigma_i < \overline{\sigma_j}$ for all natural numbers i, j
- $\mathbb{1} < \overline{\mathbb{1}}$
- $\sigma_i < \mathbb{1}$ for all natural numbers i
- $\overline{\mathbb{1}} < \overline{\sigma_i}$ for all natural numbers i

We extend this to the associated shortlex ordering over strings. Although this method was not ultimately used in the project, it did lead to the implementation of the shortlex order, now in Mathlib.

- if α is $\overline{\mathbb{1}}$ and $b = \sigma_i b_1$, return $\sigma_i \cdot$ (`concatenate_reduction a b`). (This case rewrites $\overline{\mathbb{1}}\sigma_i$ to $\sigma_i\overline{\mathbb{1}}$, and then continues on with any further trivial rewrites involving α)
- if α is negative (an element of $\overline{S^{\mathbb{1}}}$) and $b = \mathbb{1}b_1$, return $\mathbb{1} \cdot$ (`concatenate_reduction a b`). (In this case, $\alpha = \overline{\sigma_i}$ or $\alpha = \overline{\mathbb{1}}$. So, this case either rewrites $\overline{\sigma_i}\mathbb{1}$ to $\mathbb{1}\overline{\sigma_i}$ or $\overline{\mathbb{1}}\mathbb{1}$ to $\mathbb{1}\overline{\mathbb{1}}$, and then continues on with any further trivial rewrites involving α)
- otherwise, return αb

Lemma 6.16. If b is trivially-irreducible, so too is `concatenate_reduction a b`. The proof proceeds by induction on b, and then casework on α and the first element of b.

Theorem 6.17. $\alpha b \xrightarrow[R_{\text{triv}}]{*}$ `concatenate_reduction a b`. The proof proceeds in the same style as that of the above theorem.

We are now prepared to define the algorithm `move_ones` which trivially-reduces a given augmented braid word.

Definition 6.18. Given $b \in BW^{\mathbb{1}}$, we define `move_ones b` as follows :

- If b has length zero, then `move_ones b` returns the empty word ε.
- If b has length at least one, it has the form αb_1 for some $\alpha \in \mathscr{A}^{\mathbb{1}}$ and $b_1 \in BW^{\mathbb{1}}$. Let c be the result of the recursive call `move_ones b`. Then we return `concatenate_reduction a c`.

In Lean, this reads as

```
def move_ones (L : List (Option  × Bool)) :=
  match L with
  | [] => []
  | head :: tail => concatenate_reduction head (move_ones tail)
```

Theorem 6.19. $b \xrightarrow[R_{\text{triv}}]{*}$ `move_ones b`. The proof proceeds by induction, using Lemma 6.17. Notably, as $R_{\mathsf{triv}} \subseteq R_{\mathsf{grid}}$, $b \xrightarrow[R_{\text{grid}}]{*}$ `move_ones b`

Theorem 6.20. `move_ones` does not affect the relative positions of any σ_i. $s^{\mathbb{1}}$(`move_ones b`) $= s^{\mathbb{1}}(b)$

Theorem 6.21. `move_ones` produces a trivially-irreducible word; for any $b \in BW^{\mathbb{1}}$, `move_ones b` is trivially-irreducible.

Let us take stock of where we are in terms of our example. The `move_ones` algorithm effects the work done in steps (4) – (4.2). We next noted that we were poised for the next step from the R_{rev} - derivation, able to make the rewrite from step (5). The property of trivial-irreducibility just mentioned will in fact suffice, but not straightforwardly so. Instead, we introduce another notion which more immediately connects to preparedness for a nontrivial rewrite.

We begin by defining a property `pairAppears`.

Definition 6.22. An element $b \in BW^{\mathbb{1}}$ has the `pairAppears` property if for every substring $\overline{\sigma_i}\sigma_j$ appearing in $s^{\mathbb{1}}(b)$, $\overline{\sigma_i}\sigma_j$ is a substring of b.

This is not quite enough to define the desired property. For example, the string $\overline{\sigma_1}\,\overline{\mathbb{1}}\sigma_2\mathbb{1}\,\overline{\sigma_1}\sigma_2$ satisfies `pairAppears`, but the first occurrence of $\overline{\sigma_1}\sigma_2$ remains separated by a $\mathbb{1}$. We define a stronger condition based on `pairAppears` as follows:

Definition 6.23. An element $b \in BW^{\mathbb{1}}$ has the `pairsTogether` property if every substring of b satisfies `pairAppears`.

Showing that an augmented braid word is trivially-irreducible is an easy way to show that it satisfies `pairsTogether`, thanks to the following theorem :

Theorem 6.24. Any trivially-irreducible $b \in BW^{\mathbb{1}}$ satisfies `pairsTogether`

Proof. (Sketch) We show that any irreducible b satisfies `pairAppears` by induction on the length of b, and then casework on its first and second elements. Then we quickly show that any subword of a trivially-irreducible word is itself trivially-irreducible, and combine this fact with the definition of `pairsTogether` to obtain our conclusion. □

Note that `pairsTogether` is strictly weaker than trivial irreducibility. For example, $\sigma_2\overline{\mathbb{1}}\mathbb{1}$ satisfies `pairsTogether` vacuously, but is not trivially-irreducible.

We give two specific lemmas about open-pair infixes of trivially-irreducible strings, which will be used for the proof of Theorem 6.14 :

Lemma 6.25. Let $w \in BW^{\mathbb{1}}$ be trivially-irreducible, and assume $s^{\mathbb{1}}(w)$ has an infix of the form $\overline{\sigma_i}\sigma_j$ – more specifically, assume $s^{\mathbb{1}}(w) = c\overline{\sigma_i}\sigma_j d$ for $c, d \in BW^{\mathbb{1}}$. Then there exist $w_1, w_2 \in BW^{\mathbb{1}}$ such that $w = w_1\overline{\sigma_i}\sigma_j w_2$, $c = s^{\mathbb{1}}(w_1)$, and $d = s^{\mathbb{1}}(w_2)$.

Proof. (sketch) The proof proceeds by stacking induction on w with lemmas also proved by induction on w (for example, a lemma where "infix" is replaced by "prefix"), and critically relies on the trivial-irreducibility of w. Otherwise, we would only know that there exist $w_1, w_2, w_m \in BW^{\mathbb{1}}$ such that $w = w_1 w_m w_2$ with $s^{\mathbb{1}}(w_m) = \overline{\sigma_i}\sigma_j$. □

Lemma 6.26. Given trivially-irreducible $a, b \in BW^{\mathbb{1}}$ and any $c, d \in BW$, if $s^{\mathbb{1}}(a)\overline{\sigma_i}\sigma_j s^{\mathbb{1}}(b) = c\,\overline{\sigma_i}\sigma_j\,d$, then we have one of the following three cases, depending on if c is equal to, longer than, or shorter than $s^{\mathbb{1}}(a)$:

- Case 1 : $s^{\mathbb{1}}(a) = c$ and $s^{\mathbb{1}}(b) = d$.

$s^{\mathbb{1}}(a)$	$\overline{\sigma_i}\,\sigma_j$	$s^{\mathbb{1}}(b)$
c	$\overline{\sigma_i}\,\sigma_j$	d

- Case 2 : We have $b_1, b_2 \in BW^{\mathbb{1}}$ such that $b = b_1\overline{\sigma_i}\sigma_j b_2$, $c = s^{\mathbb{1}}(a)\overline{\sigma_i}\sigma_j s^{\mathbb{1}}(b_1)$, and $d = s^{\mathbb{1}}(b_2)$

		$\overbrace{\quad\quad\quad\quad\quad\quad\quad\quad}^{s^{\mathbb{1}}(b)}$		
$s^{\mathbb{1}}(a)$	$\overline{\sigma_i}\,\sigma_j$	$s^{\mathbb{1}}(b_1)$	$\overline{\sigma_i}\,\sigma_j$	$s^{\mathbb{1}}(b_2)$
c			$\overline{\sigma_i}\,\sigma_j$	d

- Case 3 : We have $a_1, a_2 \in BW^{\mathbb{1}}$ such that $a = a_1\overline{\sigma_i}\sigma_j a_2$, $c = s^{\mathbb{1}}(a_1)$, and $d = s^{\mathbb{1}}(a_2)\overline{\sigma_i\sigma_j}s^{\mathbb{1}}(b)$

$\overbrace{\quad\quad\quad\quad\quad\quad\quad\quad\quad}^{s^{\mathbb{1}}(a)}$				
$s^{\mathbb{1}}(a_1)$	$\overline{\sigma_i}\,\sigma_j$	$s^{\mathbb{1}}(a_2)$	$\overline{\sigma_i}\,\sigma_j$	$s^{\mathbb{1}}(b)$
c	$\overline{\sigma_i}\,\sigma_j$	d		

Proof. (sketch) Because $\overline{\sigma_i} \neq \sigma_j$, we know c cannot have length $s^{\mathbb{1}}(a) \pm 1$. This reduces our possible cases from five to three. Within each case, Lemma 6.25 does the majority of the work. □

With the necessary machinery set up, we prove a generalization of Theorem 6.14.

Theorem 6.27. Given $a, b \in BW$, if $a \xrightarrow[R_{\text{rev}}]{*} b$ then there exists a trivially-irreducible $g \in BW^{\mathbb{1}}$ such that $i^{\mathbb{1}}(a) \xrightarrow[R_{\text{rev}}]{*} g$ and $s^{\mathbb{1}}(g) = b$.

Proof. Assume $a \xrightarrow[R_{\text{rev}}]{*} b$. Thus, $a \underset{R_{\text{rev}}}{\twoheadrightarrow} b$. We proceed by induction on that derivation.

- Reflexivity : In this case, $a = b$. Let $g = i^{\mathbb{1}}(b)$. Then we have $i^{\mathbb{1}}(a) \xrightarrow[R_{\text{grid}}]{*} i^{\mathbb{1}}(b)$ by reflexivity. By Lemma 6.11, $i^{\mathbb{1}}(b)$ is trivially-irreducible, and by Lemma 6.7 $s^{\mathbb{1}}\left(i^{\mathbb{1}}(b)\right) = b$.
- Transitive one-step reduction : In this case, we have $c, d, x, y \in BW$ such that $a \underset{R_{\text{rev}}}{\twoheadrightarrow} cxd$, $R_{\mathsf{rev}}(x, y)$, and $b = cyd$. Our inductive hypothesis states that there is some trivially-irreducible $e \in BW^{\mathbb{1}}$ such that $i^{\mathbb{1}}(a) \underset{R_{\text{grid}}}{\twoheadrightarrow} e$ and $s^{\mathbb{1}}(e) = cxd$.

  As $R_{\mathsf{rev}}(x, y)$, $x = \overline{\sigma_i}\sigma_j$ for some i and j; moreover, there is some $y' \in BW^{\mathbb{1}}$ such that $R_{\mathsf{grid}}(x, y')$ holds by Theorem 6.9.

  Thus, $s^{\mathbb{1}}(e) = c\overline{\sigma_i}\sigma_j d$. As e is trivially-irreducible, it satisfies `pairsTogether` by Theorem 6.24. Hence $\overline{\sigma_i}\sigma_j$ is a substring of e. Let $e = w\overline{\sigma_i}\sigma_j t$. Substituting this in to the equation at the start of the paragraph, $s^{\mathbb{1}}(w\overline{\sigma_i}\sigma_j t) = c\overline{\sigma_i}\sigma_j d$. Simplifying, we obtain $s^{\mathbb{1}}(w)\overline{\sigma_i}\sigma_j s^{\mathbb{1}}(t) = c\overline{\sigma_i}\sigma_j d$.

  As w and t are both trivially-irreducible (being substrings of e), by Lemma 6.26 we have the following three cases:

  - $s^{\mathbb{1}}(w) = c$ and $s^{\mathbb{1}}(t) = d$. Let $g =$ `move_ones (w ++ y' ++ t)`. We know $i^{\mathbb{1}}(a) \underset{R_{\text{grid}}}{\twoheadrightarrow} wxt$, so since $R_{\mathsf{grid}}(x, y')$, $i^{\mathbb{1}}(a) \underset{R_{\text{grid}}}{\twoheadrightarrow} wy't$. Thus $i^{\mathbb{1}}(a) \xrightarrow[R_{\text{grid}}]{*} wy't$. By Theorem 6.19, $wy't \xrightarrow[R_{\text{grid}}]{*}$ `move_ones w ++ y' ++ t`. Then by transitivity, $i^{\mathbb{1}}(a) \xrightarrow[R_{\text{grid}}]{*}$ `move_ones w ++ y' ++ t`. We thus have $i^{\mathbb{1}}(a) \xrightarrow[R_{\text{grid}}]{*} g$ as desired. We know that $g =$ `move_ones w ++ y' ++ t` is trivially-irreducible by Theorem 6.21. Lastly,

    $$\begin{aligned} s^{\mathbb{1}}(g) &= s^{\mathbb{1}}(\texttt{move_ones w ++ y' ++ t}) \\ &= s^{\mathbb{1}}(wy't) && \text{(by Theorem 6.20)} \\ &= s^{\mathbb{1}}(w)s^{\mathbb{1}}(y')s^{\mathbb{1}}(t) \\ &= c \cdot s^{\mathbb{1}}(y') \cdot d \\ &= cyd && \text{(by Theorem 6.9)} \\ &= b \end{aligned}$$

    as desired
  - In this case, $t = t_1\overline{\sigma_i}\sigma_j t_2$, $c = s^{\mathbb{1}}(w)\overline{\sigma_i}\sigma_j s^{\mathbb{1}}$, and $d = s^{\mathbb{1}}(t_2)$:
    We use `move_ones (w ++ x ++ t1 ++ y' ++ t2)`; the proof proceeds from here analogously to that of the previous case.
  - The third and final case is entirely symmetrical with the previous

□

Finally, we give the proof of Theorem 6.14. Recall we must show that if $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$, then there exist $e, f \in BW^{\mathbb{1},+}$ such that $i^{\mathbb{1}}(\overline{a})i^{\mathbb{1}}(b) \xrightarrow[R_{\text{grid}}]{*} e\overline{f}$ with $s^{\mathbb{1}}(e) = c$ and $s^{\mathbb{1}}(f) = d$.

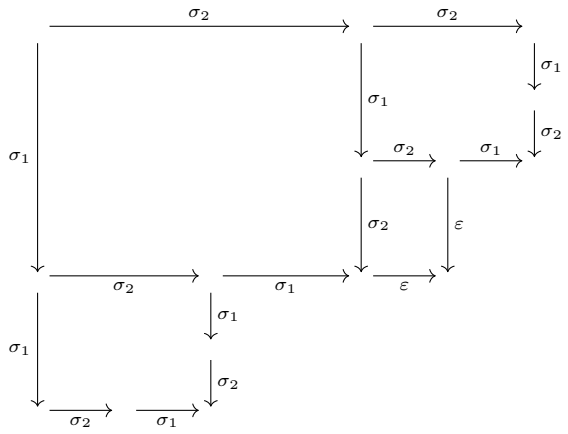
Proof. Let $a, b, c, d \in BW^+$. Assume $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$. By Theorem 6.27, there exists some trivially-irreducible $g \in BW^{\mathbb{1}}$ such that $i^{\mathbb{1}}(\overline{a}b) \xrightarrow[R_{\text{grid}}]{*} g$ and $s^{\mathbb{1}}(g) = c\overline{d}$. By Lemma 6.12, there exist positive $e, f \in BW^{\mathbb{1}}$ such that $g = e\overline{f}$. Thus, $i^{\mathbb{1}}(\overline{a})i^{\mathbb{1}}(b) \xrightarrow[R_{\text{grid}}]{*} e\overline{f}$.

Moreover, note that $s^{\mathbb{1}}(g) = s^{\mathbb{1}}(e\overline{f}) = s^{\mathbb{1}}(e)s^{\mathbb{1}}(\overline{f})$, and $s^{\mathbb{1}}(e)$ is positive and $s^{\mathbb{1}}(\overline{f})$ negative. Thus, as $s^{\mathbb{1}}(e)s^{\mathbb{1}}(\overline{f}) = c\overline{d}$, we must have $s^{\mathbb{1}}(e) = c$ and $s^{\mathbb{1}}(f) = d$ as desired. □

We now know that we may produce an R_{grid}–derivation from an R_{rev}–derivation. We next connect this R_{grid}–derivation to a grid under construction, towards our goal of building a grid $\langle a, b, c, d\rangle$.

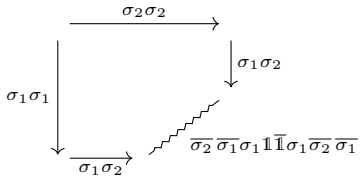
6.6 Partial Grids

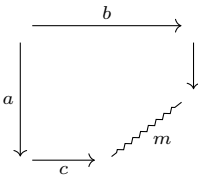
A partial grid generalizes the notion of a grid to include "unfinished" grids, which are missing cells in the bottom/right area. Let's use the following as a motivating example:



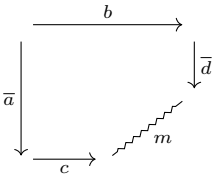
Partial grids will be defined as an inductive subset of $(BW^{\mathbb{1}})^5$. Before we give the formal definition, here is an intuitive definition to keep in mind. We define a partial grid only by its "frame" : the left side (read upwards against its direction), top side, the bottom frontier (the section that runs straight along the bottom), the middle frontier (the zigzag-like portion in the middle, read from southwest to northeast with downwards-pointing arrows read upwards against their direction), and the right frontier (the section which runs down the far-righthand side, read upwards against its direction).

The frame of the above grid is given by the tuple $\langle \overline{\sigma_1}\,\overline{\sigma_1}, \sigma_2\sigma_2, \sigma_2\sigma_1, \overline{\sigma_2}\,\overline{\sigma_1}\sigma_1, \mathbb{1}, \overline{\mathbb{1}}\sigma_1, \overline{\sigma_2}\,\overline{\sigma_1}\rangle$, seen visually as



In general, we read a partial grid 

as $\langle \overline{a}, b, c, m, \overline{d} \rangle$.

We draw a partial grid $\langle a, b, c, m, d \rangle$ as 

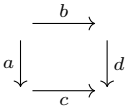
. We will soon show that both a and d must be negative augmented braid words, so $\overline{a}$ and $\overline{d}$ are positive augmented braid words (no generators with overlines appear in them).

Often, we speak of the (total) frontier of a partial grid; this is the concatenation of the bottom frontier, middle frontier, and right frontier. In our example, it would be $\sigma_1\sigma_2 \cdot \overline{\sigma_2}\,\overline{\sigma_1}\sigma_1 \mathbb{1}\overline{\mathbb{1}}\sigma_1\overline{\sigma_2}\,\overline{\sigma_1} \cdot \overline{\sigma_2}\,\overline{\sigma_1}$.

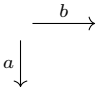
We now give a formal, inductive definition of the set of partial grids $G_P \subset (BW^{\mathbb{1}})^5$. We provide diagrams to guide the reader's intuition; they do not form part of the definition.

Definition 6.28. The set of partial grids G_P is the smallest subset of $(BW^{\mathbb{1}})^5$ satisfying

- **(a cell)** : For any $a, b, c, d \in BW^{\mathbb{1},+}$, if $(\overline{a}\,b,\ c\,\overline{d}) \in R_{\mathsf{grid}}$, we have $\langle \overline{a},\ b,\ c,\ \varepsilon,\ \overline{d} \rangle \in G_P$

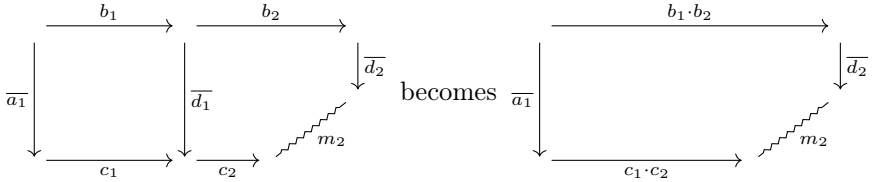
  Visually, 

- **(a spine)** : For any $a, b \in BW^{\mathbb{1},+} \setminus \{\varepsilon\}$, $\langle \overline{a},\ b,\ \varepsilon,\ \overline{a}\,b,\ \varepsilon \rangle \in G_P$

  Visually, 

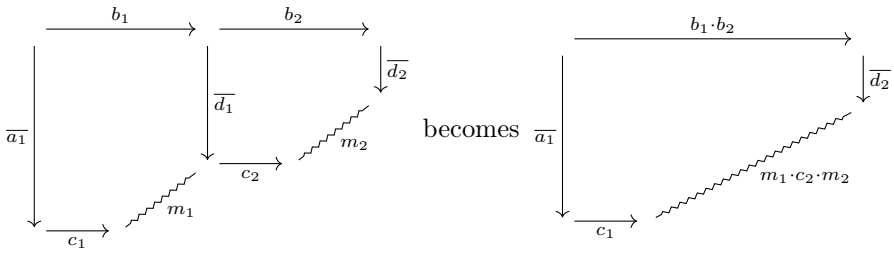
 for non-empty a and b

- **(horizontal concatenation to a partial grid with empty middle frontier)** :

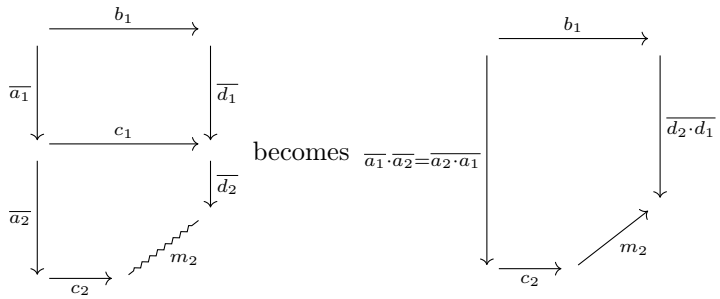
  Given $\langle a_1, b_1, c_1, \varepsilon, d_1 \rangle \in G_P$ and $\langle d_1, b_2, c_2, m_2, d_2 \rangle \in G_P$, we have $\langle a_1, b_1 \cdot b_2, c_1 \cdot c_2, m_2, d_2 \rangle \in G_P$.

- **(horizontal concatenation to a partial grid with non-empty middle frontier)** (m_1 is non-empty) :

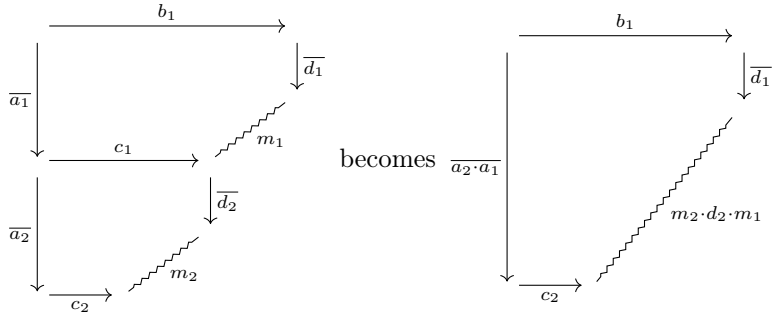
Given $\langle a_1, b_1, c_1, m_1, d_1\rangle \in G_P$ and $\langle d_1, b_2, c_2, m_2, d_2\rangle \in G_P$, we have $\langle a_1, b_1 \cdot b_2, c_1, m_1 \cdot c_2 \cdot m_2, d_2\rangle \in G_P$.

- **(vertical concatenation to a partial grid with empty middle frontier)** :



Given $\langle a_1, b_1, c_1, \varepsilon, d_1\rangle \in G_P$ and $\langle a_2, c_1, c_2, m_2, d_2\rangle \in G_P$, we have $\langle a_2 \cdot a_1, b_1, c_2, m_2, d_2 \cdot d_1\rangle \in G_P$. Note the order of the first element and the final element : $a_2 \cdot a_1$ and $d_2 \cdot d_1$. We are reading *against* the direction of the arrow on the vertical side of the partial grid, so the order is flipped.

- **(vertical concatenation to a partial grid with non-empty middle frontier)** (m_1 is non-empty) :



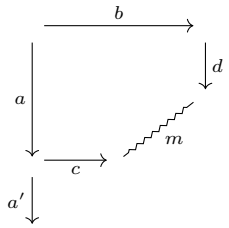
Given $\langle a_1, b_1, c_1, m_1, d_1\rangle \in G_P$ and $\langle a_2, c_1, c_2, m_2, d_2\rangle \in G_P$, we have $\langle a_2 \cdot a_1, b_1, c_2, m_2 \cdot d_2 \cdot m_1, d_1\rangle \in G_P$.

Definition 6.29. The **total frontier** (often referred to as merely "the frontier") of a partial grid $\langle a, b, c, m, d\rangle \in G_P$ is $c \cdot m \cdot d$.

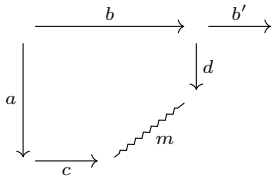
We continue with three short lemmas about partial grids, all shown by structural induction. While simple in theory, the number of cases grows rapidly.

Lemma 6.30. Consider some partial grid $\langle a, b, c, m, d\rangle \in G_P$. The left side ($a$) and right frontier ($d$) must be negative augmented braid words; the top side (b) and bottom frontier (c) must be positive augmented braid words.

Lemma 6.31. (Extending a partial grid's spine) Consider a partial grid $\langle a, b, c, m, d\rangle$. For any non-empty *negative* augmented grid word a' there exists a partial grid $\langle a' \cdot a, b, \varepsilon, a' \cdot c \cdot m, d\rangle$. Visually,



For any non-empty *positive* augmented grid word b', there exists a partial grid $\langle a, b \cdot b', c, m \cdot d \cdot b', \varepsilon \rangle$. Visually,

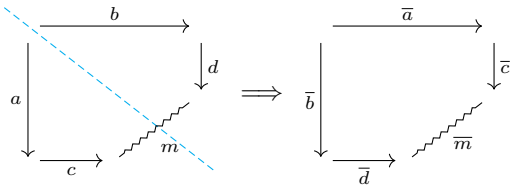


Lemma 6.32. The middle frontier of any partial grid is either empty or of the form $\overline{\alpha} m' \beta$ for some letters $\alpha, \beta \in S^{\mathbb{1}}$ and some $m' \in BW^{\mathbb{1}}$. Note that this is only true because we keep track of empty arrows in partial grids using $BW^{\mathbb{1}}$.

The following two theorems are not used in the proof of Theorem 6.3, but are basic properties of partial grids, so we present them now.

First, we can reflect a partial grid along its top-left to bottom-right diagonal.

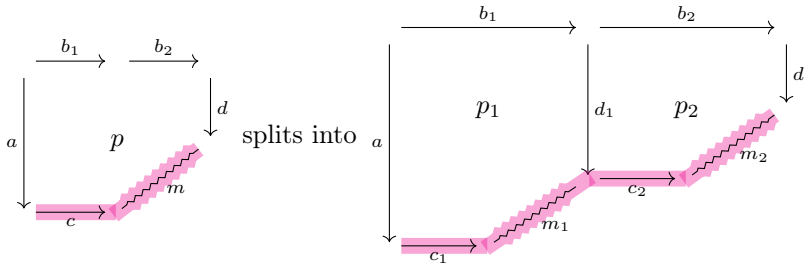
Theorem 6.33. (Reflecting) Given a partial grid $\langle a, b, c, m, d \rangle$, there is a partial grid $\langle \overline{b}, \overline{a}, \overline{d}, \overline{m}, \overline{c}, \rangle$



This theorem shortens proofs significantly; for example, the second half of Lemma 6.31 becomes an immediate consequence of the first half. We also use this mirroring trick in the following theorem about splitting partial grids:

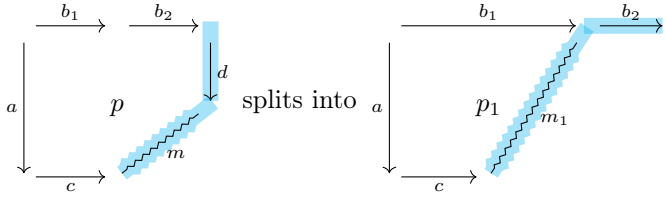
Theorem 6.34. (Vertical Splitting) Given $p = \langle a, b_1 b_2, c, m, d \rangle \in G_P$ with nonempty b_1, b_2, we have either

- p splits into two smaller partial grids $p_1 = \langle a, b_1, c_1, m_1, d_1 \rangle \in G_P$ and $p_2 = \langle d_1, b_2, c_2, m_2, d \rangle \in G_P$, with $c \cdot m = c_1 \cdot m_1 \cdot c_2 \cdot m_2$



(note that c is not necessarily equal to c_1, as m_1 could be empty, making $c = c_1c_2$. d may be either empty or nonempty)

- p splits into a smaller partial grid $p_1 = \langle a, b_1, c, m_1, \varepsilon\rangle \in G_P$ and a tail $\xrightarrow{b_2}$, with $m = m_1 \cdot b_2$. In this case, we must have $d = \varepsilon$, and so $m \cdot d = m_1 \cdot b_2$



The proof is by induction on p.

(Horizontal Splitting) We have the analogous property for a partial grid $p' = \langle a_1a_2, b, c, m, d\rangle$ with nonempty a_1, a_2. The proof reflects p', splits it vertically, and reflects the result(s).

6.7 Building the Partial Grid

We next build a partial grid from an R_{grid}–derivation. We may only do so when the derivation begins with a word in negative-positive form, as that will correspond to the spine of a partial grid.

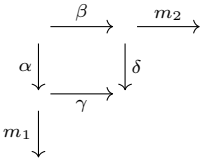
Theorem 6.35. For nonempty $a, b \in BW^{\mathbb{1},+}$, if $\overline{a}b \xrightarrow[R_{\text{grid}}]{*} c$, then there is a partial grid with spine (a, b) and frontier c.

We begin by defining a function called `add_cell` which adds a single cell to a partial grid.

Definition 6.36. `add_cell` is a function taking in some $(x, y) \in R_{\text{grid}}$ and a partial grid $P_1 = \langle a, b, c, m_1 \cdot x \cdot m_2, d\rangle$, and returning a partial grid $P_2 = \langle a, b, c', m', d'\rangle$ such that $c' \cdot m' \cdot d' = c \cdot m_1 \cdot y \cdot m_2 \cdot d$, c is a prefix of c', and d is a suffix of d'. The final two specifications are used only internally to the definition. By contrast, the first of the specifications (that $c' \cdot m' \cdot d' = c \cdot m_1 \cdot y \cdot m_2 \cdot d$) is both necessary for the definition and used externally for other theorems later in the project.

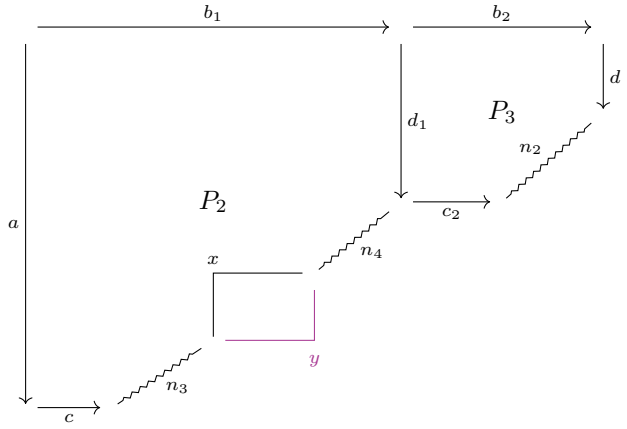
We define `add_cell` by recursion on P_1.

- **a cell** : This case would lead to a contradiction. Partial grids from the cell case have an empty middle frontier. As $(x, y) \in R_{\text{grid}}$, x is not the empty string, so P_1 has a nonempty middle frontier.
- **a spine** : Here $c = d = \varepsilon$ and $m_1 \cdot x \cdot m_2 = ab$. First, note that as $(x, y) \in R_{\text{grid}}$, there exist letters $\alpha, \beta, \gamma, \delta \in S^{\mathbb{1}}$ such that $x = \overline{\alpha}\beta$ and $y = \gamma\overline{\delta}$. Thus $m_1 \cdot \overline{\alpha}\beta \cdot m_2 = ab$. As a is negative and b is positive (by Lemma 6.30), we must have $a = m_1\overline{\alpha}$ and $b = \beta m_2$. As $(\overline{\alpha}\beta, \gamma\overline{\delta}) \in R_{\text{grid}}$, we have $P_3 = \langle\overline{\alpha}, \beta, \gamma, \varepsilon, \overline{\delta}\rangle \in G_P$. Consider the case where neither m_1 nor m_2 is the empty string (we omit the other, simpler cases). By Lemma 6.31, we may extend the left side of P_3 to obtain a partial grid $P_4 = \langle m_1\overline{\alpha}, \beta, \varepsilon, m_1\gamma, \overline{\delta}\rangle$, and then the right side of P_4 to obtain a partial grid $P_5 = \langle m_1\overline{\alpha}, \beta m_2, \varepsilon, m_1\gamma\overline{\delta}m_2, \varepsilon\rangle$. Hence, we have a partial grid $\langle a, b, c, m_1 \cdot y \cdot m_2, d\rangle$ as desired. The condition on the equality of the total frontier immediately holds. As $c = d = \varepsilon$ is both a prefix and a suffix of any word, those two conditions are also fulfilled.

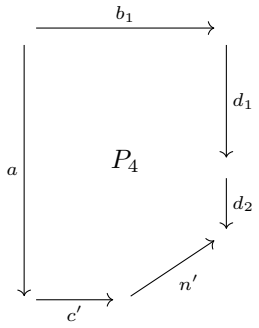


- **horizontal concatenation to a partial grid with non-empty middle frontier** :

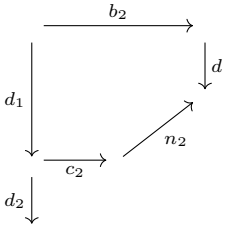
  In this case, we have $b = b_1 b_2$, partial grids $P_2 = \langle a, b_1, c, n_1, d_1 \rangle$ and $P_3 = \langle d_1, b_2, c_2, n_2, d \rangle$, with $n_1 \cdot c_2 \cdot n_2 = m_1 \cdot x \cdot m_2$. By properties of middle frontiers (Lemma 6.32), we know that either x is an infix of n_1 or of n_2. Consider the first case, where we have $n_1 = n_3 x n_4$, $m_1 = n_3$ and $m_2 = n_4 \cdot c_2 \cdot n_2$; the other case is analogous.



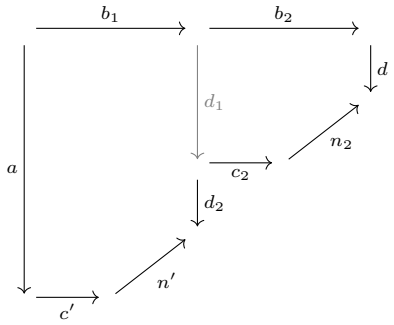
  Here, recursively call `add_cell` on P_2 to obtain a partial grid $P_4 = \langle a, b_1, c', n', d_1' \rangle$ with $c' \cdot n' \cdot d_1' = c \cdot n_3 \cdot y \cdot n_4 \cdot d_1$, c a prefix of c' and d_1 a suffix of d_1'. Since d_1 is a suffix of d_1', we know that there exists some d_2 such that $d_1' = d_1 d_2$. We consider the case in which d_2 and n' are nonempty (the other cases are simpler).



  We extend the left side of P_3 to obtain a partial grid $P_5 = \langle d_1', b_2, \varepsilon, d_2 c_2 n_2, d \rangle$.



  Next append P_4 and P_5 to obtain the desired partial grid $P_6 = \langle a, b_1 b_2, c', n' d_2 c_2 n_2, d \rangle$.



To show that the first specification holds, we must show $c' \cdot n' \cdot d_2 \cdot c_2 \cdot n_2 \cdot d = c \cdot m_1 \cdot y \cdot m_2 \cdot d$. Recall that our inductive hypothesis gave us $c' \cdot n' \cdot d_1' = c \cdot n_3 \cdot y \cdot n_4 \cdot d_1$. As $d_1' = d_2 \cdot d_1$, we have $c' \cdot n' \cdot d_2 = c \cdot n_3 \cdot y \cdot n_4$. Appending $c_2 \cdot n_2 \cdot d$ to both sides, we have $c' \cdot n' \cdot d_2 \cdot c_2 \cdot n_2 \cdot d = c \cdot n_3 \cdot y \cdot n_4 \cdot c_2 \cdot n_2 \cdot d$. In the case we have selected, we noted that $m_1 = n_3$ and $m_2 = n_4 \cdot c_2 \cdot n_2$. Thus, $c' \cdot n' \cdot d_2 \cdot c_2 \cdot n_2 \cdot d = c \cdot m_1 \cdot y \cdot m_2 \cdot d$, as desired. For the final two conditions, the inductive hypothesis states that c is a prefix of c', and d is immediately a suffix of d.

- The other horizontal case and the two vertical cases proceed similarly

We are now prepared to prove Theorem 6.35.

Proof. Consider nonempty, positive $a, b \in BW^{\mathbb{1}}$. Assume $\overline{a}b \xrightarrow[R_{\text{grid}}]{*} c$. Thus $\overline{a}b \underset{R_{\text{grid}}}{\twoheadrightarrow} c$. We proceed by induction on this derivation.

- **reflexivity :** In this case, we have $c = \overline{a}b$. Then we use the spinal partial grid given by a and b, $\langle \overline{a}, b, \varepsilon, \overline{a}b, \varepsilon \rangle$.
- **transitive one-step reduction :** We assume that $\overline{a}b \underset{R_{\text{rev}}}{\twoheadrightarrow} dxe$, $R_{\mathsf{rev}}(x, y)$, and $c = dye$. Our inductive hypothesis states that there is a partial grid $P_1 := \langle \overline{a}, b, c_1, m_1, d_1 \rangle$ with $c_1 m_1 d_1 = dxe$. We note that x must be an infix of m_1, as x is in negative-positive form, while c_1 is positive and d_1 negative. By the same reasoning, there exist $m_2, m_3 \in BW^{\mathbb{1}}$ such that $m_1 = m_2 x m_3$, $d = c_1 m_2$, and $e = m_3 d_1$. We call `add_cell` with (x, y) and P_1 to obtain $P_2 := \langle \overline{a}, b, c_1, m_2 y m_3, d_1 \rangle$. Then P_2's frontier is $c_1 m_2 y m_3 d_1 = dye = c$ as desired.

□

6.8 Empty Middle Frontiers

Finally, we connect partial grids to grids by proving that a partial grid with empty middle frontier is in fact a grid.

Theorem 6.37. Given $\langle \overline{a}, b, c, \varepsilon, \overline{d} \rangle \in G_P$, we have $\langle s^{\mathbb{1}}(a), s^{\mathbb{1}}(b), s^{\mathbb{1}}(c), s^{\mathbb{1}}(d) \rangle \in G$.

The proof is by induction on the given partial grid.

We next note that any partial grid whose total frontier is in positive-negative form has an empty middle frontier :

Lemma 6.38. If a partial grid has total frontier $c\overline{d}$ for some $c, d \in BW^{\mathbb{1},+}$, then it has bottom frontier c, middle frontier ε, and right frontier $\overline{d}$. (This is a consequence of Lemma 6.32)

6.9 Conclusion

We are now prepared to give a proof of the remaining half of Theorem 6.3, restated as follows :

Theorem 6.39. For any $a, b, c, d \in BW^+$, if $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$, then we have a grid $\langle a, b, c, d\rangle$.

Proof. Consider $a, b, c, d \in BW^+$. Assume $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$. By Theorem 6.27, we have $e, f \in BW^{\mathbb{1},+}$ such that $i^1(\overline{a})i^{\mathbb{1}}(b) \xrightarrow[R_{\text{grid}}]{*} e\overline{f}$ with $s^{\mathbb{1}}(e) = c$ and $s^{\mathbb{1}}(f) = d$. By Theorem 6.35, we have a partial grid P_1 with spine $(i^{\mathbb{1}}(\overline{a}), i^{\mathbb{1}}(b))$ and total frontier $e\overline{f}$. By Lemma 6.38, we have $P_1 = \langle i^{\mathbb{1}}(\overline{a}), i^{\mathbb{1}}(b), e, \varepsilon, \overline{f}\rangle$. Finally, by Theorem 6.37, we have a grid $\left\langle s^{\mathbb{1}}\big(i^{\mathbb{1}}(a)\big), s^{\mathbb{1}}\big(i^{\mathbb{1}}(b)\big), s^{\mathbb{1}}(e), s^{\mathbb{1}}(f)\big)\right\rangle = \langle a, b, c, d\rangle$ as required.

□

Theorem 6.3 can be used to give new, visual proofs of previous results. For example, note that properties of cancellativity and common multiples (the Ore conditions) are used for existence and uniqueness of grids, and hence for the proof of Theorem 6.3. Thus, grids essentially capture Ore localization for a small subset of braid words (for those pairs of braid words which are the source and target of some grid). We thus have a slick proof of the following :

Theorem 6.40. For any $a, b, c, d \in BW^+$, if $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$, then $ac \equiv^+ bd$.

Proof. Assume $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$. Then by Theorem 6.3, we have $\langle a, b, c, d\rangle \in G$. Hence by 5.4, $ac \equiv^+ bd$. □

Alternatively, we could have noted that since $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$, $\overline{a}b \equiv c\overline{d}$. In other notation, $a^{-1}b \equiv cd^{-1}$, and thus $ac \equiv bd$. As ac and bd are both positive braid words, by Ore localization (Theorem 3.15), we know $ac \equiv^+ bd$ as desired.

7 Metrics

This chapter introduces metrics on grids, partial grids, and Semi-Thue derivations. In the Lean formalization, doing so requires us to distinguish between two ways in which an inductively defined mathematical object may be given : as a proposition asserting that the object exists, or as data recording how the object was constructed. A proposition in Lean has type `Prop`, while mathematical data lives in `Type`. To disambiguate the two, we begin by giving simple examples of `Prop` and `Type`.

Suppose that we have a predicate $p : \mathbb{N} \to$ Prop. The proposition "There exists an n such that $p(n)$ holds" is written as `n : , p n`, which has type `Prop`. A proof of this proposition establishes that some natural number satisfying p exists. We can discuss an arbitrary such number using `Classical.choose` to give it a name, and `Classical.choose_spec` to assert that p holds of said named number. This selection uses the axiom of choice; `Classical.choose` establishes that a suitable n exists, but does not provide a computable procedure for finding it. Thus, a definition whose value depends on `Classical.choose` is marked `noncomputable`.

The corresponding `Type`-valued concept is the subtype `{n : // p n}`. An element of this subtype, say `x : {n : // p n}`, consists of a particular natural number (`x.val`) with which we may compute, together with a proof that `p (x.val)` holds (called `x.prop`). For example, a computable function can return `x.val + 2`. We can prove specifications about said computable function using `x.prop`. This subtype can also be written as the Sigma type `(n : ), p(n)`, again

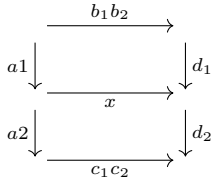
of type `Type`. Often in this formalization, we ignore the style guidelines to use subtypes, and rather favor sigma types. In the Lean infoview, a Sigma type (unlike a subtype) will show the type of n explicitly, which is important when its type is complicated and requires rewriting to another form.

Now let us discuss the case of inductive definitions. If an inductive definition takes values in `Prop`, Lean remembers only that the corresponding proposition has been proved; the particular constructor tree used to prove it is not computational data. If instead the inductive definition takes values in `Type`, its constructors form data. Accordingly, we will pass from several of our earlier `Prop`-valued relations to closely related `Type`-valued versions whenever their construction sequence is needed to define a metric.

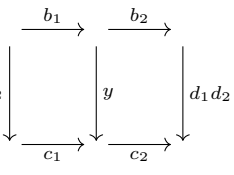
Why not simply make all our structures `Type`-valued from the beginning? Working with unneeded data is unwieldy. Thus, the Mathlib library is designed to be `Prop`-valued, so we cannot benefit from its extensive codebase when working with data (for example, we had to implement our own small data-carrying library of facts about prefixes and suffixes of lists). Similarly, many automated tactics, such as `aesop` and grind do not work on `Type`-valued objects. Thus, we work in `Prop` as much as possible, and only pass to `Type` when the data is absolutely necessary.

7.1 Grids

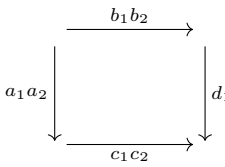
Currently, grids are `Prop`-valued in Lean. Saying that $\langle a, b, c, d\rangle \in G$ means that there exists some method of using the constructors of G to form a grid with frame $\langle a, b, c, d\rangle$. Note that there can be multiple ways of arriving at the same frame :



and



are both



although they are constructed differently. In order to give a size metric on grids, we will need to track the data of how a given grid was constructed. It is entirely possible to give a definition of size which gives differing values for different construction sequences leading to the same frame. Our metric will provably give the same length to any pair of grids with matching sources and targets, but that is not the case for every metric. (In fact, I am not able to prove such a specification for the eventual size metric I will give on partial grids.)

We thus enhance our definition of grids as follows :

Definition 7.1.

We define the set of grids with constructions G^D inductively [24] :

[24] Most precisely, we define the set of constructions K and grids with constructions G^D mutually inductively as follows:

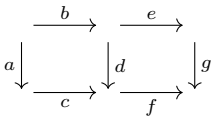
The set of constructions K is defined as the smallest set satisfying

- $\mathsf{trivial\ cell} \in K$
- $\mathsf{nontrivial\ cell} \in K$
- $\mathsf{horizontal}\,(g_1, g_2) \in K$ for $g_1, g_2 \in G^D$ with appropriate bounds on the frames of g_1 and g_2 (that is, the fourth element of g_1 equals the first element of g_2)
- $\mathsf{vertical}\,(g_1, g_2) \in K$ for $g_1, g_2 \in G^D$ with appropriate bounds on the frames of g_1 and g_2 (that is, the third element of g_1 equals the second element of g_2)

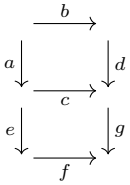
The set of grids with constructions G^D is defined as the smallest set satisfying

- The trivial cells :
  - $\langle \varepsilon, \varepsilon, \varepsilon, \varepsilon, \mathsf{trivial\ cell}\rangle \in G^D$
  - For any $i \in \mathbb{N}$, $\langle \sigma_i, \varepsilon, \varepsilon, \sigma_i, \mathsf{trivial\ cell}\rangle \in G^D$

- The trivial cells[25] :
  - $\langle \varepsilon, \varepsilon, \varepsilon, \varepsilon, \mathsf{trivial\ cell} \rangle \in G^D$
  - For any $i \in \mathbb{N}$, $\langle \sigma_i, \varepsilon, \varepsilon, \sigma_i, \mathsf{trivial\ cell} \rangle \in G^D$
  - For any $i \in \mathbb{N}$, $\langle \varepsilon, \sigma_i, \sigma_i, \varepsilon, \mathsf{trivial\ cell} \rangle \in G^D$
- The nontrivial cells :
  - For any $i \in \mathbb{N}$, $\langle \sigma_i, \sigma_i, \varepsilon, \varepsilon, \mathsf{nontrivial\ cell} \rangle \in G^D$
  - For $i, j \in \mathbb{N}$ with $|i - j| = 1$, $\langle \sigma_j, \sigma_i, \sigma_i\sigma_j, \sigma_j\sigma_i, \mathsf{nontrivial\ cell} \rangle \in G^D$.
  - For $i, j \in \mathbb{N}$ with $|i - j| > 1$, $\langle \sigma_j, \sigma_i, \sigma_i, \sigma_j, \mathsf{nontrivial\ cell} \rangle \in G^D$
- Horizontal abutting: If $g_1 = \langle a, b, c, d, \kappa_1 \rangle$ and $g_2 = \langle d, e, f, g, \kappa_2 \rangle$ are elements of G^D, then $\langle a, be, cf, g, \mathsf{horizontal}(g_1, g_2) \rangle \in G^D$.

  Visually, 

- Vertical abutting: If $g_1 = \langle a, b, c, d, \kappa_1 \rangle$ and $\langle e, c, f, g, \kappa_2 \rangle$ are elements of G^D, then $\langle ae, b, f, dg, \mathsf{vertical}(g_1, g_2) \rangle \in G^D$.

  Visually, 

In Lean, this is handled entirely for us by switching out `Prop` (which asserts the proposition that we have some grid) to `Type`, which preserves the data of the grid's construction.

```
inductive GridData : FreeMonoid → FreeMonoid → FreeMonoid → FreeMonoid → Type
```

It is immediate to delete the construction from a grid-with-construction and get a grid.

Theorem 7.2. If $\langle a, b, c, d, \kappa \rangle \in G^D$, then $\langle a, b, c, d \rangle \in G$.

```
def GridData.to_grid (h : GridData a b c d) : grid a b c d
```


  - For any $i \in \mathbb{N}$, $\langle \varepsilon, \sigma_i, \sigma_i, \varepsilon, \mathsf{trivial\ cell} \rangle \in G^D$
- The nontrivial cells :
  - For any $i \in \mathbb{N}$, $\langle \sigma_i, \sigma_i, \varepsilon, \varepsilon, \mathsf{nontrivial\ cell} \rangle \in G^D$
  - For $i, j \in \mathbb{N}$ with $|i - j| = 1$, $\langle \sigma_j, \sigma_i, \sigma_i\sigma_j, \sigma_j\sigma_i, \mathsf{nontrivial\ cell} \rangle \in G^D$.
  - For $i, j \in \mathbb{N}$ with $|i - j| > 1$, $\langle \sigma_j, \sigma_i, \sigma_i, \sigma_j, \mathsf{nontrivial\ cell} \rangle \in G^D$
- Horizontal abutting: Given $a, b, c, d, e, f, g \in BW^{\mathbb{1},+}$ and $\kappa_1, \kappa_2 \in K$, if $g_1 = \langle a, b, c, d, \kappa_1 \rangle$ and $g_2 = \langle d, e, f, g, \kappa_2 \rangle$ are elements of G^D, then $\langle a, be, cf, g, \mathsf{horizontal}(g_1, g_2) \rangle \in G^D$.
- Vertical abutting: Given $a, b, c, d, e, f, g \in BW^{\mathbb{1},+}$ and $\kappa_1, \kappa_2 \in K$, if $g_1 = \langle a, b, c, d, \kappa_1 \rangle$ and $\langle e, c, f, g, \kappa_2 \rangle$ are elements of G^D, then $\langle ae, b, f, dg, \mathsf{vertical}(g_1, g_2) \rangle \in G^D$.

[25] In Lean, each of the six cell cases is listed individually; here we combine them into two groups, trivial and nontrivial, to condense the presentation

Casting grids-with-constructions down to regular grids helps us convert many theorems (here, we show Theorem 5.4 and the Remark below Definition 5.5) as follows:

```
theorem braid_eq (h : GridData a b c d) :
    BraidMonoidInf.mk (a * c) = BraidMonoidInf.mk (b * d) :=
    Grid.braid_eq_of_grid (to_grid h)
```

```
theorem diag_length_eq (h : GridData a b c d) : a.length + c.length = b.length +
    d.length := Grid.diag_length_eq (to_grid h)
```

Conversely, given a grid, we know that there must be at least one construction sequence giving that frame :

Theorem 7.3. If $\langle a, b, c, d\rangle \in G$, then there exists some κ such that $\langle a, b, c, d, \kappa\rangle \in G^D$.

Note that the existential quantifier on κ asserts that the set of construction sequences generating such a frame is nonempty. In Lean, `Nonempty` has the same meaning :

```
def GridData.of_grid (h : grid a b c d) : Nonempty (GridData a b c d)
```

Thus, any theorems about grids can be translated to grids-with-constructions.

We began with `Prop`-valued grids because of the Mathlib support, and only move to the `Type`-valued grids (grids-with-constructions) when necessary. That occurs when we define a metric on grids-with-constructions. The metric is intended to represent the number of nontrivial cells occurring in the grid.

Definition 7.4. Define a metric on grids-with-constructions $\ell_G : G^D \to \mathbb{N}$ recursively as follows :

- $\ell_G\big(\langle a, b, c, d, \mathsf{trivial\ cell}\rangle\big) = 0$
- $\ell_G\big(\langle a, b, c, d, \mathsf{nontrivial\ cell}\rangle\big) = 1$
- $\ell_G\big(\langle a, b, c, d, \mathsf{horizontal}(g_1, g_2)\rangle\big) = \ell_G(g_1) + \ell_G(g_2)$
- $\ell_G\big(\langle a, b, c, d, \mathsf{vertical}(g_1, g_2)\rangle\big) = \ell_G(g_1) + \ell_G(g_2)$

We expand a number of theorems to discuss the grid size. For example, when splitting a grid, we show that the sum of the sizes of two new, smaller grids equals the size of the original grid. We run through the full proof of splittability given in Lemma 5.7 again, slightly augmented with facts about size, to obtain the following lemma :

Lemma 7.5. (Update to Lemma 5.7) Given $G_0 = \langle a_1a_2, b, c, d, \kappa_0\rangle \in G$, there exist grids $G_1 = \langle a_1, b, m, d_1, \kappa_1\rangle$ and $G_2 = \langle a_2, m, c, d_2, \kappa_2\rangle$ with $d = d_1d_2$ and $\ell_G(G_0) = \ell_G(G_1) + \ell_G(G_2)$.
Similarly, given $G_3 = \langle a, b_1b_2, c, d, \kappa_3\rangle \in G$, there exist grids $G_4 = \langle a, b_1, c_1, m, \kappa_4\rangle$ and $\langle m, b_2, c_2, d, \kappa_5\rangle$ with $c = c_1c_2$ and $\ell_G(G_3) = \ell_G(G_4) + \ell_G(G_5)$.

Our main theorem is that the spine of a grid suffices to determine its length. We begin with a lemma discussing certain small cases :

Lemma 7.6. [Determinative Spines – Length] For $a, b, c, d \in BW^+$ and $i, j \in \mathbb{N}$,

- If $\langle \varepsilon, b, c, d\rangle \in G$, then $\ell_G\big(\langle \varepsilon, b, c, d\rangle\big) = 0$.
- If $\langle a, \varepsilon, c, d\rangle \in G$, then $\ell_G\big(\langle a, \varepsilon, c, d\rangle\big) = 0$.
- If $\langle a, a, c, d\rangle \in G$, then $\ell_G\big(\langle a, a, c, d\rangle\big) = |a|$. Here, $|a|$ is the length of the word a.

- If $\langle \sigma_i, \sigma_j, c, d \rangle \in G$ with $|i - j| = 1$, then $\ell_G\big(\langle \sigma_i, \sigma_j, c, d \rangle\big) = 1$.
- If $\langle \sigma_i, \sigma_j, c, d \rangle \in G$ with $|i - j| > 1$, then $\ell_G\big(\langle \sigma_i, \sigma_j, c, d \rangle\big) = 1$.

Now, thanks to uniqueness of grid spines (Theorem 5.9), we may show the full theorem.

Theorem 7.7. Given $g_1 = \langle a, b, c, d, \kappa_1 \rangle \in G^D$ and $g_2 = \langle a, b, e, f, \kappa_2 \rangle \in G^D$, $\ell_G(g_1) = \ell_G(g_2)$.

Proof. (Sketch) We proceed by induction on g_1. In the base cases where g_1 is a single cell, Lemma 7.6 allows us to conclude that g_2 has the same size as g_1. We present the horizontal inductive case; the vertical inductive case is analogous.

Assume that $g_1 = \big\langle a, b, c, d, \mathsf{horizontal}(g_3, g_4) \big\rangle$, with $b = b_1 b_2$, $c = c_1 c_2$, and $g_3 = \langle a, b_1, c_1, m, \kappa_1 \rangle$ and $g_4 = \langle m, b_2, c_2, d, \kappa_2 \rangle$ to which the inductive hypothesis applies. Thus, $\ell_G(g_1) = \ell_G(g_3) + \ell_G(g_4)$. Then we split g_2 to obtain $g_5 = \langle a, b_1, c_5, d_5, \kappa_1 \rangle$ and $g_6 = \langle d_5, b_2, c_6, f, \kappa_2 \rangle$ with $\ell_G(g_2) = \ell_G(g_5) + \ell_G(g_6)$. Uniqueness of (data-carrying) grid spines (Theorem 5.9 combined with Theorem 7.2) applied to g_3 and g_5 gives us $c_5 = c_1$ and $d_5 = m$, and so we may apply our induction hypothesis to those two to obtain $\ell_G(g_3) = \ell_G(g_5)$. Since $d_5 = m$, $g_6 = \langle m, b_2, c_6, f, \kappa_2 \rangle$, and another application of Theorem 5.9 to g_4 and g_6 gives us $c_6 = c_2$. Finally, another application of our inductive hypothesis gives us $\ell_G(g_4) = \ell_G(g_6)$, which is sufficient to conclude. □

We use the fact that the spine of a grid suffices to determine its size to inspire the following metric ℓ_B on pairs of braid monoid words.

Definition 7.8. Given $a, b \in BW^+$, we know by Theorem 5.16 and Theorem 7.3 that there exist $c, d \in BW^+$ and a construction κ such that $g = \langle a, b, c, d, \kappa \rangle \in G^D$. We define

$$\ell_B(a, b) := \ell_G(g).$$

This is well-defined independently of the choice of c, d, and κ by Theorem 7.7.

In Lean, we take a slightly different approach to well-definedness. We have noncomputable `GridData.existence` (via `Prop`-valued grid existence and Theorem 7.3). We then use the axiom of choice to select such a grid to serve as g. We define `ab_len` to be the length of this selected grid. We do not use Theorem 7.7 and do not need to prove that our metric is independent of c, d, and κ. Eventually, when we use this metric, we will show that ℓ_B is bounded in the desired manner for any c, d, κ, which will suffice in place of well-definedness. Furthermore, there is no issue with ℓ_B being noncomputable – we can show termination of a computable algorithm via noncomputable termination metrics.

7.2 Partial Grids

We similarly update the definition of partial grids to track the data of their construction. In fact, throughout our Lean formalization we have used `Type`-valued partial grids. Unlike grids, where there were significant theorems not mentioning size (such as stability, Theorem 5.12), almost every theorem relating to partial grids discusses size. So we had little use for `Prop`–valued partial grids, and hence did not implement them. Therefore, the reader should view the following definition of G_P as the true definition, and look back upon Definition 6.28 as a simplified version. Thus, we do not need the clunky notation G_P^D.

Definition 7.9. (Update to Definition 6.28) We define the set of partial grids G_P as follows[26] :

[26] Again, this would be most rigorously defined mutually inductively with a set of constructions K_p.

- **a trivial cell** For any $a, b, c, d \in BW^{\mathbb{1},+}$, if $(\overline{a}\, b,\ c\, \overline{d}) \in R_{trivial}$, we have $\langle \overline{a},\ b,\ c,\ \varepsilon,\ \overline{d}, \mathsf{trivial\ cell} \rangle \in G_P$
- **a nontrivial cell** For any $a, b, c, d \in BW^{\mathbb{1},+}$, if $(\overline{a}\, b,\ c\, \overline{d}) \in R_{nontrivial}$, we have $\langle \overline{a},\ b,\ c,\ \varepsilon,\ \overline{d}, \mathsf{nontrivial\ cell} \rangle \in G_P$
- **a spine** For any $a, b \in \left\{BW^{\mathbb{1},+} \setminus \{\varepsilon\}\right\}$, $\langle \overline{a},\ b,\ \varepsilon,\ \overline{a}\, b,\ \varepsilon, \mathsf{spine} \rangle \in G_P$
- **horizontal concatenation to a partial grid with empty middle frontier** :

  Given partial grids $g_1 = \langle a_1, b_1, c_1, \varepsilon, d_1, \kappa_1 \rangle \in G_P$ and $g_2 = \langle d_1, b_2, c_2, m_2, d_2, \kappa_2 \rangle \in G_P$, we have $\left\langle a_1, b_1 \cdot b_2, c_1 \cdot c_2, m_2, d_2, \mathsf{horizontal}_\varepsilon(g_1, g_2) \right\rangle \in G_P$.
- **horizontal concatenation to a partial grid with non-empty middle frontier** (m_1 is non-empty) :

  Given $g_1 = \langle a_1, b_1, c_1, m_1, d_1, \kappa_1 \rangle \in G_P$ and $g_2 = \langle d_1, b_2, c_2, m_2, d_2, \kappa_2 \rangle \in G_P$, we have $\left\langle a_1, b_1 \cdot b_2, c_1, m_1 \cdot c_2 \cdot m_2, d_2, \mathsf{horizontal}(g_1, g_2) \right\rangle \in G_P$.
- **vertical concatenation to a partial grid with empty middle frontier** :

  Given $g_1 = \langle a_1, b_1, c_1, \varepsilon, d_1, \kappa_1 \rangle \in G_P$ and $\langle a_2, c_1, c_2, m_2, d_2, \kappa_2 \rangle \in G_P$, we have $\left\langle a_2 \cdot a_1, b_1, c_2, m_2, d_2 \cdot d_1, \mathsf{vertical}_\varepsilon(\mathsf{g}_1, \mathsf{g}_2) \right\rangle \in G_P$.
- **vertical concatenation to a partial grid with non-empty middle frontier**

  Given $\langle a_1, b_1, c_1, m_1, d_1, \kappa_1 \rangle \in G_P$ and $\langle a_2, c_1, c_2, m_2, d_2, \kappa_2 \rangle \in G_P$, we have $\left\langle a_2 \cdot a_1, b_1, c_2, m_2 \cdot d_2 \cdot m_1, d_1, \mathsf{vertical}(g_1, g_2) \right\rangle \in G_P$.

Definition 7.10. Define a metric on partial grids ℓ_P by the following :

- $\ell_P\left(\langle a, b, c, m, d, \mathsf{trivial\ cell} \rangle\right) = 0$
- $\ell_P\left(\langle a, b, c, m, d, \mathsf{nontrivial\ cell} \rangle\right) = 1$
- $\ell_P\left(\langle a, b, c, m, d, \mathsf{spine} \rangle\right) = 0$
- $\ell_P\left(\langle a, b, c, m, d, \mathsf{horizontal}_\varepsilon(p_1, p_2) \rangle\right) = \ell_P(p_1) + \ell_P(p_2)$
- $\ell_P\left(\langle a, b, c, m, d, \mathsf{horizontal}(p_1, p_2) \rangle\right) = \ell_P(p_1) + \ell_P(p_2)$
- $\ell_P\left(\langle a, b, c, m, d, \mathsf{vertical}_\varepsilon(p_1, p_2) \rangle\right) = \ell_P(p_1) + \ell_P(p_2)$
- $\ell_P\left(\langle a, b, c, m, d, \mathsf{vertical}(p_1, p_2) \rangle\right) = \ell_P(p_1) + \ell_P(p_2)$

It is possible to extend a partial grid's spine without affecting its size.

Lemma 7.11. (Update to Theorem 6.31) Consider a partial grid $p = \langle a, b, c, m, d, \kappa \rangle$. For any non-empty *negative* augmented grid word a' there exists a partial grid $p_1 = \langle a' \cdot a, b, \varepsilon, a' \cdot c \cdot m, d, \kappa_1 \rangle$ such that $\ell_P(p) = \ell_P(p_1)$. For any non-empty *positive* augmented grid word b', there exists a partial grid $p_2 = \langle a, b \cdot b', c, m \cdot d \cdot b', \varepsilon, \kappa_2 \rangle$ such that $\ell_P(p) = \ell_P(p_2)$.

Splittability also behaves as we might expect in regard to the size of the resulting partial grids.

Theorem 7.12. (Update to Theorem 6.34) Given $p = \langle a, b_1 b_2, c, m, d \rangle \in G_P$ with nonempty b_1, b_2, we have either

- p splits into two smaller partial grids $p_1 = \langle a, b_1, c_1, m_1, d_1 \rangle \in G_P$ and $p_2 = \langle d_1, b_2, c_2, m_2, d \rangle \in G_P$, with $c \cdot m = c_1 \cdot m_1 \cdot c_2 \cdot m_2$, and $\ell_P(p) = \ell_P(p_1) + \ell_P(p_2)$.

- p splits into a smaller partial grid $p_1 = \langle a, b_1, c, m_1, \varepsilon\rangle \in G_P$ and a tail $\xrightarrow{b_2}$, with $m = m_1 \cdot b_2$ and $\ell_P(p) = \ell_P(p_1)$.

We update our `add_cell` function to add in a fact about the size of the resulting partial grid :

Definition 7.13. `add_cell` is a function taking in some $(x, y) \in R_{\mathsf{grid}}$ and a partial grid $p_1 = \langle a, b, c, m_1 \cdot x \cdot m_2, d, \kappa_1\rangle$, and returning a partial grid $p_2 = \langle a, b, c', m', d', \kappa_2\rangle$ where $c' \cdot m' \cdot d' = c \cdot m_1 \cdot y \cdot m_2 \cdot d$, c is a prefix of c', d is a suffix of d', and the following property holds : if $(x, y) \in R_{\mathsf{triv}}$, $\ell_P(p_2) = \ell_P(p_1)$; otherwise, we have $(x, y) \in R_{\mathsf{nontriv}}$, and $\ell_P(p_2) = \ell_P(p_1) + 1$.

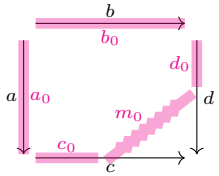
Note that we could separate out the final property about length, and prove it separately as a theorem about the output of `add_cell` (in contrast to, say, the prefix and suffix properties, which are required for the definition of `add_cell`). However, like the definition of `add_cell`, such a proof would have 17 cases[27], so it is easier to fold the length property into the main definition.

Our main goal will be to show that given a grid and a partial grid sharing essentially the same spine (aside from any empty arrows), the size of the partial grid is at most that of the grid. Our proof requires us to strengthen the inductive statement with an auxiliary fact about prefixes and suffixes of the grid's target (part (1) below).

Theorem 7.14. Let $g = \langle a, b, c, d\rangle \in G$ and $p = \langle a_0, b_0, c_0, m_0, d_0\rangle \in G_P$. Recall that $s^{\mathbb{1},+} : BW^{\mathbb{1},+} \to BW^+$ removes the signs on the letters, and deletes all instances of $\mathbb{1}$ from a word. It is generally used to convert (positive) partial grid edges to grid edges. If $a = s^{\mathbb{1},+}(\overline{a_0})$ and $b = s^{\mathbb{1},+}(b_0)$ (that is, the spines agree, ignoring empty arrows), then

1. $s^{\mathbb{1},+}(c_0)$ is a prefix of c and $s^{\mathbb{1},+}(\overline{d_0})$ is a prefix of d
2. $\ell_P(p) \le \ell_G(g)$.

Here is the image to keep in mind :



We note that we could prove (1) independently, and then use (1) to later prove (2). (We cannot proceed conversely; (2) cannot be proven without (1).) Due to the large number of cases in the proof, and the work to organize the hypotheses for each case, it is preferable to combine (1) and (2) as we have done, proving them simultaneously.

Proof. (Sketch) We proceed by induction on g, and present one illustrative (sub-sub-)case. Consider the case where $g = \langle a, b, c, d, \mathsf{horizontal}(g_1, g_2)\rangle$ for grids $g_1 = \langle a, b_1, c_1, d_1, \kappa_1\rangle$ and $g_2 = \langle d_1, b_2, c_2, d, \kappa_2\rangle$ to which the inductive hypothesis applies, with $b = b_1 b_2$ and $c = c_1 c_2$. Of course, $\ell_G(g) = \ell_G(g_1) + \ell_G(g_2)$. As $s^{\mathbb{1},+}(b_0) = b = b_1 b_2$, we know there exist some $b_3, b_4 \in BW^{1,+}$ such that $b_0 = b_3 b_4$ and $b_1 = s^{\mathbb{1},+}(b_3)$ and $b_2 = s^{\mathbb{1},+}(b_4)$. We may thus split $p = \langle a_0, b_3 b_4, c_0, m_0, d_0\rangle$ vertically. Consider the case in which p splits in to two sub-grids $p_1 = \langle a_0, b_3, c_3, m_3, d_3\rangle$ and $p_2 = \langle d_3, b_4, c_4, m_4, d_0\rangle$ with $c_3 m_3 c_4 m_4 = c_0 m_0$ and $\ell_P(p) = \ell_P(p_1) + \ell_P(p_2)$ (the other case is simpler). Further take the case in which m_3 is non-empty, and thus $c_0 = c_3$ and $m_0 = m_3 c_4 m_4$.

[27] The cases for horizontal and vertical appending to a partial grid with non-empty frontier have 5 sub-cases each. The case for a single cell comprises 4 subcases. The other 3 inductive cases (horizontal and vertical appending to a partial grid with empty middle frontier and the spinal case) do not split further.

We apply our inductive hypothesis to g_1 and p_1 to conclude that $s^{\mathbb{1},+}(c_3)$ is a prefix of c_1 (and thus $s^{\mathbb{1},+}(c_0)$ is a prefix of $c = c_1c_2$ as desired), $s^{\mathbb{1},+}\left(\overline{d_3}\right)$ is a prefix of d_1, and $\ell_P(p_1) \leq \ell_G(g_1)$. Thus, there is some negative augmented braid word d_5 such that $d_1 = s^{\mathbb{1},+}\left(\overline{d_3}\right) s^{\mathbb{1},+}\left(\overline{d_5}\right)$. We may extend the left frontier of p_2 to obtain a partial grid $p_3 = \langle d_5 \cdot d_3, b_4, \varepsilon, d_5 \cdot c_4 \cdot m_4, d_0 \rangle$ with $\ell_P(p_3) = \ell_P(p_2)$. Then we know $d_1 = s^{\mathbb{1},+}\left(\overline{d_5 \cdot d_3}\right)$, and so we may apply our inductive hypothesis to g_2 and p_3 to conclude that $s^{\mathbb{1},+}(\overline{d_0})$ is a prefix of d as desired, and that $\ell_P(p_3) \leq \ell_G(g_2)$. Thus, our final condition, that $\ell_P(p) \leq \ell_G(g)$, is satisfied. □

7.3 Semi-Thue Derivations

We return to our two definitions of Semi-Thue systems, defining data-carrying versions so that we may define a metric on them. In Lean, this once again corresponds to switching the output type from `Prop` to `Type`.

Definition 7.15.

Consider a set A and a binary relation R on A-strings. Then $\xrightarrow[R]{*}$ is the smallest relation satisfying :

- reflexivity : For all $a \in A^*$, $\langle a \xrightarrow[R]{*} a, \mathsf{refl} \rangle$
- closure : For all $a, b \in A^*$, $a \underset{R}{\longrightarrow} b$ implies $\langle a \xrightarrow[R]{*} b, \mathsf{closure}(a, b) \rangle$
- transitivity : For all $a, b, c \in A^*$, if $s_0 = \langle a \xrightarrow[R]{*} b, \kappa_0 \rangle$ and $s_1 = \langle b \xrightarrow[R]{*} c, \kappa_1 \rangle$, then $\langle a \xrightarrow[R]{*} c, \mathsf{trans}(s_0, s_1) \rangle$

Definition 7.16. We define a metric on Semi-Thue systems ℓ_{S_R}, parametrized by the relation R and a size metric $\ell_R : R \to \mathbb{N}$ on pairs of A-strings satisfying R. To avoid triple subscripts, we do not add the particular choice of ℓ_R to our notation, allowing it to be inferred from context. In the below definition, for readability we use an _ as a placeholder, as no information from the first element of the pair is needed.

- $\ell_S\big(\langle _, \mathsf{refl} \rangle\big) = 0$
- $\ell_{S_R}\big(\langle _, \mathsf{closure}(a, b) \rangle\big) = \ell_R(a, b)$
- $\ell_{S_R}\big(\langle _, \mathsf{trans}(s_0, s_1) \rangle\big) = \ell_{S_R}(s_0) + \ell_{S_R}(s_1)$

Definition 7.17. Given a set A and a binary relation R over A-strings, let $\underset{R}{\twoheadrightarrow}$ be the smallest relation satisfying the below two properties :

- reflexivity : For all $a \in A^*$, $\langle a \underset{R}{\twoheadrightarrow} a, \mathsf{refl} \rangle$
- one-step transitivity : For all $a, b_1, x, b_2, y \in A^*$, given $d_0 = \langle a \underset{R}{\twoheadrightarrow} b_1 x b_2, \kappa_0 \rangle$ and xRy, then $\langle a \underset{R}{\twoheadrightarrow} b_1 y b_2, \mathsf{step\ transitivity}(d_0, x, y) \rangle$

Definition 7.18. We define a metric on derivation-style Semi-Thue systems ℓ_{D_R}, again parametrized by the relation R and a size metric ℓ_R.

- $\ell_{D_R}\big(\langle _, \mathsf{refl} \rangle\big) = 0$
- $\ell_{D_R}\big(\langle _, \mathsf{step\ transitivity}(d_0, x, y) \rangle\big) = \ell_{D_R}(d_0) + \ell_R(x, y)$

Theorem 7.19. Given $s_0 = \langle a \xrightarrow[R]{*} b, \kappa_0 \rangle$, there exists a derivation $d_0 = \langle a \underset{R}{\twoheadrightarrow} b, \kappa_1 \rangle$ with $\ell_{S_R}(s_0) = \ell_{D_R}(d_0)$. The converse of this statement also holds.

With the basics set up, we now move on to discuss our specific relations.

Definition 7.20. We define $\ell_{R_{\mathsf{rev}}}$ simply : For all $(x, y) \in R_{\mathsf{rev}}$, $\ell_{R_{\mathsf{rev}}}(x, y) = 1$.
$\ell_{R_{\mathsf{grid}}}$ is defined as follows :

- For all $(x, y) \in R_{\mathsf{triv}}$, $\ell_{R_{\mathsf{grid}}}(x, y) = 0$
- For all $(x, y) \in R_{\mathsf{nontriv}}$, $\ell_{R_{\mathsf{grid}}}(x, y) = 1$

Next, we update our results from the previous chapter with facts about our metrics. The proofs proceed in an entirely analogous manner. Here are the main conclusions (of course, each of these requires a number of lemmas to be updated as well; we omit those statements and proofs) :

Theorem 7.21. (Update to the backwards direction of Theorem 6.3) Given a partial grid $p = \langle a, b, c, m, d, \kappa \rangle$, there exists a derivation $s = \langle \overline{a}b \xrightarrow[R_{\mathsf{rev}}]{*} cm\overline{d}, \kappa' \rangle$ with $\ell_P(p) = \ell_{S_{R_{\mathsf{rev}}}}(s)$.

Theorem 7.22. (Update to Theorem 6.14) Let $a, b, c, d \in BW^{\mathbb{1},+}$. Given $s_0 = \langle \overline{a}b \xrightarrow[R_{\mathsf{rev}}]{*} c\overline{d}, \kappa_0 \rangle$, there exist $e, f \in BW^{\mathbb{1},+}$ and a derivation $s_1 = \langle i^{\mathbb{1}}(\overline{a}b) \xrightarrow[R_{\mathsf{grid}}]{*} e\overline{f}, \kappa_1 \rangle$ with $s^{\mathbb{1}}(e) = c$, $s^{\mathbb{1}}(f) = d$ and $\ell_{S_{R_{\mathsf{rev}}}}(s_0) = \ell_{S_{R_{\mathsf{grid}}}}(d_1)$.

Theorem 7.23. (Update to Theorem 6.35) For nonempty $a, b \in BW^{\mathbb{1},+}$, given $s_0 = \langle \overline{a}b \xrightarrow[R_{\mathsf{grid}}]{*} c, \kappa_0 \rangle$, then there is a partial grid p with spine (a, b) and frontier c such that $\ell_{S_{R_{\mathsf{rev}}}}(s_0) = \ell_P(p)$.

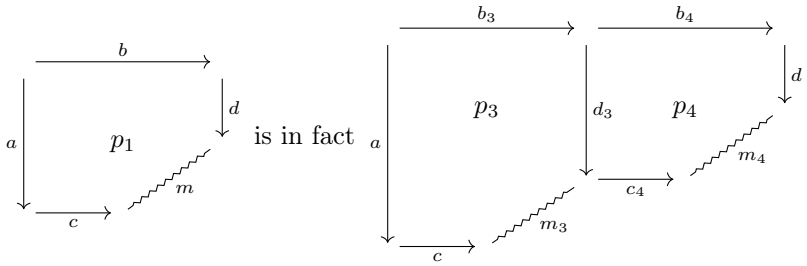
7.4 Alternative Choices

We have made a number of seemingly arbitrary decisions, mainly in the precise definition of partial grids, as well as that of their size. Here we present their shortfalls, and discuss alternative strategies with their own benefits and shortfalls. As the definitions and theorems here do not form part of the final project, the Lean formalizations linked may be a bit rougher, and we take a few more liberties in our presentation, deviating from the exact Lean formalization for the sake of exposition.

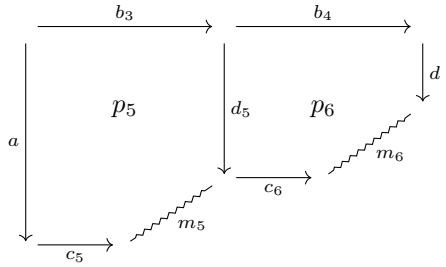
Let us begin with the shortfalls of our current definitions. There is one tantalizing theorem we have not been able to prove using structural induction. It would be nice to know that the frame of a partial grid determines its length :

Conjecture 7.24. Given partial grids $p_1 = \langle a, b, c, m, d, \kappa_1 \rangle$ and $p_2 = \langle a, b, c, m, d, \kappa_2 \rangle$, we have $\ell_P(p_1) = \ell_P(p_2)$

The obvious strategy of inducting on one of either p_1 or p_2 fails. Let us play through one case of this proof attempt to see where the failure lies. Say we induct on p_1, and are in the case of horizontal appending (to a partial grid with non-empty middle frontier).



That is, we know there are nonempty b_3, b_4 with $b = b_3 b_4$ and nonempty m_3 and some c_4, m_4 with $m = m_3 c_4 m_4$, partial grids $p_3 = \langle a, b_3, c, m_3, d_3, \kappa_3 \rangle$ and $p_4 = \langle d_3, b_4, c_4, m_4, d, \kappa_4 \rangle$ to which the inductive hypothesis may be applied, with $p_1 = \langle a, b_3 \cdot b_4, c, m_3 \cdot c_4 \cdot m_4, d, \mathsf{horizontal}(p_3, p_4) \rangle$. We would next split p_2 horizontally using Theorem 7.12, in hopes of applying our inductive hypothesis for p_3 and p_4 to the resultant grids. Consider the case in which p_2 splits into two partial grids p_5, p_6 as shown:



Here, we have partial grids $p_5 = \langle a, b_3, c_5, m_5, d_5 \rangle$ and $p_6 = \langle d_5, b_4, c_6, m_6, d \rangle$, with $c \cdot m = c_5 \cdot m_5 \cdot c_6 \cdot m_6$. We may further select the case where m_5 is nonempty, and so by properties of middle frontiers (Lemma 6.32), $c = c_5$ and $m = m_5 c_6 m_6$. We wish to apply the inductive hypothesis to p_5 and p_3. However, we do not know that their frames match. Their spines do match, as do their lower frontiers, but not their middle frontiers (m_3 and m_5) nor their right frontiers (d_3 and d_5). As $m_3 \cdot c_4 \cdot m_4 = m_5 \cdot c_6 \cdot m_6$, we know that the shorter of m_3 and m_5 must be a prefix of the longer, but we do not know the crucial concluding condition : that m_3 and m_5 have the same length. We could also deduce, with significant extra work[28], that either d_3 is a suffix of d_5 or vice versa. Alas, these facts about the middle and right frontier are not enough to conclude. (The fact that two partial grids have the same spine and bottom frontier, middle frontiers one of which is a prefix of the other, and right frontiers one of which is a suffix of the other is not enough to conclude that they have matching frames).

The situation is even worse for p_4 and p_6, as their spines do not necessarily match. Now, this could be amended in a number of cases. If d_5 were shorter than d_3, we could extend the spine of p_6 to match d_3, as we know that extending a partial grid's spine preserves its size (however, this would have consequences for its bottom and middle frontiers).

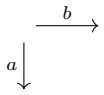
If d_5 were longer than d_3, we might be able to use an alternate form of induction, some sort of strong induction (which we will discuss below), to extend the spine of p_4 and then apply our more generalized induction hypothesis. However, this still does not solve the problem of the mismatched frontiers between p_4 and p_6, so we remain at an impasse.

Thus, Conjecture 7.24 remains inconclusive. It is possible to complete the project without Conjecture 7.24, but is certainly displeasing. Intuitively, partial grids with the same frame contain the same (multi)set of cells, and should have the same size. We wonder if perhaps it is our definition of a partial grid, with its complicated division of the frontier into bottom, middle, and right segments, that is to blame. Here is an alternative, simpler definition of a partial grid :

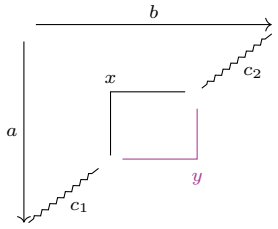
Definition 7.25. The set of frontier-style partial grids G_F is a subset of $(BW^{\mathbb{1}})^3$ defined inductively by

- a spine : For any positive, nonempty $a, b \in BW^{\mathbb{1}}$, we have $\langle \overline{a}, b, \overline{a}b, \mathsf{spine} \rangle \in G_f$. Visually,

[28] To do so, we would need to define a new version of grids which track empty arrows, then note that by existence and uniqueness of such grids, p_3 and p_5 lie within the same grid. Finally, we'd need to update the prefix-suffix fact in Theorem 7.14 to discuss such empty-arrow-tracking grids.



- cell addition : Given $(x, y) \in R_{\mathsf{grid}}$ and $f_1 = \langle a, b, c_1 x c_2, \kappa_1 \rangle \in G_F$, we have $\langle a, b, c_1 y c_2, \mathsf{cell\ addition}(x, y, f_1) \rangle \in G_F$. Visually,

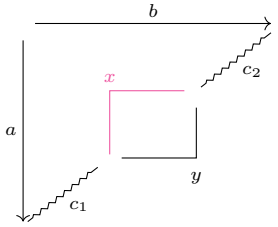


Definition 7.26. We define the length metric $\ell_{G_F} : G_F \to \mathbb{N}$ as follows :

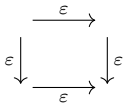
- a spine : $\ell_{G_F}\big(\langle _, _, _, \mathsf{spine} \rangle\big) = 0$
- cell addition : $\ell_{G_F}\Big(\big\langle _, _, _, \mathsf{cell\ addition}(x, y, f_1) \big\rangle\Big) = \ell_{G_F}(f_0) + \ell_{R_{\mathsf{grid}}}(x, y)$.

Alas, we again encounter issues trying to prove Conjecture 7.24, even with G_F in place of G_P. The issue is that frontier-style partial grids do not have a unique destructor. We wish we had a theorem of the following form :

Fallacy 7.27. Given $(x, y) \in R_{\mathsf{grid}}$ and a frontier-style partial grid $f_1 = \langle a, b, c_1 y c_2, \kappa_1 \rangle$, there is a frontier-style partial grid $f_0 = \langle a, b, c_1 x c_2, \kappa_0 \rangle$ with $\ell_F(f_1) = \ell_F(f_0) + \ell_{R_{\mathsf{grid}}}(x, y)$.



In fact, Fallacy 7.27 does not even hold for the base cases. Consider the very simple frontier-style partial grid $f_1 = \langle \overline{\mathbb{1}}, \mathbb{1}, \mathbb{1}\overline{\mathbb{1}}, \mathsf{cell\ addition}(\overline{\mathbb{1}}\mathbb{1}, \mathbb{1}\overline{\mathbb{1}}, \langle \overline{\mathbb{1}}, \mathbb{1}, \overline{\mathbb{1}}\mathbb{1}, \mathsf{spine} \rangle)$



We know that, say, $(\overline{\sigma_3}\sigma_3, \mathbb{1}\overline{\mathbb{1}}) \in R_{\mathsf{grid}}$ and so our conjecture states that we should have some frontier-style partial grid $f_0 = \langle \overline{\mathbb{1}}, \mathbb{1}, \sigma_3\overline{\sigma_3}, \kappa_0 \rangle$. Clearly, such a partial grid does not exist. We can amend Fallacy 7.27 to the following statement, which is both tedious to prove and nearly useless. Hence, it is not formalized in Lean, and listed merely as a fact.

Fact 7.28. Given $(x, y) \in R_{\mathsf{grid}}$ and a frontier-style partial grid $f_1 = \langle a, b, c_1 y c_2, \kappa_1 \rangle$, there is a frontier-style partial grid $f_0 = \langle a, b, c_1 z c_2, \kappa_0 \rangle$ for some $z \in BW^{\mathbb{1}}$ with $(z, y) \in R_{\mathsf{grid}}$, and $\ell_F(f_1) = \ell_F(f_0) + \ell_{R_{\mathsf{grid}}}(z, y)$.

Due to the lack of a unique destructor, we are unable to prove that the frame of a frontier-style partial grid suffices to determine its size.

Conjecture 7.29. Given frontier-style partial grids $f_1 = \langle a, b, c, \kappa_1 \rangle$ and $f_2 = \langle a, b, c, \kappa_2 \rangle$, we know $\ell_F(f_1) = \ell_F(f_2)$.

Again, it is instructive to see where the straightforward proof attempt fails. Say we proceed by induction on f_1. Consider the case where $c = c_1 \cdot \mathbb{1}\overline{\mathbb{1}} \cdot c_2$, we have $f_0 = \langle a, b, c_1 \cdot \overline{\sigma_3}\sigma_3 \cdot c_2 \rangle$ to which the inductive hypothesis applies, and $f_1 = \langle a, b, c_1 \cdot \mathbb{1}\overline{\mathbb{1}} \cdot c_2, \mathsf{cell\ appending}(\overline{\sigma_3}\sigma_3, 1\overline{1}, f_0) \rangle$. We would like to in some manner destruct f_2 to apply the inductive hypothesis, but alas, we have no unique destructor. For all we know, there is a frontier-style partial grid $f_3 = \langle a, b, c_1 \cdot \overline{\sigma_{42}}\sigma_{42} \cdot c_2, \kappa_3 \rangle$ and $f_2 = \langle a, b, c_1 \cdot \mathbb{1}\overline{\mathbb{1}}, \mathsf{cell\ appending}(\overline{\sigma_{42}}\sigma_{42}, \mathbb{1}\overline{\mathbb{1}}, f_3) \rangle$.

So, frontier-style partial grids suffer the same shortcoming as partial grids — we have not been able to prove that their frame determines their size. Do they improve on our original definition of partial grids in any substantial way? In some ways, yes. For example, the `add_cell` function would be nearly trivial to implement for frontier-style partial grids, as compared to the seventeen involved cases for our original definition. In other ways, certainly not. I could not find a proof using frontier-style partial grids for Theorem 6.37 (if a partial grid's frontier is in negative-positive form, then it is in fact a grid). Thus, we would need to keep both structures and transfer theorems between them. The work of setting up a (size-preserving) bijection between G_P and G_F is at least equal to the work needed to define `add_cell` directly on G_P, and so we dispense with frontier-style partial grids for good.

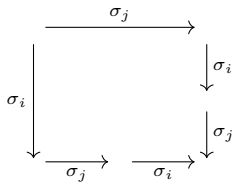
The enterprising reader is most welcome to formulate his own definition for partial grids. Here are three theorems such a definition must satisfy :

- Theorem 6.37 : if a partial grid's frontier is in negative-positive form, then it is in fact a grid.
- Definition 7.13 : there exists a function with the same specification as `add_cell`.
- Theorem 7.23 : if $\overline{a}b$ rewrites to c, then there is a partial grid with spine (a, b) and frontier c.

The ideal definition would also satisfy Conjecture 7.24; that is, the frame of the partial grid determines its size.

This is not the only unwieldy issue with the partial grids we have defined. We do not have a rigorous definition of a sub-partial-grid, which could provide a method of strong induction on partial grids. Surprisingly, there is not a single proof in this project requiring strong induction on partial grids. Nevertheless, one might find it easier to use than structural induction in some cases, or perhaps give a new theorem, or a proof of a conjecture, requiring strong induction.

Here are a few simple ideas for inducting on a partial grid $\langle a, b, c, m, d \rangle$, none of which work :

- We might try to induct on the length of the spine (that is, $|a| + |b|$). However, due to cells of the form 

, this metric often increases when we split a partial grid. The issue persists even if we ignored the empty cells by using $|s^{\mathbb{1}}(a)| + |s^{\mathbb{1}}(b)|$ for our metric
- We might try to induct on the diagonal length of the partial grid, $|a| + |c|$, or possibly $|b| + |d|$ (or even the sum of the two). However, this metric remains static when adding a cell that touches only the middle frontier.

- We might try to induct on the size of the partial grid using ℓ_P. However, due to the presence of trivial cells, it is possible to split a partial grid and obtain one part with size zero, and one part with the original size.

The last option could be amended to be workable : we could change our metric to ℓ'_P, which gives every cell size 1 :

Definition 7.30. Define an alternative metric on partial grids ℓ'_P identically to ℓ_P, except for the very first case, which we replace by

- $\ell_P\big(\langle a, b, c, m, d, \mathsf{trivial\ cell}\rangle\big) = 1$

This would almost work for strong induction, except trimming off tails (such as arises in the second case of splittability) does not decrease ℓ'_P. It could be perhaps amended to use the length of the spine as a tie-breaker (formally, a lexicographic ordering on the pair $\langle \ell'_P, \mathsf{spine\ length}\rangle$). This new metric is less elegant, especially as ℓ'_P does not match nicely with $\ell_{S_{\mathsf{rev}}}$.

So, even the simplest methods for defining strong induction are reasonably complex (in order to derive strong induction from the just-mentioned lexicographic order, one would have to consider a grid g and a smaller (by lexicographic ordering) grid g_1, and then repeatedly split g_1 and apply determinative spines, grid uniqueness, and grid appending in order to obtain grids to which the structural inductive hypothesis would apply). Strong induction is never used in this project, and so we do not formalize ℓ'_P nor the associated lexicographic ordering.

In sum, we have defined metrics on our main structures (Semi-Thue derivations, grids, and partial grids). We have been able to show a number of properties of each metric, properties which will let us prove termination of the subword reversing algorithm in the following chapter.

8 The Reversing Procedure

We are now prepared to implement Dehornoy's subword reversing algorithm, which determines whether a braid group element is equivalent to the identity element. (Of course, to check equivalence between elements a and b, we need simply check if $a^{-1}b$ is equivalent to the identity.) The heart of this algorithm is a process called "reversing" which rewrites any braid group element to one in a certain standard form, preserving equivalence in the braid group. As an intuitive overview, a string in BW undergoes successive rewritings according to R_{rev} until no such rewrites remain to be performed. We begin with "simple" reversing, which takes in a braid word in positive-negative form. We then extend simple reversing to full reversing, which takes in any braid word. Finally, we implement the full subword reversing algorithm, which reverses a word, exchanges the positive and negative parts, and then reverses again.

8.1 Monoid Solver

We will first implement a solver for the braid monoid word problem. Much of the machinery will be expanded and re-used for the braid group word problem solver. We begin by defining simple reversing, in which we iteratively apply reversing rewrites to $\overline{a}b$ for $a, b \in BW^+$. (Note that many of our functions are given the superscript $^+$, as this simple version begins with two elements of the braid *monoid*). Our eventual solver will include simple reversing as its main step.

To aid in exposition, we give terminology to the first element of any R_{rev} pair as follows :

Definition 8.1. For any $i, j \in \mathbb{N}$, we say that the braid word $\overline{\sigma_i}\sigma_j$ is an **open pair**.

As a first attempt, we might define said iterative rewriting $\mathsf{rev}_1^+ : BW \to BW$ recursively as so: If there exist $c_1, c_2 \in BW$ such that for some $i, j \in \mathbb{N}$, $c = c_1 \cdot \overline{\sigma_i}\sigma_j \cdot c_2$ where c_1 contains no open pair, then

$$\mathsf{rev}_1^+(c) = \mathsf{rev}_1^+(c_1 \cdot \overline{\sigma_i}\sigma_j \cdot c_2) := \begin{cases} \mathsf{rev}_1^+(c_1 \cdot c_2) & i = j, \\ \mathsf{rev}_1^+(c_1 \cdot \sigma_j\overline{\sigma_i} \cdot c_2) & |i-j| \geq 2, \\ \mathsf{rev}_1^+(c_1 \cdot \sigma_j\sigma_i\overline{\sigma_j\sigma_i} \cdot c_2) & |i-j| = 1. \end{cases}$$

Otherwise,

$$\mathsf{rev}_1^+(c) = c.$$

The immediate problem is to show well-definedness of this function – that is, that the recursive calls in rev_1^+ do not go on endlessly[29]. This will require the fact that our original input is of the form $\overline{a}\,b$, and $\overline{a}\,b$ rewrites under R_{rev} to the output of rev_1^+ :

$$\overline{a}\,b \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} \mathsf{rev}_1^+(\overline{a}\,b)\,.$$

That is immediately circular – how can we speak about the output of a function before we know it terminates? Instead, we build this fact into the definition of the function by restricting the domain and codomain of rev^+.

Definition 8.2. Define the sets T parametrized by $a, b \in BW^+$ as[30]

$$T_{a,b} = \{\, c \in BW \mid \overline{i^+(a)}\, i^+(b) \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} c) \,\}.$$

Here we are briefly explicit about i^+ for precision. Note that we are using the data-carrying definition of $\underset{R_{\mathsf{rev}}}{\twoheadrightarrow}$. Our formalization defines elements of $T_{a,b}$ as a pair (x, y), where $x \in BW$ and $y := \langle \overline{i^+(a)}\, i^+(b) \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} x, \kappa \rangle$. We thus use this convenient notation in our writeup.

Then we inductively define the family of functions

$$\mathsf{rev}_{a,b}^+ : T_{a,b} \to T_{a,b}$$

as follows; note that the only differences from the first definition are the name of the function and the domain and range (and hence the added derivation).

Definition 8.3. Given $a, b \in BW^+$, $\mathsf{rev}_{a,b}^+ : T_{a,b} \to T_{a,b}$ is defined by

(Recursive case) If there exist $x_1, x_2 \in BW$ such that for some $i, j \in \mathbb{N}$, $x = x_1 \cdot \overline{\sigma_i}\sigma_j \cdot x_2$ where

[29] The replacement may introduce new negative-positive pairs, so termination is not straightforward

[30] We use the letter T as any element of $T_{a,b}$ corresponds to a partial grid with a potentially jagged frontier, hence roughly in in the shape of a triangle.

x_i contains no open pair, then

$$\mathsf{rev}^+_{a,b}(x_1 \cdot \overline{\sigma_i}\sigma_j \cdot x_2, y) = \begin{cases} \mathsf{rev}^+_{a,b}\Big(x_1 \cdot x_2, & \\ \qquad \big\langle \overline{i^+(a)}\, i^+(b) \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} x_1 \cdot x_2, & i = j, \\ \qquad\quad \mathsf{step}\,\mathsf{transitivity}(y, \overline{\sigma_i\sigma_j}, \varepsilon)\big\rangle\Big) & \\ \mathsf{rev}^+_{a,b}\Big(x_1 \cdot \sigma_j\sigma_i\overline{\sigma_j}\,\overline{\sigma_i} \cdot x_2, & \\ \qquad \big\langle \overline{i^+(a)}\, i^+(b) \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} x_1 \cdot \sigma_j\sigma_i\overline{\sigma_j}\,\overline{\sigma_i} \cdot x_2, & |i-j| = 1, \\ \qquad\quad \mathsf{step}\,\mathsf{transitivity}\big(y, \overline{\sigma_i\sigma_j}, \sigma_j\sigma_i\overline{\sigma_j}\,\overline{\sigma_i}\big)\big\rangle\Big) & \\ \mathsf{rev}^+_{a,b}\Big(x_1 \cdot \sigma_j\overline{\sigma_i} \cdot x_2, & \\ \qquad \big\langle \overline{i^+(a)}\, i^+(b) \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} x_1 \cdot \sigma_j\overline{\sigma_i} \cdot x_2, & |i-j| > 1. \\ \qquad\quad \mathsf{step}\,\mathsf{transitivity}\big(y, \overline{\sigma_i\sigma_j}, \sigma_j\overline{\sigma_i}\big)\big\rangle\Big) & \end{cases}$$

(Non-recursive case) Otherwise,

$$\mathsf{rev}^+_{a,b}(x, y) = (x, y).$$

Verifying the domain and range for the different cases in the definition of $\mathsf{rev}^+_{a,b}$ is easily done.

For our formalization, we write a simple recursive function, `FindOpenPair`, to find the first occurrence of a substring of the form $\overline{\sigma_i}\,\sigma_j$. However, we cannot use the existing `List.Infix` infrastructure in Mathlib as it is non-data-carrying. We thus develop a small repository of data-carrying proofs about basic List properties.

Next, we must show termination of $\mathsf{rev}^+_{a,b}$. We first define a metric on elements of $T_{a,b}$ (what all that data-carrying work has been for!).

Definition 8.4. We define the family of functions $\ell_{T_{a,b}} : T_{a,b} \to \mathbb{N}$ (each often written ℓ_T when a and b can be inferred). Given $(x, y) \in T_{a,b}$, we know $x \in BW$ and we have a derivation $y = x \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} \overline{i^+(a)}\, i^+(b)$. Then we define

$$\ell_T(x, y) := \ell_B(a, b) - \ell_S(y)$$

Theorem 8.5. For any $(x, y) \in T_{a,b}$, we have $\ell_S(y) \le \ell_B(a, b)$.

Proof. Consider first the case for nonempty a, b. Since $y = \overline{i^+(a)}\, i^+(b) \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} x$, by Theorem 6.35 we know there is a partial grid $p = \langle a, b, c, m, d, \kappa\rangle$ with $x = s^{\mathbb{1}}(cmd)$ and $\ell_P(p) = \ell_S(y)$. We also know that $\ell_P(p) \le \ell_B(a, b)$ by definition of ℓ_B (Definition 7.8) and Theorem 7.14. Hence, $\ell_S(y) \le \ell_B(a, b)$ as desired.

When either of a or b is empty, we know that $\overline{i^+(a)}\, i^+(b)$ contains no open pair, and thus we have $\ell_S(y) = 0$, satisfying our desired inequality. □

Theorem 8.6. The metric ℓ_T decreases in each recursive call in $\mathsf{rev}^+_{a,b}$.

Proof. By Theorem 8.5, it suffices to show that $\ell_S(y)$ increases in each recursive call. This is immediate, as each recursive call generates a new derivation using step transitivity, which increases $\ell_S(y)$ by the size of the relation used, and $\ell_{R_{\mathsf{rev}}}$ is always 1. Hence, in each recursive call, $\ell_S(y)$ increases by 1, and thus $\ell_B(a, b) - \ell_S(y)$ decreases by 1. □

As ℓ_T is a natural-number metric, it is well-founded, and hence our iterative rewriting function rev^+ terminates. We finally write a wrapper function which will be used to solve the word problem in the monoid.

Definition 8.7. We define simple reversing of a monoid pair $r^+ : BW^+ \to BW^+ \to BW$ by

$$r^+(a,b) = \mathsf{rev}^+_{a,b}\left(\overline{a}\,b, \left\langle \overline{a}\,b \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} \overline{a}\,b, \mathsf{refl} \right\rangle\right)$$

We define a monoid solver $\mathsf{solve}^+ : (BW^+)^2 \to \mathsf{Bool}$ by $\mathsf{solve}^+(a,b) = \mathsf{true}$ iff $r^+(a,b) = \varepsilon$.

We amass a number of theorems about r^+ in order to show correctness of solve^+.

Lemma 8.8. $\overline{a}\,b \xrightarrow[R_{\mathrm{rev}}]{*} r^+(a,b)$ holds immediately.

We next show that the output of $r^+(a,b)$ is in positive-negative form.

Lemma 8.9. For $a, b \in BW^+$, $r^+(a,b)$ contains no open pair (substring of the form $\overline{\sigma_i}\sigma_j$).

Proof. We show that for any $x \in BW$, $y = \langle \overline{a}b \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} x, \kappa\rangle$ we must have that $\mathsf{rev}^+_{a,b}(x,y)$ contains no open pair. We proceed by induction on $(x,y) \in T_{a,b}$ using strong induction on $\ell_{T_{a,b}}$. We then split into the four cases in the definition of $\mathsf{rev}^+_{a,b}$. Let's look at one illustrative case. Say we have $x = x_1 \cdot \overline{\sigma_i}\sigma_j \cdot x_2$ with $|i-j| > 1$, and thus $\mathsf{rev}^+_{a,b}(x,y) = \mathsf{rev}^+_{a,b}(x_1 \cdot \sigma_j\overline{\sigma_i} \cdot x_2, \langle \overline{i^+(a)}\, i^+(b) \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} x_1 \cdot \sigma_j\overline{\sigma_i} \cdot x_2, \mathsf{step}\,\mathsf{trans}\big(y, \overline{\sigma_i\sigma_j}, \sigma_j\overline{\sigma_i}\big)\rangle)$.

Note that $\ell_{T_{a,b}}(x,y) = 1 + \ell_{T_{a,b}}((x_1 \cdot \sigma_j\overline{\sigma_i} \cdot x_2, \langle \overline{i^+(a)}\, i^+(b) \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} x_1 \cdot \sigma_j\overline{\sigma_i} \cdot x_2, \mathsf{step}\,\mathsf{trans}\big(y, \overline{\sigma_i\sigma_j}, \sigma_j\overline{\sigma_i}\big)\rangle))$, and so we apply our inductive hypothesis to conclude that $\mathsf{rev}^+_{a,b}(x_1 \cdot \sigma_j\overline{\sigma_i} \cdot x_2, \langle \overline{i^+(a)}\, i^+(b) \underset{R_{\mathsf{rev}}}{\twoheadrightarrow} x_1 \cdot \sigma_j\overline{\sigma_i} \cdot x_2, \mathsf{step}\,\mathsf{trans}\big(y, \overline{\sigma_i\sigma_j}, \sigma_j\overline{\sigma_i}\big)\rangle)$, and thus $\mathsf{rev}^+_{a,b}(x,y)$ contains no open pair. Finally, we conclude that $r^+(a,b)$ contains no open pair as desired. □

Lemma 8.10. If a word $w \in BW$ contains no open pairs, then it is in positive-negative form – that is, there exist $c, d \in BW^+$ such that $w = c\overline{d}$. (Proof by induction on w.)

Finally, we put together the two previous lemmas to obtain our desired theorem :

Theorem 8.11. For any $a, b \in BW^+$, there exist $c, d \in BW^+$ such that $r^+(a,b) = c\overline{d}$.

We can now prove a correctness theorem for our monoid solver:

Theorem 8.12. $\mathsf{solve}^+(a,b) = \mathsf{true}$ if and only if $a \equiv^+ b$

Proof. Forward direction: Assume $\mathsf{solve}^+(a,b) = \mathsf{true}$, so $r^+(a,b) = \varepsilon$. Then by Lemma 8.8, $\overline{a}b \xrightarrow[R_{\mathrm{rev}}]{*} \varepsilon$. Thus by Theorem 2.23, $\overline{a}b \equiv \varepsilon$, so $a \equiv b$ and by Ore localization 3.14, $a \equiv^+ b$

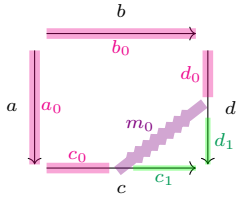
Backward direction: Assume $a \equiv^+ b$. Then by Theorem 5.13, there is a grid with source (a,b) and target $(\varepsilon,\varepsilon)$. Further, by Theorem 8.11 we know that for some $c, d \in BW^+$, $r^+(a,b) = c\overline{d}$. Hence by Lemma 8.8, $\overline{a}b \xrightarrow[R_{\mathrm{rev}}]{*} c\overline{d}$. By Theorem 6.3, there is a grid with source (a,b) and target (c,d). By uniqueness of grid sinks (Theorem 5.9), $c = \varepsilon$ and $d = \varepsilon$. Thus $r^+(a,b) = \varepsilon$ and $\mathsf{solve}^+(a,b) = \mathsf{true}$. □

In fact, for the forward direction, there is a method which does not use Ore's theorem, but rather works through grids. Since $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} \varepsilon\,(= \varepsilon\overline{\varepsilon})$, by Theorem 6.3, there is a grid from (a, b) to $(\varepsilon, \varepsilon)$, and hence by Theorem 5.4 $a \equiv^{+} b$.

We also have an alternative proof for the backward direction, which uses a sort of restricted confluence for Semi-Thue systems over R_{rev}. We begin with two theorems about partial grids "nested" inside grids which share essentially the same spine (up to deletion of $\mathbb{1}$ and $\overline{\mathbb{1}}$), which will aid in the confluence proof :

Theorem 8.13. Let $g = \langle a, b, c, d\rangle \in G$ and $p = \langle a_0, b_0, c_0, m_0, d_0\rangle \in G_P$. Recall that $s^{\mathbb{1}}$ deletes all instances of $\mathbb{1}$ or $\overline{\mathbb{1}}$ from a braid word. If $a = s^{\mathbb{1},+}(\overline{a_0})$, $b = s^{\mathbb{1},+}(b_0)$, and there exists $c_1, d_1 \in BW^{+}$ such that $c = s^{\mathbb{1},+}(c_0) \cdot c_1$ and $d = s^{\mathbb{1},+}(\overline{d_0}) \cdot d_1$, then we have $s^{\mathbb{1}}(m_0) \xrightarrow[R_{\text{rev}}]{*} c_1 \cdot \overline{d_1}$.

Here is the image to keep in mind – we claim, roughly, that the purple section rewrites under R_{rev} to the green section, both read from the bottom-left to the top-right :



The proof follows the strategy of Theorem 7.14, inducting on the grid and proceeding by extensive casework in each inductive case, splitting the partial grid as appropriate, and using prefix-suffix fact (1) from Theorem 7.14 to help apply the inductive hypothesis.

From the above theorem, we can immediately deduce our second needed theorem, that the middle frontier of a partial grid reverses to the sink of the grid in which it is nested :

Theorem 8.14. Let $g = \langle a, b, c, d\rangle \in G$ and $p = \langle a_0, b_0, c_0, m_0, d_0\rangle \in G_P$. If $a = s^{\mathbb{1},+}(\overline{a_0})$ and $b = s^{\mathbb{1},+}(b_0)$, then we have $s^{\mathbb{1}}(c_0 m_0 d_0) \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$.

Now we are prepared to state and prove restricted confluence.

Theorem 8.15. Let $a, b \in BW^{+}$ and $w_1, w_2 \in BW$. If $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} w_1$ and $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} w_2$, then there exists some $w \in BW$ such that $w_1 \xrightarrow[R_{\text{rev}}]{*} w$ and $w_2 \xrightarrow[R_{\text{rev}}]{*} w$.

Proof. First consider the case of nonempty a and b. We know there are $c, d \in BW^{+}$ such that $\langle a, b, c, d\rangle \in G$, by existence of grids (Theorem 5.16). Let $w = c\overline{d}$. Since $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} w_1$, there is a partial grid with spine (a, b) and total frontier t_1 such that $s^{\mathbb{1}} = w_1$, so by Theorem 8.14, $s^{\mathbb{1}}(t_1) \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$, and thus $w_1 \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$ as desired. We show that $w_2 \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$ in exactly the same manner.

In the case where either a or b is empty, $\overline{a}b$ contains no open pairs, so $\overline{a}b = w_1$ and $\overline{a}b = w_2$. Thus, we may use w_1 for w, and both desired derivations are instances of reflexivity. □

Now we can return to our second proof of the reverse direction of the monoid solver's correctness from Theorem 8.12. Assume that $a \equiv^{+} b$. Then by Theorem 5.13, there is a grid with source (a, b) and target $(\varepsilon, \varepsilon)$. Thus, by the forward direction of 6.3, $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} \varepsilon$. Since $\overline{a}b \xrightarrow[R_{\text{rev}}]{*} r^{+}(a, b)$, by Theorem 8.15, there exists some $c \in BW$ such that $\varepsilon \xrightarrow[R_{\text{rev}}]{*} c$ (and thus $c = \varepsilon$) and $r^{+}(a, b) \xrightarrow[R_{\text{rev}}]{*} c$. Since $r^{+}(a, b)$ is in positive-negative form, we must have $r^{+}(a, b) = c = \varepsilon$ as desired.

This second proof is not very different from the first, but substitutes one instance of grids for restricted confluence.

We thus have created both a solver for the braid monoid (solve^+), as well as a method for reversing any word in negative-positive form (rev^+).

8.2 Group Solver

We now work to implement a solver for the word problem in the braid group. This again largely relies on a method to iteratively apply reversing rewrites to a braid word, akin to rev^+. However, we may no longer restrict the input word to one in negative-positive form. We will still need to restrict the codomain of this new reversing function.

Definition 8.16. We define the set of braid words in positive-negative form as BW^{PN} – that is, the set of all $w \in BW$ of the form $c\overline{d}$ for $c, d \in BW^+$.

Definition 8.17. We define the reversing function $\mathsf{rev} : BW \to BW^{PN}$ as follows.[31] Consider some $w \in BW$. We know that there must exist some $u_0, v_0 \in BW^+$, not both the empty string, and some $w' \in BW$ such that $w = \overline{u_0} v_0 w'$.[32] Note that w' has length strictly less than w.

Let $c, d \in BW^+$ be such that $r^+(u_0, v_0) = c\overline{d}$. Let our recursive call $\mathsf{rev}(w')$ return $e\overline{f}$. Then define $\mathsf{rev}(w) = c \cdot r^+(d, e) \cdot \overline{f}$. Note that this is indeed in positive-negative form.

We can show that rev preserves equivalence via induction and appeals to the behavior of r^+ :

Theorem 8.18. For any $w \in BW$, we have $w \xrightarrow[R_{\mathrm{rev}}]{*} \mathsf{rev}(w)$.

For our Lean formalization, it is easier (but not strictly necessary) to bundle this fact into the definition of rev. Thus, we define the codomain of rev as follows :

```
structure ReverseResult (L : List ( × Bool)) where
  result : List ( × Bool)
  ordered : SignedList.PosNegData result
  derivation : SemiThueData reversing L result
```

That is, the codomain is the set of braid words (`result`) in positive-negative order (`ordered`) such that $w \xrightarrow[R_{\mathrm{rev}}]{*} \mathsf{rev}(w)$ (`derivation`).

With reversing for words in BW defined, we may move on to the word problem in the braid group. Our group solver will be a bit more complex than our monoid solver – it does not suffice to check if the output of rev is the empty string.

Definition 8.19. We define a procedure $\mathsf{solve} : BW^2 \to \mathsf{Bool}$ for determining equality in the braid group. Given $w_1, w_2 \in BW$, let $\mathsf{rev}(\overline{w_1} w_2) = c\overline{d}$. Then, $\mathsf{solve}(w_1, w_2) = \mathsf{true}$ iff $\mathsf{solve}^+(d, c) = \mathsf{true}$ (that is, iff $r^+(d, c) = \varepsilon$).

Next we show correctness of this procedure.

Theorem 8.20. For any $w_1, w_2 \in BW$, $\mathrm{solve}(w_1, w_2) = \mathsf{true}$ if and only if $w_1 \equiv w_2$.

Proof. Forward direction : Assume $\mathsf{solve}(w_1, w_2) = \mathsf{true}$. Letting $\mathsf{rev}(\overline{w_1} w_2) = c\overline{d}$, we know we have $\mathsf{solve}^+(d, c) = \mathsf{true}$. By correctness of solve^+, we know that $c \equiv^+ d$. Thus $c \equiv d$ and $c\overline{d} \equiv \varepsilon$.

[31] Here, unlike in our definition of rev^+, we do not restrict the initial input in any manner, so we will not need any wrapper function akin to r^+.

[32] In Lean, this is implemented as `separate_first_pair`

Returning to the fact that $\mathsf{rev}(\overline{w_1}w_2) = c\overline{d}$, we know by Theorems 8.18 and 6.2 that $\overline{w_1}w_2 \equiv c\overline{d}$, and so $\overline{w_1}w_2 \equiv \varepsilon$ as desired.

Reverse direction : Assume $w_1 \equiv w_2$. Then $\overline{w_1}w_2 \equiv 1$. Let $\mathsf{rev}(\overline{w_1}w_2) = c\overline{d}$. Then we have $\overline{w_1}w_2 \equiv c\overline{d}$ again by Theorems 8.18 and 6.2, and so $c\overline{d} \equiv 1$. Thus, $c \equiv d$, and so by Ore localization, $c \equiv^+ d$. By correctness of our monoid solver, we know $\mathsf{solve}^+(d, c) = \mathsf{true}$. As $\mathrm{solve}(w_1, w_2) = \mathsf{solve}^+(d, c)$, we have our desired conclusion. □

Et voila! Thus, we have a terminating and correct algorithm to solve the braid word problem! Given two words in the braid group, we can run `braid_group_word_solver`. For example,

```
#eval braid_word_solver [(4, false), (1, true), (2, true), (1, true)]
  [(2, true), (1, true), (2, true), (4, false)]
```

returns true, as the two words represent equal elements in the group.

Or given two elements in the group B, we can use two applications of `Quotient.exists_rep` to build `braid_solver`. For example, we can run

```
#eval braid_solver (i 1 * i 2 * i 1) (i 2 * i 1 * i 2 * (i 3)⁻¹* (i 1 * i 3)⁻¹)
```

This comes with the specification that

```
theorem braid_solver_correct {a b : BraidGroupInf} : braid_solver a b  a = b
```

Results have been instantaneous using `#eval` on small cases (approximately .33 seconds using the `#time` command). Comparing two elements of B_8 each with length 500 took about 17 seconds. The runtime of the code could certainly be improved. A number of constructive definitions carry unnecessary data – data that makes proofs easier (by showing as an object is defined that a given data-carrying property holds, instead of later proving the property by structural induction), but is technically unnecessary. While `Prop`-valued properties are ignored by the compiler, data-carrying properties, such as `SignedList.PosNegData` or `SemiThueData` are computed, and so slow down the calculation. The main contribution of this project is the mathematical theory, and so we have not spent significant time attempting to speed up code runtime (especially as it would come at the cost of much more work to set up the proofs).

8.3 Finite Braid Group Solver

We now return at last to the case of the finite braid groups. Note that we have been able to avoid the work of implementing Ore localization and grids for finite braid groups – we haven't even shown simple monoid properties like common multiples nor cancellativity.

The solver is incredibly simple : we cast the finite braid group words to infinite braid group words, and run our braid group solver. In this section, unlike Section 2.6, we are careful to keep track of the following casts:

Definition 8.21. Let i_n be the inclusion map from BW_n to BW. Let s_n map appropriately-bounded elements of BW to BW_n. (By abuse of notation, we also let both of these functions be used with BW^+ and BW_n^+ in place of BW and BW_n respectively).

Definition 8.22. We define the finite braid group word solver as $\mathsf{solve}_n(a, b) := \mathsf{solve}\big(i_n(a), i_n(b)\big)$ for $a, b \in BW_n$.

Termination is thus immediate. Correctness requires one main lemma about reversing derivations, shown by induction :

Lemma 8.23. Given $c \in BW$, if the index of the largest generator occurring in c is strictly less than $n-1$, and $c \xrightarrow[R_{\text{rev}}]{*} d$, then the index of the largest generator occurring in d is also strictly less than $n-1$, and moreover, $s_n(c) \equiv_n s_n(d)$.

Theorem 8.24. Given $a, b \in BW_n$, $\mathsf{solve_n}(a,b) = \mathsf{true}$ iff $a \equiv_n b$.

Proof. Forward direction : Assume $\mathsf{solve_n}(a,b) = \mathsf{true}$. Then $\mathsf{solve}\big(i_n(a), i_n(b)\big) = \mathsf{true}$. Letting $\mathsf{rev}\big(\overline{(i_n(a)}, i_n(b)\big) = c\overline{d}$, we know we have $\mathsf{solve}^+(c,d) = \mathsf{true}$. By correctness of solve^+, we know we have $c \equiv^+ d$. Returning to the fact that $\mathsf{rev}\big(\overline{(i_n(a)}, i_n(b)\big) = c\overline{d}$, we know that $\overline{(i_n(a)}, i_n(b) \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$. We know that the index of the largest generator in $\overline{(i_n(a)}, i_n(b)$ must be strictly less than $n-1$, so by Lemma 8.23, the same holds for the word $c\overline{d}$, and hence c and d themselves. Thus Theorem 2.32 allows us to conclude $s_n(c) \equiv_n^+ s_n(d)$ from $c \equiv d$. We obtain $s_n(c\overline{d}) \equiv \varepsilon$. By the second part of Lemma 8.23, we know from $\overline{(i_n(a)}, i_n(b) \xrightarrow[R_{\text{rev}}]{*} c\overline{d}$ that $s_n(\overline{(i_n(a)}, i_n(b)) \equiv_n s_n(c\overline{d})$. As $s_n(c\overline{d}) \equiv \varepsilon$, we have $\overline{a}b \equiv_n \varepsilon$, so we have $a \equiv_n b$ as desired.

Reverse direction : Assume $a \equiv_n b$. Then $i_n(a) \equiv i_n(b)$ and so $\mathsf{solve}(i_n(a), i_n(b)) = \mathsf{true}$. Hence $\mathsf{solve_n}(a,b) = \mathsf{true}$ as desired. □

As a consequence of the correctness of this algorithm, we may now prove one elusive fact from the end of Section 2.6 :

Theorem 8.25. If $a, b \in BW$ have no generator with index $n-1$ or larger, and $a \equiv b$, then $s_n(a) \equiv_n s_n(b)$.

Proof. Assume $a \equiv b$. By correctness of solve, we know $\mathsf{solve}(a,b) = \mathsf{true}$. We know $a = i_n(s_n(a))$ and the analogous property holds for b, so $\mathsf{solve}(i_n(s_n(a)), i_n(s_n(b))) = \mathsf{true}$. By correctness of $\mathsf{solve_n}$, we have $s_n(a) \equiv_n s_n(b)$ as desired. □

9 Future Work

The most tempting work to be completed is a proof of Conjecture 7.24 (any two partial grids with the same frame have the same size). On the topic of grids, another improvement would be to generalize their definition to one parametrized by a set of relations. Further, we could classify which relations lead to grid existence and/or uniqueness, following Dehornoy's many papers which provide criteria such as right complemented presentations, the cube criterion, homogeneity, etc [9, 11, 14, 17]. Such work could eventually lead to a generalization of the word problem solvers presented here to Garside groups and others.

There are also other algorithms to solve the word problem which rely on the results developed in this project. For example, Garside's method of universal denominators (as mentioned in the introduction) could now be implemented fairly easily. We have also completed initial work necessary for more complex, popular, and efficient-in-practice methods such as Dehornoy's handle reduction. A formalization of this algorithm could also be useful to other projects, such as Hurtig's generalization of braid group for machine knitting, which uses the normal form from handle reduction as part of his algorithm to determine equivalence in a certain braided monoidal category [23]. (There is a plan to complete a Lean formalization of the mathematics in this paper).[33] Other projects are working in the same vein of category-theoretic braids, such as Marco's formalization project.[34] It would thus be

[33] https://github.com/nhurtig/lean-category
[34] https://marcodavid.net/ainfinity/

potentially useful to identify the braid group with its category-theoretic generalization, as discussed in the introduction.

A larger goal is to formalize topological definitions of the braid group (for example, one under homotopy equivalence and another under ambient isotopy), and prove that the algebraic definition matches the topological ones. (There is some emerging, unpublished activity from AI theorem provers on a related problem using the configuration-space definition of braid groups.)

Beyond formalization of proofs, I am also working on a Lean widget to visualize braids. Given a formalized algorithm, the widget can show a braid transforming as the algorithm is applied to it (such as building its fraction form). In the reverse order, moving from imagery to Lean code, the strings in the widget will be able to be clicked and dragged in order to generate proofs.

10 Acknowledgements

I would like to thank Jeremy Avigad for his thoughtful feedback and careful edits throughout the project. I am grateful to Patrick Massot for his advice on the formalization of Ore localization for presented monoids. I thank Jim McCann, Jenny Lin, and Nat Hurtig for their brainstorming contributions concerning the definitions of partial grids, and Bhavik Mehta for his enthusiastic, spirited discussions of alternative definitions of partial grids, which led to the inclusion of Section **??**. I also thank the Mathlib community (especially Yaël Dillies, Johann Commelin, and numerous other maintainers) for their suggestions and assistance, particularly regarding the definitions of presented monoids and braid groups. Lastly, this project has been greatly bolstered by the congenial atmosphere of the Hoskinson Center for Formal Mathematics built by all the students and visitors who have gathered together there to talk Lean and support one another.

This work was supported in part by NSF Grant DMS-2434614 and DARPA ExpMath Grant HR0011262E028.

References

[1] E. Artin. "Theory of Braids". en. In: *The Annals of Mathematics* 48.1 (Jan. 1947), p. 101. ISSN: 0003486X. DOI: 10.2307/1969218. URL: https://www.jstor.org/stable/1969218?origin=crossref.

[2] Emil Artin. "Theorie der Zöpfe". de. In: *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg* 4.1 (Dec. 1925), pp. 47–72. ISSN: 1865-8784. DOI: 10.1007/BF02950718. URL: https://doi.org/10.1007/BF02950718.

[3] Stephen Bigelow. "Braid groups are linear". en. In: *Journal of the American Mathematical Society* 14.2 (Dec. 2000), pp. 471–486. ISSN: 1088-6834, 0894-0347. DOI: 10.1090/S0894-0347-00-00361-1. URL: https://www.ams.org/jams/2001-14-02/S0894-0347-00-00361-1/.

[4] Joan Birman, Ki Hyoung Ko, and Sang Jin Lee. "A New Approach to the Word and Conjugacy Problems in the Braid Groups". en. In: *Advances in Mathematics* 139.2 (Nov. 1998), pp. 322–353. ISSN: 00018708. DOI: 10.1006/aima.1998.1761. URL: https://linkinghub.elsevier.com/retrieve/pii/S0001870898917613.

[5] William W. Boone. "The Word Problem". In: *The Annals of Mathematics* 70.2 (Sept. 1959), p. 207. ISSN: 0003486X. DOI: 10.2307/1970103. URL: https://www.jstor.org/stable/1970103?origin=crossref.

[6] Robin Carlier. *Unbiasing symmetric monoidal categories in Lean*. Version Number: 1. 2026. DOI: 10.48550/ARXIV.2603.00896. URL: https://arxiv.org/abs/2603.00896.

[7] Reinis Cirpons et al. "Certifying the Decidability of the Word Problem in Monoids at Large". en. In: *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs*. Rennes France: ACM, Jan. 2026, pp. 128–142. ISBN: 979-8-4007-2341-4. DOI: 10.1145/3779031.3779101. URL: https://dl.acm.org/doi/10.1145/3779031.3779101.

[8] Leonardo De Moura and Sebastian Ullrich. "The Lean 4 Theorem Prover and Programming Language". en. In: *Automated Deduction – CADE 28*. Ed. by André Platzer and Geoff Sutcliffe. Vol. 12699. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2021, pp. 625–635. ISBN: 978-3-030-79875-8 978-3-030-79876-5. DOI: 10.1007/978-3-030-79876-5_37. URL: https://link.springer.com/10.1007/978-3-030-79876-5_37.

[9] Patrick Dehornoy. *A cancellativity criterion for presented monoids*. arXiv:1802.04607. Sept. 2018. URL: http://arxiv.org/abs/1802.04607.

[10] Patrick Dehornoy. "Braid-based cryptography". en. In: *Contemporary Mathematics*. Ed. by Alexei G. Myasnikov and Vladimir Shpilrain. Vol. 360. Providence, Rhode Island: American Mathematical Society, 2004, pp. 5–33. ISBN: 978-0-8218-3444-2 978-0-8218-5695-6 978-0-8218-7950-4. DOI: 10.1090/conm/360/06566. URL: https://www.ams.org/conm/360.

[11] Patrick Dehornoy. "Complete positive group presentations". en. In: *Journal of Algebra* 268.1 (Oct. 2003), pp. 156–197. ISSN: 00218693. DOI: 10.1016/S0021-8693(03)00326-0. URL: https://linkinghub.elsevier.com/retrieve/pii/S0021869303003260.

[12] Patrick Dehornoy. "Deux propriétés des groupes de tresses". In: *Comptes rendus de l'Académie des sciences*. 1st ser. 315.6 (1992), pp. 633–638.

[13] Patrick Dehornoy. "Efficient solutions to the braid isotopy problem". en. In: *Discrete Applied Mathematics* 156.16 (Sept. 2008), pp. 3091–3112. ISSN: 0166218X. DOI: 10.1016/j.dam.2007.12.009. URL: https://linkinghub.elsevier.com/retrieve/pii/S0166218X08000437.

[14] Patrick Dehornoy. "On completeness of word reversing". en. In: *Discrete Mathematics* 225.1-3 (Oct. 2000), pp. 93–119. ISSN: 0012365X. DOI: 10.1016/S0012-365X(00)00149-7. URL: https://linkinghub.elsevier.com/retrieve/pii/S0012365X00001497.

[15] Patrick Dehornoy. "The Subword Reversing Method". en. In: *International Journal of Algebra and Computation* 21.01n02 (Feb. 2011), pp. 71–118. ISSN: 0218-1967, 1793-6500. DOI: 10.1142/S0218196711006091. URL: https://www.worldscientific.com/doi/abs/10.1142/S0218196711006091.

[16] Patrick Dehornoy, Danièle Gibbons, and Greg Gibbons. *The Calculus of Braids: an introduction, and beyond*. London Mathematical Society student texts 100. Cambridge ; New York, NY: Cambridge University Press, 2022. ISBN: 978-1-108-84394-2.

[17] Patrick Dehornoy and Emilie Tesson. "Garside combinatorics for Thompson's monoid F^+ and a hybrid with the braid monoid B_∞^+". In: *Algebraic Combinatorics* 2.4 (2019), pp. 683–709. DOI: 10.5802/alco.52.

[18] Patrick Dehornoy and Bert Wiest. "On Word Reversing in Braid Groups". en. In: *International Journal of Algebra and Computation* 16.05 (Oct. 2006), pp. 941–957. ISSN: 0218-1967, 1793-6500. DOI: 10.1142/S021819670600327X. URL: https://www.worldscientific.com/doi/abs/10.1142/S021819670600327X.

[19] Antonin Delpeuch and Jamie Vicary. "The Word Problem for Braided Monoidal Categories is Unknot-Hard". en. In: *Electronic Proceedings in Theoretical Computer Science* 372 (Nov. 2022), pp. 72–87. ISSN: 2075-2180. DOI: 10.4204/EPTCS.372.6. URL: http://arxiv.org/abs/2105.04237v2.

[20] I A Dynnikov. "On a Yang-Baxter map and the Dehornoy ordering". In: *Russian Mathematical Surveys* 57.3 (June 2002), pp. 592–594. ISSN: 0036-0279, 1468-4829. DOI: 10.1070/RM2002v057n03ABEH000519. URL: https://www.mathnet.ru/eng/rm519.

[21] F. A. Garside. "The Braid Group and Other Groups". en. In: *The Quarterly Journal of Mathematics* 20.1 (1969), pp. 235–254. ISSN: 0033-5606, 1464-3847. DOI: 10.1093/qmath/20.1.235. URL: https://academic.oup.com/qjmath/article-lookup/doi/10.1093/qmath/20.1.235.

[22] Juan González-Meneses and Marithania Silvero. "Polynomial braid combing". en. In: *Mathematics of Computation* 88.318 (Nov. 2018), pp. 2027–2045. ISSN: 1088-6842, 0025-5718. DOI: 10.1090/mcom/3392. URL: https://www.ams.org/mcom/2019-88-318/S0025-5718-2018-03392-0/.

[23] Nathan Hurtig et al. "Polynomial-Time Program Equivalence for Machine Knitting". en. In: *Proceedings of the ACM on Programming Languages* 9.ICFP (Aug. 2025), pp. 371–399. ISSN: 2475-1421. DOI: 10.1145/3747517. URL: https://dl.acm.org/doi/10.1145/3747517.

[24] A. Joyal and R. Street. "Braided Tensor Categories". en. In: *Advances in Mathematics* 102.1 (Nov. 1993), pp. 20–78. ISSN: 00018708. DOI: 10.1006/aima.1993.1055. URL: https://linkinghub.elsevier.com/retrieve/pii/S0001870883710558.

[25] Egbert R. van Kampen. "On Some Lemmas in the Theory of Groups". In: *American Journal of Mathematics* 55.1/4 (1933), p. 268. ISSN: 00029327. DOI: 10.2307/2371129. URL: https://www.jstor.org/stable/2371129?origin=crossref.

[26] Daan Krammer. "Braid Groups are Linear". In: *The Annals of Mathematics* 155.1 (Jan. 2002), p. 131. ISSN: 0003486X. DOI: 10.2307/3062152. URL: https://www.jstor.org/stable/3062152?origin=crossref.

[27] Eonkyung Lee. "Braid Groups in Cryptology". In: *IEICE TRANSACTIONS on Fundamentals* E87-A.5 (2004), pp. 986–992.

[28] Roger C. Lyndon and Paul E. Schupp. *Combinatorial Group Theory*. eng. Classics in Mathematics 89. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001. ISBN: 978-3-540-41158-1 978-3-642-61896-3. DOI: 10.1007/978-3-642-61896-3.

[29] Jean Michel. *The development version of the CHEVIE package of GAP3*. arXiv:1310.7905 [math]. Oct. 2013. DOI: 10.48550/arXiv.1310.7905. URL: http://arxiv.org/abs/1310.7905.

[30] Pyotr Sergeyevich Novikov. "On the Algorithmic Unsolvability of the Word Problem in Group Theory". In: *Trudy Matematicheskogo Instituta imeni V. A. Steklova* 44 (1955).

[31] Damien Pous. "String Diagrams for Monoidal Categories, in Rocq". en. In: *LIPIcs, Volume 382, ITP 2026* 382 (2026). Ed. by Ekaterina Komendantskaya and Tobias Nipkow. Artwork Size: 20 pages, 810898 bytes ISBN: 9783959774369 Medium: application/pdf, 28:1–28:20. ISSN: 1868-8969. DOI: 10.4230/LIPICS.ITP.2026.28. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2026.28.

[32] T. V. H. Prathamesh. "Formalising Knot Theory in Isabelle/HOL". In: *Interactive Theorem Proving*. Ed. by Christian Urban and Xingyuan Zhang. Cham: Springer International Publishing, 2015, pp. 438–452. ISBN: 978-3-319-22102-1. DOI: 10.1007/978-3-319-22102-1_29.

[33] Bhakti Shah et al. *ViCAR: Visualizing Categories with Automated Rewriting in Coq*. Version Number: 2. 2024. DOI: 10.48550/ARXIV.2404.08163. URL: https://arxiv.org/abs/2404.08163.

[34] The Mathlib Community. "The lean mathematical library". en. In: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. New Orleans LA USA: ACM, Jan. 2020, pp. 367–381. ISBN: 978-1-4503-7097-4. DOI: 10.1145/3372885.3373824. URL: https://dl.acm.org/doi/10.1145/3372885.3373824.

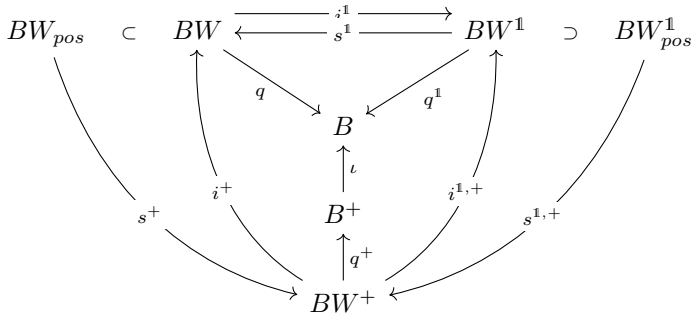
[35] Jean-Luc Thiffeault and Marko Budisic. *Braidlab: A Software Package for Braids and Loops*. arXiv:1410.0849 [math]. Nov. 2019. DOI: 10.48550/arXiv.1410.0849. URL: http://arxiv.org/abs/1410.0849.

[36] William Thurston. "Braid Groups". In: *Word processing in groups*. Boston: Jones and Bartlett Publishers, 1992. ISBN: 978-0-86720-244-1.

[37] Nicholas Touikan. *van Kampen diagrams and the geometry of the word problem*. Sept. 2024. URL: https://ntouikan.ext.unb.ca/MATH6022/IntroCGGT/html_output/section-14.html.

A Maps

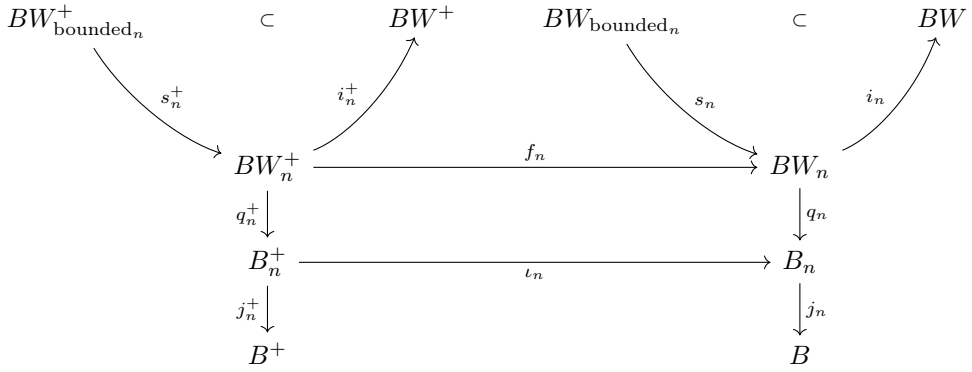
Here, we summarize the many maps used between braid structures in this project. First, we give a diagram of functions between the various infinite braid structures. In the paper, some are often left implicit, such as i^+, but all are carefully noted in the Lean formalization.



Map	Domain	Codomain	Lean name
q	BW	B	`BraidGroupInf.mk`
$q^{\mathbb{1}}$	$BW^{\mathbb{1}}$	B	`BraidGroupInf.mk  SignedOptionList.toSignedList`
q^+	BW^+	B^+	`BraidMonoidInf.mk`
$i^{\mathbb{1}}$	BW	$BW^{\mathbb{1}}$	`SignedList.toSignedOptionList`
$s^{\mathbb{1}}$	$BW^{\mathbb{1}}$	BW	`SignedOptionList.toSignedList`
i^+	BW^+	BW	`to_horizontal_edge_no_epsilon` (`to_vertical_edge_no_epsilon` is the composition of the overbar operator $\overline{\cdot}$ with `to_horizontal_edge_no_epsilon`)
$i^{\mathbb{1},+}$	BW^+	$BW^{\mathbb{1}}$	`to_horizontal_edge` (note that $i^{\mathbb{1},+}(\varepsilon) := \mathbb{1}$; in Lean, we have `to_horizontal_edge [] := [(none, true]`) (`to_vertical_edge` is the composition of the overbar operator $\overline{\cdot}$ with `to_horizontal_edge`)
s^+	BW_{pos}	BW^+	`SignedList.toList`
$s^{\mathbb{1},+}$	$BW^{\mathbb{1}}_{pos}$	BW^+	`SignedOptionList.toList`
ι	B^+	B	`BraidMonoidInf.toBraidGroupInf`

Note: In Lean, the two inclusions are denoted using `SignedList.is_true`.

We next consider functions between the finite braid structures, as well as maps to and from infinite braid structures. We define BW_{bounded_n} as the set of infinite braid group words for which the index of the largest generator is strictly less than $n-1$.

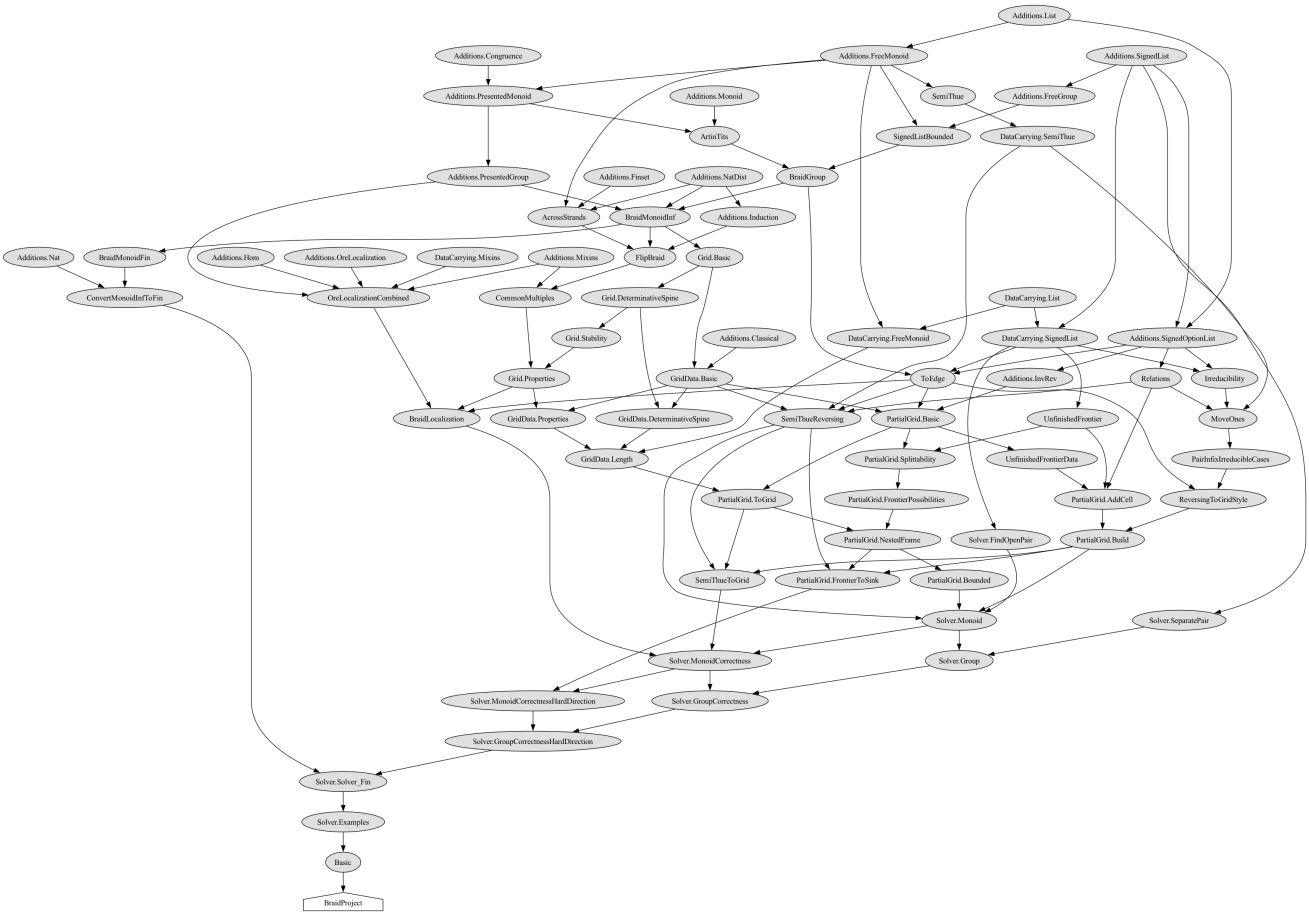


Map	Domain	Codomain	Lean name
s_n^+	$BW^+_{\text{bounded}_n}$	BW_n^+	`FreeMonoid.mapNatToFin`
s_n	BW_{bounded_n}	BW_n	`SignedList.mapNatToFin`
i_n^+	BW_n^+	BW^+	`List.map (fun x => x.1)`
i_n	BW_n	BW	`List.map (fun x => (x.1.val, x.2))`
f_n	BW_n^+	BW_n	`to_horizontal_edge_no_epsilon`
q_n^+	BW_n^+	B_n^+	`BraidMonoidFin.mk n`
q_n	BW_n	B_n	`BraidGroupFin.mk n`
ι_n	B_n^+	B_n	`BraidMonoidFin.toBraidGroupFin n`
j_n^+	B_n^+	B^+	`BraidMonoidFin.toBraidMonoidInf n`
j_n	B_n	B	`BraidGroupFin.toBraidGroupInf n`

Note: To implement the inclusion maps in Lean, we use `SignedList.bounded` to define BW_{bounded_n}, and hypotheses of the form $\forall$`x ∈ w, x < n.pred` to discuss $BW^+_{\text{bounded}_n}$.

B Dependency Graph

Here is the dependency graph for the project:



The repository currently contains about 15,000 lines of code.